

My Deep Learning

jonghwan lee

Chapter 0: Preface

- Learning Objectives

- 0.1 Who This Book Is For

- 0.2 Philosophy

- 0.3 How to Use This Book

- 0.4 Notation Conventions

- 0.5 The From-Scratch Constraint

- 0.6 Book Roadmap

- 0.7 Environment Setup

- 0.8 What's Next

Chapter 1: The Perceptron

- 1.1 Biological Inspiration

- 1.2 Mathematical Model

 - Geometric Interpretation

 - The Perceptron Learning Rule

- 1.3 The Perceptron Learning Algorithm

- 1.4 Training on Linearly Separable Data

- 1.5 Decision Boundary Animation

- 1.6 Weight Vector Evolution

- 1.7 The XOR Problem

 - Why XOR is not linearly separable

- 1.8 Logic Gates with Perceptrons

- 1.9 Exercises

 - Chapter Summary

Chapter 2: Linear Regression

- 2.1 The Problem

- 2.2 Loss Function: Mean Squared Error

- 2.3 Closed-Form Solution (Normal Equation)

- 2.4 Gradient Descent from Scratch

 - Deriving the gradients

- 2.5 Learning Rate Comparison

- 2.6 Polynomial Regression

- 2.7 Mini-batch Gradient Descent

- 2.8 Exercises

Chapter 3: Logistic Regression

- Learning Objectives

- 3.3 Binary Cross-Entropy Loss

 - Derivation from Maximum Likelihood

- 3.6 Decision Boundary Evolution

- 3.9 Exercises

 - Exercise 1: Implementing the Log-Sum-Exp Trick

- Exercise 2: Regularized Logistic Regression
- Exercise 3: Multi-class Metrics
- Chapter 4: Neural Network Foundations
 - 4.3 A Single Hidden Layer
 - 4.6 Hidden Layer Size Experiment
 - 4.9 Exercises
 - Exercise 1: Dead ReLU Exploration
 - Exercise 2: Three-Class Spiral
 - Exercise 3: Batch Normalization Preview
- Chapter 5: Backpropagation
 - 5.3 Backprop by Hand: A Tiny Example
 - Network Architecture
 - Weights
 - Summary of All Partial Derivatives
 - 5.6 Gradient Checking
 - Numerical Gradient
 - Relative Error Metric
 - 5.9 Exercises
 - Exercise 1: Extend the Tiny Example
 - Exercise 2: Implement a New Activation
 - Exercise 3: Gradient Checkpointing
 - Summary
- Chapter 6: Multi-Layer Networks
 - 6.1 Going Deep
 - Why depth?
 - Depth vs. Width
 - Building a deep network with Sequential
 - 6.2 The Training Loop
 - 6.3 Loss Functions
 - 6.4 Experiments on Synthetic Datasets
 - 6.5 Depth Experiment
 - 6.6 The Vanishing Gradient Problem
 - Reading the Heatmaps
 - 6.7 Width vs. Depth
 - 6.8 Multi-Class Classification
 - 6.9 Exercises
- Chapter 7: Optimizers
 - Learning Objectives
 - 7.1 The Optimization Landscape
 - Convex vs Non-Convex
 - 7.2 Stochastic Gradient Descent (SGD)
 - Variants by Batch Size
 - 7.3 Momentum
 - Intuition: The Ball Rolling Downhill
 - Mathematics
 - Nesterov Momentum
 - 7.4 RMSProp
 - Mathematics
 - 7.5 Adam
 - Mathematics
 - Bias Correction
 - Default Hyperparameters
 - 7.6 Learning Rate Schedules
 - 7.7 Optimizer Comparison – Loss Curves

- 7.8 Optimizer Trajectories – Hero Figure
- 7.9 Exercises
- Chapter 8: Regularization
 - Learning Objectives
 - 8.1 Overfitting
 - The Bias-Variance Tradeoff
 - 8.2 L1 and L2 Regularization
 - L2 Regularization (Weight Decay)
 - L1 Regularization (Lasso)
 - Implementation
 - 8.3 Dropout
 - Inverted Dropout
 - Implementation
 - 8.4 Early Stopping
 - Patience
 - 8.5 Data Augmentation Preview
 - 8.6 Regularization Comparison (Hero Figure)
 - What to Observe
 - 8.7 Finding Optimal Lambda
 - Key Takeaway
 - 8.8 Exercises
- Chapter 9: Batch Normalization
 - Learning Objectives
 - 9.1 Internal Covariate Shift
 - The Core Idea
 - 9.2 The Batch Normalization Transform
 - Stage 1: Normalize
 - Stage 2: Scale and Shift
 - Training vs Inference
 - 9.3 The Backward Pass – Derivation
 - 9.4 Activation Distributions – The Hero Figure
 - 9.5 Training with BatchNorm
 - 9.6 Higher Learning Rates
 - 9.7 Gradient Flow with BatchNorm
 - 9.8 Layer Normalization
 - Layer Normalization
 - 9.9 Exercises
- Chapter 10: Convolutional Neural Networks
 - 10.1 Motivation
 - The parameter explosion
 - Translation invariance
 - Spatial hierarchy
 - 10.2 The Convolution Operation
 - 1D Convolution
 - 2D Convolution
 - 10.3 Building Blocks
 - Padding
 - Stride
 - Multiple Channels
 - 10.4 The Conv2D Layer with Backprop
 - The im2col Trick
 - Forward Pass
 - Backward Pass
 - 10.5 Building a Simple ConvNet

- 10.6 Feature Map Visualization (Hero Figure)
- 10.7 Learned Filters
- 10.8 Receptive Field
 - Computing the receptive field
 - Our architecture
- 10.9 Exercises
 - Chapter Summary
- Chapter 11: Recurrent Neural Networks
 - Learning Objectives
 - 11.3 Backpropagation Through Time (BPTT)
 - The chain rule through time
 - The vanishing gradient problem
 - Truncated BPTT
 - 11.6 LSTM Gate Activations (Hero Figure)
 - 11.9 RNN vs LSTM: Long-Range Dependency
 - 11.12 Exercises
 - Exercise 1: Bidirectional RNN
 - Exercise 2: Sequence-to-Sequence Addition
 - Exercise 3: Hyperparameter Sensitivity
- Chapter 12: Advanced Topics
 - Learning Objectives
 - 12.3 Residual Connections
 - The Problem: Vanishing Gradients in Deep Networks
 - The Solution: Skip Connections
 - Implementation
 - 12.6 Modern Architecture: ResNet-style
- 13.1 The Task
- 13.2 Data Preparation
- 13.3 Model Architecture
- 13.4 Training Loop
- 13.5 Training
- 13.6 Training Dashboard (Hero Figure)
- 13.7 Results Analysis
- 13.8 Hyperparameter Search
 - Hyperparameter Search Results
- 13.9 Visualizing Learned Features
- 13.10 Misclassified Examples
- 13.11 What We Built
 - Core Layers
 - Activations
 - Normalization & Regularization
 - Architecture Patterns
 - Loss Functions
 - Optimizers
 - Learning Rate Schedulers
 - Recurrent Components
 - Utilities
- 13.12 What's Next
 - What We Skipped (and Where Frameworks Add Value)
 - Learning Roadmap
 - Recommended Resources
 - Final Words

```
import sys
sys.path.insert(0, "..")

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

from style.dl_style import apply_style, PALETTE

apply_style()
np.random.seed(42)
```

Chapter 0: Preface

Deep Learning from Scratch with NumPy

Learning Objectives

By the end of this preface, you will:

- Understand the philosophy and goals of this book
- Know who this book is written for and what prerequisites are expected
- Be familiar with the notation conventions used throughout
- See the full roadmap of chapters and how they connect
- Have your environment set up and ready to go

0.1 Who This Book Is For

This book is for anyone who wants to **truly understand** how deep learning works under the hood. You are the ideal reader if:

- **You know Python.** You are comfortable with functions, classes, list comprehensions, and NumPy basics. You don't need to be an expert, but you should be able to read and write Python without constant reference to documentation.
- **You have some calculus.** You remember (or can quickly review) derivatives, the chain rule, and partial derivatives. We will build intuition alongside the math, but a prior encounter with these concepts will make the journey smoother.
- **You want to understand DL internals.** You are not satisfied with treating neural networks as black boxes. You want to see every gradient computed, every weight updated, every forward pass executed – in plain code you can step through line by line.

Whether you are a software engineer looking to deepen your ML knowledge, a student preparing for research, or a practitioner who wants to move beyond `model.fit()`, this book meets you where you are and takes you to a place of genuine understanding.

0.2 Philosophy

| “If you can’t code it from scratch, you don’t truly understand it.”

This single idea drives every page of this book. Frameworks like PyTorch and TensorFlow are extraordinary engineering achievements, but they also hide the mechanics that make deep learning work. Automatic differentiation computes your gradients. Optimized kernels handle your matrix multiplications. Pre-built layers assemble your architectures.

There is nothing wrong with using these tools – in fact, you *should* use them for production work. But if your goal is **understanding**, then you need to build the machinery yourself, at least once.

In this book, we will:

1. **Derive** the math behind each algorithm
2. **Implement** it from scratch using only NumPy
3. **Visualize** what is happening at every step
4. **Experiment** with the code to build intuition

By the end, you will not just know *that* backpropagation works – you will know *why* it works, because you will have written it yourself.

0.3 How to Use This Book

Each chapter is a self-contained Jupyter notebook. Here is how to get the most out of them:

- **Run cells sequentially.** Every notebook is designed to be executed from top to bottom. Later cells depend on variables and functions defined in earlier ones.
- **Experiment with parameters.** When you see a learning rate, a number of epochs, or a network architecture, *change it*. See what happens. Break things. The best way to build intuition is to explore the boundaries.
- **Read the code, not just the prose.** The implementations *are* the explanations. Every line of code is there for a reason, and we use comments and docstrings liberally.
- **Do the exercises.** Each chapter ends with exercises that range from straightforward to challenging. They are not optional – they are where the deepest learning happens.
- **Consult the roadmap.** The chapters build on each other. If something in Chapter 7 feels confusing, the roadmap below will tell you which earlier chapter to revisit.

0.4 Notation Conventions

We use consistent mathematical notation throughout. Here is your reference card:

Symbol	Meaning	Example
$\backslash(x, y, \backslash\alpha\backslash)$	Scalars (italic lowercase)	learning rate $\backslash(\alpha = 0.01\backslash)$
$\backslash(\mathbf{x}, \mathbf{h}\backslash)$	Vectors (bold lowercase)	input $\backslash(\mathbf{x} \in \mathbb{R}^n\backslash)$
$\backslash(\mathbf{W}, \mathbf{X}\backslash)$	Matrices (bold uppercase)	weight matrix $\backslash(\mathbf{W} \in \mathbb{R}^{m \times n}\backslash)$
$\backslash(\frac{\partial L}{\partial \mathbf{w}}\backslash)$	Partial derivative	gradient of loss w.r.t. weight
$\backslash(\nabla_{\mathbf{W}} L\backslash)$	Gradient (matrix form)	full gradient matrix
$\backslash(\sigma(\cdot)\backslash)$	Activation function	sigmoid, ReLU, etc.
$\backslash(\hat{y}\backslash)$	Prediction	model output
$\backslash(L\backslash)$ or $\backslash(\mathcal{L}\backslash)$	Loss function	cross-entropy, MSE
$\backslash(\odot\backslash)$	Element-wise product	Hadamard product

When we move between math and code, we keep the mapping explicit:

- $\backslash(\mathbf{W}\backslash)$ in math becomes `W` in code (a 2D NumPy array)
- $\backslash(\mathbf{x}\backslash)$ in math becomes `x` in code (a 1D or 2D NumPy array)
- $\backslash(\frac{\partial L}{\partial \mathbf{W}}\backslash)$ in math becomes `dW` in code

0.5 The From-Scratch Constraint

To keep our focus on understanding, we impose a strict constraint on ourselves:

Allowed: - `numpy` – array operations, linear algebra, random numbers - `matplotlib` – visualization only - Standard library modules (`math`, `collections`, `struct`, etc.)

Not allowed: - No automatic differentiation (no `autograd`, no `torch.autograd`) - No deep learning frameworks (no `PyTorch`, no `TensorFlow`, no `JAX`) - No high-level ML libraries (no `scikit-learn` for model building)

Every gradient will be derived by hand and implemented explicitly. Every optimization step will be coded out. Every layer will be built from raw matrix operations.

This constraint is not about masochism – it is about **clarity**. When there is no magic, there is no mystery.

0.6 Book Roadmap

The following visualization shows all 13 chapters and how they depend on each other. An arrow from Chapter A to Chapter B means A is a prerequisite for B.

```
# ----- Chapter Roadmap: Directed Dependency Graph -----
```

```
chapters = {
    0: "Ch 0\nPreface",
    1: "Ch 1\nNumPy Essentials",
    2: "Ch 2\nLinear Algebra",
    3: "Ch 3\nCalculus &\nGradients",
    4: "Ch 4\nLinear\nRegression",
    5: "Ch 5\nLogistic\nRegression",
    6: "Ch 6\nNeural Network\nFundamentals",
    7: "Ch 7\nBackpropagation",
    8: "Ch 8\nOptimization",
    9: "Ch 9\nRegularization",
    10: "Ch 10\nConvolutional\nNetworks",
    11: "Ch 11\nRecurrent\nNetworks",
    12: "Ch 12\nAttention &\nTransformers",
}
```

```
# Dependency edges: (from, to)
```

```
edges = [
    (0, 1),
    (1, 2), (1, 3),
    (2, 4), (3, 4),
    (4, 5),
    (5, 6),
    (6, 7),
    (7, 8), (7, 9),
    (7, 10), (7, 11),
    (11, 12), (8, 12),
]
```

```
# --- Layout positions (x, y) for each chapter node ---
```

```
# Arranged in rows to reflect the dependency flow (top to bottom)
```

```
positions = {
    0: (5.0, 10.0),
    1: (5.0, 8.5),
    2: (3.0, 7.0),
    3: (7.0, 7.0),
    4: (5.0, 5.5),
    5: (5.0, 4.0),
    6: (5.0, 2.5),
    7: (5.0, 1.0),
    8: (2.0, -0.5),
    9: (4.5, -0.5),
    10: (7.0, -0.5),
    11: (9.0, -0.5),
    12: (5.5, -2.0),
}
```



```

}

# Color assignment: group chapters by theme
colors = {
    0: PALETTE["warm_gray"],
    1: PALETTE["steel_blue"],
    2: PALETTE["royal_blue"],
    3: PALETTE["royal_blue"],
    4: PALETTE["emerald"],
    5: PALETTE["emerald"],
    6: PALETTE["coral"],
    7: PALETTE["coral"],
    8: PALETTE["amber"],
    9: PALETTE["amber"],
    10: PALETTE["deep_blue"],
    11: PALETTE["deep_blue"],
    12: PALETTE["deep_blue"],
}

# --- Build the figure ---
fig, ax = plt.subplots(figsize=(11, 13))
ax.set_xlim(-0.5, 11.5)
ax.set_ylim(-3.5, 11.5)
ax.set_aspect("equal")
ax.axis("off")
fig.patch.set_facecolor(PALETTE["paper"])

# Title
ax.text(5.5, 11.2, "Deep Learning from Scratch -- Chapter
        Roadmap",
        ha="center", va="center", fontsize=16, fontweight="bold",
        color=PALETTE["deep_blue"])

box_w, box_h = 2.2, 1.0

# --- Draw edges (arrows) first, so boxes sit on top ---
for (src, dst) in edges:
    x0, y0 = positions[src]
    x1, y1 = positions[dst]

    # Compute direction to shorten arrow so it starts/ends at box
    # edge
    dx = x1 - x0
    dy = y1 - y0
    length = np.sqrt(dx**2 + dy**2)
    ux, uy = dx / length, dy / length

    # Offset start and end by half-box dimension along the
    # direction
    shrink_src = box_h / 2 if abs(uy) > abs(ux) else box_w / 2
    shrink_dst = box_h / 2 if abs(uy) > abs(ux) else box_w / 2
    sx = x0 + ux * shrink_src * 0.95
    sy = y0 + uy * shrink_src * 0.95

```

```

ex = x1 - ux * shrink_dst * 0.95
ey = y1 - uy * shrink_dst * 0.95

arrow = FancyArrowPatch(
    (sx, sy), (ex, ey),
    arrowstyle="->",
    mutation_scale=14,
    lw=1.5,
    color=PALETTE["slate"],
    alpha=0.6,
    connectionstyle="arc3,rad=0.0",
)
ax.add_patch(arrow)

# --- Draw chapter nodes ---
for ch_id, label in chapters.items():
    x, y = positions[ch_id]
    bg_color = colors[ch_id]

    # Rounded rectangle
    box = FancyBboxPatch(
        (x - box_w / 2, y - box_h / 2), box_w, box_h,
        boxstyle="round,pad=0.12",
        facecolor=bg_color,
        edgecolor="white",
        linewidth=1.8,
        alpha=0.92,
        zorder=3,
    )
    ax.add_patch(box)

    # Chapter label
    ax.text(x, y, label, ha="center", va="center",
           fontsize=8, fontweight="bold", color="white",
           linespacing=1.15, zorder=4)

# --- Legend for color groups ---
legend_items = [
    (PALETTE["warm_gray"], "Preface"),
    (PALETTE["steel_blue"], "Foundations"),
    (PALETTE["royal_blue"], "Math Toolkit"),
    (PALETTE["emerald"], "Classical Models"),
    (PALETTE["coral"], "Neural Networks"),
    (PALETTE["amber"], "Training Techniques"),
    (PALETTE["deep_blue"], "Advanced Architectures"),
]
legend_patches = [
    mpatches.Patch(facecolor=c, edgecolor="white", label=l)
    for c, l in legend_items
]
ax.legend(handles=legend_patches, loc="lower left",
         fontsize=8, frameon=True, fancybox=True,
         framealpha=0.9, edgecolor="#CCCCCC")

```

```
plt.tight_layout()
plt.savefig("../figures/ch00_roadmap.png", dpi=200,
            facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()
```

0.7 Environment Setup

All you need is Python 3.9+ and a handful of packages. We recommend setting up a virtual environment:

```
# Clone the repository
git clone https://github.com/<your-repo>/deep-learning-from-
    scratch.git
cd deep-learning-from-scratch

# Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate    # On Windows: .venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt
```

The requirements.txt contains only:

```
numpy
matplotlib
jupyter
```

That is it. No deep learning frameworks. No GPU required. Everything runs on your laptop.

To launch the notebooks:

```
cd notebooks
jupyter notebook
```

0.8 What's Next

In **Chapter 1: NumPy Essentials**, we will build the computational foundation that everything else rests on. You will learn to think in terms of arrays and vectorized operations – the language that NumPy speaks and that makes our from-scratch implementations both readable and fast.

We will cover: - Array creation, indexing, and reshaping - Broadcasting rules and why they matter - Vectorized operations vs. Python loops - A first taste of implementing a simple algorithm (dot product, matrix multiply) from scratch

Turn the page – or rather, open the next notebook – and let's begin.

Next: *[Chapter 1 – NumPy Essentials](#)*

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
apply_style()
np.random.seed(42)
```

Chapter 1: The Perceptron

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain how the biological neuron inspired the perceptron model
2. Write down the mathematical formulation of a perceptron: $(y = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b))$
3. Implement the Perceptron Learning Algorithm from scratch in NumPy
4. Train a perceptron on linearly separable data and visualize the decision boundary
5. Understand **why** the perceptron fails on non-linearly separable problems (the XOR problem)
6. Build basic logic gates (AND, OR, NAND) using single perceptrons

1.1 Biological Inspiration

The story of artificial neural networks begins with biology. In 1943, **Warren McCulloch** and **Walter Pitts** proposed the first mathematical model of a neuron. Their *McCulloch-Pitts neuron* was a simple binary threshold unit:

- It receives binary inputs through connections (analogous to **dendrites**).
- Each connection has an associated weight (analogous to **synaptic strength**).
- The neuron sums the weighted inputs and fires (outputs 1) if the total exceeds a threshold; otherwise it stays silent (outputs 0).

In 1958, **Frank Rosenblatt** extended this idea into the **Perceptron** – a learning machine that could automatically adjust its weights from data. Rosenblatt's perceptron was physically built as an electromechanical device called the *Mark I Perceptron*, connected to a 20x20 grid of photocells for image recognition.

Biological Neuron	Perceptron Model
=====	=====
dendrites (inputs)	x ₁ --w ₁ --\
> sign() --> y	x ₂ --w ₂ ----> [sum + bias] --
synapses (weights)	x ₃ --w ₃ --/
cell body (summation)	b (bias)
axon (output)	

The key insight: **learning is weight adjustment**. By changing the weights, we change what the neuron responds to.

1.2 Mathematical Model

A perceptron is a binary linear classifier. Given an input vector $\mathbf{x} \in \mathbb{R}^n$, the perceptron computes:

$$y = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

where: $\mathbf{w} \in \mathbb{R}^n$ is the **weight vector** - $b \in \mathbb{R}$ is the **bias** (threshold) - $\text{sign}(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$

Geometric Interpretation

The equation $\mathbf{w} \cdot \mathbf{x} + b = 0$ defines a **hyperplane** in $\mathbb{R}^n$.

- In 2D, this is a **line**: $w_1 x_1 + w_2 x_2 + b = 0$
- The weight vector $\mathbf{w}$ is **perpendicular** to this line
- The bias b shifts the line away from the origin
- Points on one side are classified as $(+1)$; on the other side as (-1)

The Perceptron Learning Rule

For each misclassified sample $(\mathbf{x}_i, y_i)$:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i \quad b \leftarrow b + \eta y_i$$

where η is the **learning rate**.

Convergence Theorem (Rosenblatt, 1962): If the data is linearly separable, the perceptron learning algorithm is guaranteed to converge in a finite number of steps.

1.3 The Perceptron Learning Algorithm

```
class Perceptron:
    """A single-layer perceptron for binary classification.

    Labels are expected to be +1 or -1.
    """

    def __init__(self, n_features, lr=0.01):
        """Initialize weights to small random values and bias to zero."""
        self.w = np.random.randn(n_features) * 0.01
        self.b = 0.0
        self.lr = lr
```

```

# History tracking
self.weight_history = []
self.bias_history = []
self.accuracy_history = []
self.n_misclassified_history = []

def predict(self, X):
    """Compute  $\text{sign}(Wx + b)$  for each sample.

    Parameters
    -----
    X : ndarray of shape (n_samples, n_features)

    Returns
    -----
    ndarray of shape (n_samples,) with values in {-1, +1}
    """
    linear = X @ self.w + self.b
    return np.where(linear >= 0, 1, -1)

def fit(self, X, y, epochs=100):
    """Train the perceptron using the Perceptron Learning
    Algorithm.

    For each epoch, iterate over all samples. When a sample is
    misclassified, update weights:  $w += lr * y_i * x_i$ ,  $b +=$ 
     $lr * y_i$ .

    Parameters
    -----
    X : ndarray of shape (n_samples, n_features)
    y : ndarray of shape (n_samples,) with values in {-1, +1}
    epochs : int
    """
    n_samples = X.shape[0]

    for epoch in range(epochs):
        n_misclassified = 0

        for i in range(n_samples):
            y_pred = np.sign(X[i] @ self.w + self.b)
            if y_pred == 0:
                y_pred = -1 # treat zero as negative

            if y_pred != y[i]:
                # Update rule
                self.w += self.lr * y[i] * X[i]
                self.b += self.lr * y[i]
                n_misclassified += 1

        # Record history
        self.weight_history.append(self.w.copy())
        self.bias_history.append(self.b)

```

```

        self.n_misclassified_history.append(n_misclassified)
        accuracy = (n_samples - n_misclassified) / n_samples
        self.accuracy_history.append(accuracy)

        # Early stop if perfectly classified
        if n_misclassified == 0:
            print(f"Converged at epoch {epoch + 1} with 100%
accuracy.")
            break

    return self

print("Perceptron class defined.")

```

1.4 Training on Linearly Separable Data

Let's generate a simple 2D dataset where the two classes can be separated by a straight line, then watch the perceptron learn.

```

# --- Generate linearly separable 2D data ---
np.random.seed(42)
n_samples = 100

# Class +1: centred at (2, 2)
X_pos = np.random.randn(n_samples // 2, 2) * 0.8 + np.array([2,
2])
# Class -1: centred at (-2, -2)
X_neg = np.random.randn(n_samples // 2, 2) * 0.8 + np.array([-2,
-2])

X = np.vstack([X_pos, X_neg])
y = np.concatenate([np.ones(n_samples // 2),
-np.ones(n_samples // 2)])

# Shuffle
shuffle_idx = np.random.permutation(n_samples)
X, y = X[shuffle_idx], y[shuffle_idx]

# --- Train ---
pct = Perceptron(n_features=2, lr=0.1)
pct.fit(X, y, epochs=100)

# --- Quick accuracy report ---
y_pred = pct.predict(X)
train_acc = np.mean(y_pred == y)
print(f"Final weights: w = {pct.w}, b = {pct.b:.4f}")
print(f"Training accuracy: {train_acc:.1%}")

# --- Convergence curves ---
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

epochs_range = range(1, len(pct.accuracy_history) + 1)

```

```

# Accuracy
axes[0].plot(epochs_range, pct.accuracy_history,
             color=PALETTE["royal_blue"], marker="o",
             markersize=4)
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy")
axes[0].set_title("Training Accuracy")
axes[0].set_ylim(0, 1.05)

# Misclassifications
axes[1].plot(epochs_range, pct.n_misclassified_history,
             color=PALETTE["coral"], marker="s", markersize=4)
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Misclassified samples")
axes[1].set_title("Misclassifications per Epoch")

plt.tight_layout()
plt.savefig("../figures/01_convergence.png")
plt.show()

```

1.5 Decision Boundary Animation

Below we show how the decision boundary evolves over training. Each subplot displays the boundary at a different epoch.

```

def plot_boundary_at_epoch(ax, X, y, w, b, epoch_label):
    """Draw scatter + decision line for given weights."""
    # Scatter points
    mask_pos = y == 1
    mask_neg = y == -1
    ax.scatter(X[mask_neg, 0], X[mask_neg, 1],
               c=PALETTE["royal_blue"], edgecolors="white",
               linewidth=0.5, s=25, label="Class -1", zorder=3)
    ax.scatter(X[mask_pos, 0], X[mask_pos, 1],
               c=PALETTE["coral"], edgecolors="white",
               linewidth=0.5, s=25, label="Class +1", zorder=3)

    # Decision boundary:  $w_1x_1 + w_2x_2 + b = 0 \Rightarrow x_2 = -\frac{(w_1x_1 + b)}{w_2}$ 
    xlim = [X[:, 0].min() - 0.5, X[:, 0].max() + 0.5]
    ax.set_xlim(xlim)
    ax.set_ylim([X[:, 1].min() - 0.5, X[:, 1].max() + 0.5])

    if abs(w[1]) > 1e-8:
        x1_line = np.linspace(xlim[0], xlim[1], 200)
        x2_line = -(w[0] * x1_line + b) / w[1]
        ax.plot(x1_line, x2_line, color=PALETTE["emerald"],
                linewidth=2.5, zorder=2)

    ax.set_title(f"Epoch {epoch_label}", fontsize=11)

```



```

ax.set_xticks([])
ax.set_yticks([])

# Retrain with enough epochs to have history at desired
# checkpoints
np.random.seed(42)
pct2 = Perceptron(n_features=2, lr=0.1)
pct2.fit(X, y, epochs=100)

# Pick checkpoints (0-indexed epochs in history)
checkpoints = [1, 5, 10, 20, 50, 100]
# Clamp to actual history length
max_epoch = len(pct2.weight_history)
checkpoints = [min(e, max_epoch) for e in checkpoints]

fig, axes = plt.subplots(2, 3, figsize=(14, 9))
axes = axes.ravel()

for idx, ep in enumerate(checkpoints):
    w_ep = pct2.weight_history[ep - 1]
    b_ep = pct2.bias_history[ep - 1]
    plot_boundary_at_epoch(axes[idx], X, y, w_ep, b_ep, ep)

# Add a legend to the first subplot
axes[0].legend(loc="upper left", fontsize=8)

fig.suptitle("Decision Boundary Evolution", fontsize=15,
             fontweight="bold", color=PALETTE["deep_blue"],
             y=1.01)
plt.tight_layout()
plt.savefig("../figures/01_decision_boundary_evolution.png")
plt.show()

```

1.6 Weight Vector Evolution

Tracking how the weight vector and bias change over training gives us insight into the learning dynamics.

```

w_hist = np.array(pct2.weight_history)
b_hist = np.array(pct2.bias_history)
epochs_range = range(1, len(w_hist) + 1)

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))

# w1 over epochs
axes[0].plot(epochs_range, w_hist[:, 0],
             color=PALETTE["royal_blue"], linewidth=2)
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("$w_1$")
axes[0].set_title("Weight $w_1$")

```

```

axes[0].axhline(y=0, color=PALETTE["warm_gray"], linestyle="--",
               linewidth=0.8)

# w2 over epochs
axes[1].plot(epochs_range, w_hist[:, 1],
            color=PALETTE["coral"], linewidth=2)
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("$w_2$")
axes[1].set_title("Weight $w_2$")
axes[1].axhline(y=0, color=PALETTE["warm_gray"], linestyle="--",
               linewidth=0.8)

# Bias over epochs
axes[2].plot(epochs_range, b_hist,
            color=PALETTE["emerald"], linewidth=2)
axes[2].set_xlabel("Epoch")
axes[2].set_ylabel("$b$")
axes[2].set_title("Bias $b$")
axes[2].axhline(y=0, color=PALETTE["warm_gray"], linestyle="--",
               linewidth=0.8)

fig.suptitle("Weight Vector Evolution During Training",
            fontsize=14,
            fontweight="bold", color=PALETTE["deep_blue"],
            y=1.02)
plt.tight_layout()
plt.savefig("../figures/01_weight_evolution.png")
plt.show()

```

1.7 The XOR Problem

In 1969, **Marvin Minsky** and **Seymour Papert** published *Perceptrons*, in which they proved that a single perceptron **cannot** learn the XOR function. This result discouraged neural network research for over a decade (the first “AI winter”).

Why XOR is not linearly separable

The XOR truth table:

$\backslash(x_1)$	$\backslash(x_2)$	$\backslash(x_1 \oplus x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

No single straight line can separate the class-0 points $\backslash((0,0), (1,1)\backslash)$ from the class-1 points $\backslash((0,1), (1,0)\backslash)$. The positive examples sit on opposite corners of the unit square.

Let’s see the perceptron try – and fail.

```

# --- XOR dataset ---
X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_xor = np.array([-1, 1, 1, -1]) # -1/+1 labels

# Train perceptron on XOR
np.random.seed(42)
pct_xor = Perceptron(n_features=2, lr=0.1)
pct_xor.fit(X_xor, y_xor, epochs=200)

y_pred_xor = pct_xor.predict(X_xor)
acc = np.mean(y_pred_xor == y_xor)
print(f"XOR accuracy after 200 epochs: {acc:.1%}")
print(f"Predictions: {y_pred_xor}")
print(f"True labels: {y_xor}")

fig, ax = plt.subplots(figsize=(6, 6))

# Scatter: XOR points
mask_pos = y_xor == 1
mask_neg = y_xor == -1
ax.scatter(X_xor[mask_neg, 0], X_xor[mask_neg, 1],
           c=PALETTE["royal_blue"], s=200, edgecolors="white",
           linewidth=1.5, zorder=5, label="Class -1 (XOR=0)")
ax.scatter(X_xor[mask_pos, 0], X_xor[mask_pos, 1],
           c=PALETTE["coral"], s=200, edgecolors="white",
           linewidth=1.5, zorder=5, label="Class +1 (XOR=1)")

# Annotate each point
for i, (xi, yi) in enumerate(zip(X_xor, y_xor)):
    label_str = f"({xi[0]}, {xi[1]})\ny={'+1' if yi == 1 else '-1'}"
    ax.annotate(label_str, (xi[0], xi[1]),
               textcoords="offset points", xytext=(15, 10),
               fontsize=9, color=PALETTE["charcoal"])

# "Best" decision line from trained perceptron
w_xor = pct_xor.w
b_xor = pct_xor.b
xlim = [-0.5, 1.5]
ax.set_xlim(xlim)
ax.set_ylim([-0.5, 1.5])

if abs(w_xor[1]) > 1e-8:
    x1_line = np.linspace(xlim[0], xlim[1], 200)
    x2_line = -(w_xor[0] * x1_line + b_xor) / w_xor[1]
    ax.plot(x1_line, x2_line, color=PALETTE["emerald"],
           linewidth=2.5, linestyle="--", label="Best boundary",
           zorder=2)

# Shaded region to emphasize failure
ax.fill_between(x1_line, x2_line, 2.0, alpha=0.08,
               color=PALETTE["coral"])

```

```

ax.fill_between(x1_line, x2_line, -1.0, alpha=0.08,
                color=PALETTE["royal_blue"])

ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")
ax.set_title("XOR Problem: No Linear Boundary Can Separate These
              Classes")
ax.legend(loc="lower right", fontsize=9)
ax.set_aspect("equal")

plt.tight_layout()
plt.savefig("../figures/01_xor_problem.png")
plt.show()

```

As we can see, the perceptron's best attempt at a decision boundary inevitably misclassifies at least one point. **A single perceptron can only represent linearly separable functions.** This fundamental limitation is what motivated the development of **multi-layer** networks – which we will explore in the next chapter.

1.8 Logic Gates with Perceptrons

Despite the XOR limitation, a single perceptron can implement any **linearly separable** Boolean function. The three most important are AND, OR, and NAND. Let's build them with hand-picked weights and visualize their decision boundaries.

```

# --- Define logic gate perceptrons ---
gate_inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

gates = {
    "AND": {"w": np.array([1.0, 1.0]), "b": -1.5},
    "OR": {"w": np.array([1.0, 1.0]), "b": -0.5},
    "NAND": {"w": np.array([-1.0, -1.0]), "b": 1.5},
}

# Truth tables
expected = {
    "AND": np.array([-1, -1, -1, 1]),
    "OR": np.array([-1, 1, 1, 1]),
    "NAND": np.array([1, 1, 1, -1]),
}

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for idx, (name, params) in enumerate(gates.items()):
    ax = axes[idx]
    w, b = params["w"], params["b"]
    y_gate = expected[name]

    # Verify
    preds = np.where(gate_inputs @ w + b >= 0, 1, -1)
    assert np.all(preds == y_gate), f"{name} gate mismatch!"

```

```

# Scatter
pos = y_gate == 1
neg = y_gate == -1
ax.scatter(gate_inputs[neg, 0], gate_inputs[neg, 1],
           c=PALETTE["royal_blue"], s=180, edgecolors="white",
           linewidth=1.5, zorder=5, label="Output 0")
ax.scatter(gate_inputs[pos, 0], gate_inputs[pos, 1],
           c=PALETTE["coral"], s=180, edgecolors="white",
           linewidth=1.5, zorder=5, label="Output 1")

# Decision boundary
xlim = [-0.5, 1.5]
ax.set_xlim(xlim)
ax.set_ylim([-0.5, 1.5])

if abs(w[1]) > 1e-8:
    x1_line = np.linspace(xlim[0], xlim[1], 200)
    x2_line = -(w[0] * x1_line + b) / w[1]
    ax.plot(x1_line, x2_line, color=PALETTE["emerald"],
            linewidth=2.5, zorder=2)

# Shaded regions
ax.fill_between(x1_line, x2_line, 2.0, alpha=0.06,
               color=PALETTE["coral"])
ax.fill_between(x1_line, x2_line, -1.0, alpha=0.06,
               color=PALETTE["royal_blue"])

# Annotate points with output
for i in range(4):
    out_val = "1" if y_gate[i] == 1 else "0"
    ax.annotate(out_val, (gate_inputs[i, 0], gate_inputs[i,
1])),
               textcoords="offset points", xytext=(10, 8),
               fontsize=11, fontweight="bold",
               color=PALETTE["charcoal"])

ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")
ax.set_title(f"{name} Gate\n$w$={w.tolist()}, $b$={b}",
            fontsize=11)
ax.set_aspect("equal")
if idx == 0:
    ax.legend(loc="center", fontsize=8)

fig.suptitle("Logic Gates Implemented with Single Perceptrons",
            fontsize=14,
            fontweight="bold", color=PALETTE["deep_blue"],
            y=1.02)
plt.tight_layout()
plt.savefig("../figures/01_logic_gates.png")
plt.show()

```

1.9 Exercises

Exercise 1.1 – Learning Rate Sensitivity

Re-run the perceptron training from Section 1.4 with three different learning rates: $\eta = 0.001$, $\eta = 0.1$, and $\eta = 1.0$. For each run:

1. Plot the number of misclassified samples per epoch.
 2. Record the epoch at which the model converges (if it does within 200 epochs).
 3. What happens when the learning rate is too large? Too small?
-

Exercise 1.2 – Perceptron as a Geometry Engine

The decision boundary $\mathbf{w} \cdot \mathbf{x} + b = 0$ is a line in 2D.

1. Show algebraically that the distance from a point $\mathbf{x}_0$ to this line is $\frac{|\mathbf{w} \cdot \mathbf{x}_0 + b|}{\|\mathbf{w}\|}$.
 2. After training the perceptron from Section 1.4, compute the distance of every sample to the final decision boundary.
 3. Which point is closest to the boundary? Plot it highlighted in a different colour. This is the **margin** – a concept we will revisit in the SVM chapter.
-

Exercise 1.3 – Building XOR from NAND Gates

Since XOR cannot be solved by a single perceptron, combine multiple perceptrons to build it:

1. Recall that $\text{XOR}(a, b) = \text{NAND}(\text{NAND}(a, \text{NAND}(a, b)), \text{NAND}(b, \text{NAND}(a, b)))$.
 2. Implement this as a small network of three perceptrons (two hidden + one output).
 3. Verify that it produces the correct XOR truth table.
 4. Reflect: what have we just built? (Hint: this is a **multi-layer perceptron** – the topic of Chapter 2.)
-

Chapter Summary

Concept	Key Takeaway
Perceptron	Simplest neural network – a linear binary classifier
Decision boundary	A hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$
Learning rule	$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$ for misclassified points
Convergence	Guaranteed if the data is linearly separable
XOR limitation	A single perceptron cannot represent non-linear boundaries

Concept	Key Takeaway
Logic gates	AND, OR, NAND are linearly separable; XOR is not

Next up: In Chapter 2, we stack perceptrons into layers, add non-linear activations, and learn about **backpropagation** – unlocking the power to solve XOR and far more complex problems.

```
import sys, os
sys.path.insert(0, os.path.abspath('../'))

import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
from mpl_toolkits.mplot3d import Axes3D

from style.dl_style import PALETTE, apply_style
apply_style()

np.random.seed(42)

print('Setup complete.')
```

Chapter 2: Linear Regression

Linear regression is the simplest supervised learning algorithm and the foundation for understanding neural networks. In this chapter we build it entirely from scratch.

Learning Objectives

By the end of this chapter you will be able to:

1. Define the linear regression problem and generate synthetic data.
2. Formulate the Mean Squared Error (MSE) loss and visualize its surface.
3. Derive and implement the closed-form (normal equation) solution.
4. Implement gradient descent from scratch and trace its path on a loss contour.
5. Understand the effect of learning rate on convergence.
6. Extend linear regression to polynomial features and observe overfitting.
7. Compare batch, mini-batch, and stochastic gradient descent.

2.1 The Problem

In **regression** we want to predict a continuous target y from input features $\mathbf{x}$.

The simplest model assumes a linear relationship:

$$\hat{y} = w x + b$$

where w (weight) and b (bias) are the parameters we need to learn.

Let's generate a synthetic 1-D dataset to work with.

```
# --- Generate synthetic 1-D regression data ---
rng = np.random.default_rng(42)

N = 80
w_true, b_true = 2.5, -1.0 # ground-truth parameters

X = rng.uniform(-3, 3, (N, 1))
y = (X @ np.array([[w_true]]).ravel() + b_true + rng.normal(0,
    0.8, N)

print(f'Dataset: {N} samples, X shape {X.shape}, y shape
    {y.shape}')
print(f'True parameters: w = {w_true}, b = {b_true}')

# --- Scatter plot ---
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(X, y, c=PALETTE['royal_blue'], alpha=0.7,
    edgecolors='white',
    linewidth=0.5, s=50, label='Data points')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Synthetic Regression Data')
ax.legend()
plt.tight_layout()
plt.show()
```

2.2 Loss Function: Mean Squared Error

To measure how well our model fits the data we need a **loss function**. The most common choice for regression is the **Mean Squared Error (MSE)**:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

The MSE penalises large errors quadratically, encouraging predictions close to the true values.

Below we implement MSE from scratch and then visualise the loss surface as a function of w and b .

```
def mse_loss(y_true, y_pred):
    """Mean Squared Error."""
    return np.mean((y_true - y_pred) ** 2)

# Quick test
y_dummy = X.ravel() * 1.0 + 0.0 # poor guess: w=1, b=0
print(f'MSE with w=1, b=0 : {mse_loss(y, y_dummy):.4f}')

y_good = X.ravel() * w_true + b_true
print(f'MSE with true params: {mse_loss(y, y_good):.4f}')
```



```

# --- 3-D Loss Surface Visualisation ---
w_range = np.linspace(-1, 6, 120)
b_range = np.linspace(-5, 3, 120)
W_grid, B_grid = np.meshgrid(w_range, b_range)

# Compute MSE for every (w, b) pair
Loss_grid = np.zeros_like(W_grid)
for i in range(W_grid.shape[0]):
    for j in range(W_grid.shape[1]):
        y_pred = X.ravel() * W_grid[i, j] + B_grid[i, j]
        Loss_grid[i, j] = mse_loss(y, y_pred)

fig = plt.figure(figsize=(11, 7))
ax = fig.add_subplot(111, projection='3d')

# Surface plot with PALETTE colour map
from matplotlib.colors import LinearSegmentedColormap
cmap_blues = LinearSegmentedColormap.from_list(
    'palette_blues', PALETTE['gradient_blues'], N=256)

surf = ax.plot_surface(W_grid, B_grid, Loss_grid,
                       cmap=cmap_blues, alpha=0.85,
                       edgecolor='none', antialiased=True)

# Mark the true minimum
ax.scatter([w_true], [b_true],
           [mse_loss(y, X.ravel() * w_true + b_true)],
           color=PALETTE['coral'], s=120, zorder=10,
           edgecolors='white', linewidth=1.2, label='True
           minimum')

ax.set_xlabel('w (weight)', labelpad=10)
ax.set_ylabel('b (bias)', labelpad=10)
ax.set_zlabel('MSE Loss', labelpad=10)
ax.set_title('MSE Loss Surface', pad=15)
ax.view_init(elev=30, azim=-50)
ax.legend(loc='upper left')
fig.colorbar(surf, shrink=0.45, aspect=15, pad=0.12)
plt.tight_layout()
plt.show()

```

2.3 Closed-Form Solution (Normal Equation)

Because the MSE loss is convex for linear regression, we can find the optimal parameters analytically.

Rewriting in matrix form with the design matrix $\mathbf{X}$ (with a column of ones for the bias):

$$\hat{\mathbf{y}} = \mathbf{X} \boldsymbol{\theta}, \quad \boldsymbol{\theta} = \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix}$$

The optimal parameters satisfy the **normal equation**:

$$[\mathbf{\theta}]^* = (\mathbf{X}^{\text{top}} \mathbf{X})^{-1} \mathbf{X}^{\text{top}} \mathbf{y}]$$

In practice we solve the linear system $(\mathbf{X}^{\text{top}} \mathbf{X}) \mathbf{\theta} = \mathbf{X}^{\text{top}} \mathbf{y}$ with `np.linalg.solve` for numerical stability.

```
# --- Closed-form solution via normal equation ---
X_b = np.hstack([np.ones((N, 1)), X]) # add bias column

theta_star = np.linalg.solve(X_b.T @ X_b, X_b.T @ y)
b_star, w_star = theta_star

print(f'Closed-form solution: w = {w_star:.4f}, b = {b_star:.4f}')
print(f'True parameters:      w = {w_true}, b = {b_true}')
print(f'MSE: {mse_loss(y, X.ravel() * w_star + b_star):.4f}')

# --- Plot the fit ---
x_plot = np.linspace(-3.5, 3.5, 200)

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(X, y, c=PALETTE['royal_blue'], alpha=0.6,
           edgecolors='white',
           linewidth=0.5, s=50, label='Data')
ax.plot(x_plot, w_star * x_plot + b_star,
        color=PALETTE['coral'], linewidth=2.5,
        label=f'Fit: y = {w_star:.2f}x + ({b_star:.2f})')
ax.plot(x_plot, w_true * x_plot + b_true,
        color=PALETTE['emerald'], linewidth=2, linestyle='--',
        label=f'True: y = {w_true}x + ({b_true})')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Closed-Form Solution')
ax.legend()
plt.tight_layout()
plt.show()
```

2.4 Gradient Descent from Scratch

The closed-form solution is elegant but doesn't scale to large datasets or non-linear models. **Gradient descent** is the workhorse of deep learning: we iteratively update parameters in the direction that reduces the loss.

Deriving the gradients

For a single sample the prediction is $(\hat{y}_i = w x_i + b)$, so the MSE is:

$$[L = \frac{1}{N} \sum_{i=1}^N (y_i - (w x_i + b))^2]$$

Taking partial derivatives:

$$\left[\frac{\partial L}{\partial w} = -\frac{2}{N} \sum_{i=1}^N (y_i - \hat{y}_i) x_i\right]$$

$$\left[\frac{\partial L}{\partial b} = -\frac{2}{N} \sum_{i=1}^N (y_i - \hat{y}_i)\right]$$

The update rule (with learning rate η):

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

```
class LinearRegression:
    """Linear regression trained with gradient descent."""

    def __init__(self):
        self.w = 0.0
        self.b = 0.0
        self.loss_history = []
        self.param_history = [] # for visualising the GD path

    def predict(self, X):
        return X.ravel() * self.w + self.b

    def fit(self, X, y, lr=0.01, epochs=200):
        """Train using full-batch gradient descent."""
        x = X.ravel()
        n = len(y)
        self.loss_history = []
        self.param_history = [(self.w, self.b)]

        for _ in range(epochs):
            y_pred = x * self.w + self.b
            error = y - y_pred

            # Gradients
            dw = -2.0 / n * np.sum(error * x)
            db = -2.0 / n * np.sum(error)

            # Update
            self.w -= lr * dw
            self.b -= lr * db

            loss = np.mean(error ** 2)
            self.loss_history.append(loss)
            self.param_history.append((self.w, self.b))

        return self

# Train
model = LinearRegression()
model.fit(X, y, lr=0.05, epochs=150)

print(f'Gradient descent: w = {model.w:.4f}, b = {model.b:.4f}')
print(f'Final MSE: {model.loss_history[-1]:.4f}')
```

```

# === HERO FIGURE: Gradient Descent Path on Loss Contour ===

fig, ax = plt.subplots(figsize=(9, 7))

# Contour of the loss surface
levels = np.logspace(-0.2, np.log10(Loss_grid.max()), 30)
cs = ax.contour(W_grid, B_grid, Loss_grid, levels=levels,
                cmap=cmap_blues, linewidths=0.9, alpha=0.8)
ax.contourf(W_grid, B_grid, Loss_grid, levels=levels,
            cmap=cmap_blues, alpha=0.25)
ax.clabel(cs, fontsize=7, fmt='%.1f', inline=True)

# Gradient descent path with arrows
path = np.array(model.param_history)
ax.plot(path[:, 0], path[:, 1], '-o',
        color=PALETTE['coral'], markersize=3, linewidth=1.5,
        markeredgewidth=0.4, zorder=5,
        label='Gradient descent path')

# Draw arrows along the path (every few steps)
arrow_steps = list(range(0, min(30, len(path) - 1), 3))
for i in arrow_steps:
    dx = path[i + 1, 0] - path[i, 0]
    dy = path[i + 1, 1] - path[i, 1]
    ax.annotate('', xy=(path[i + 1, 0], path[i + 1, 1]),
                xytext=(path[i, 0], path[i, 1]),
                arrowprops=dict(arrowstyle='->',
                                color=PALETTE['coral'],
                                lw=1.8, mutation_scale=14))

# Mark start and end
ax.scatter(*path[0], s=120, color=PALETTE['amber'],
          edgecolors='white',
          linewidth=1.5, zorder=6, label='Start (w=0, b=0)')
ax.scatter(*path[-1], s=120, color=PALETTE['emerald'],
          edgecolors='white',
          linewidth=1.5, zorder=6,
          label=f'End (w={model.w:.2f}, b={model.b:.2f})')

# Mark true minimum
ax.scatter(w_true, b_true, s=100, marker='*',
          color=PALETTE['deep_blue'],
          edgecolors='white', linewidth=1, zorder=6, label='True
          params')

ax.set_xlabel('w (weight)', fontsize=12)
ax.set_ylabel('b (bias)', fontsize=12)
ax.set_title('Gradient Descent Path on MSE Loss Contour',
            fontsize=14)
ax.legend(loc='upper right', fontsize=9)
plt.tight_layout()
plt.show()

```

2.5 Learning Rate Comparison

The **learning rate** (η) is the most important hyperparameter in gradient descent:

- **Too small:** convergence is very slow.
- **Just right:** smooth, fast convergence.
- **Too large:** the loss oscillates or **diverges**.

Let's train with three different learning rates and compare.

```
learning_rates = [0.001, 0.01, 0.1]
lr_colors = [PALETTE['steel_blue'], PALETTE['emerald'],
             PALETTE['coral']]
lr_models = {}

for lr_val in learning_rates:
    m = LinearRegression()
    m.fit(X, y, lr=lr_val, epochs=300)
    lr_models[lr_val] = m

# --- Side-by-side loss curves ---
fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=False)

for ax, lr_val, color in zip(axes, learning_rates, lr_colors):
    losses = lr_models[lr_val].loss_history
    ax.plot(losses, color=color, linewidth=2)
    ax.set_xlabel('Epoch')
    ax.set_ylabel('MSE Loss')
    ax.set_title(f'lr = {lr_val}')
    ax.set_ylim(bottom=0)

fig.suptitle('Learning Rate Comparison', fontsize=14,
             fontweight='bold',
             color=PALETTE['deep_blue'], y=1.03)
plt.tight_layout()
plt.show()

# --- Divergence example with very large lr ---
m_diverge = LinearRegression()
m_diverge.fit(X, y, lr=0.5, epochs=50)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(m_diverge.loss_history, color=PALETTE['coral'],
        linewidth=2.2)
ax.set_xlabel('Epoch')
ax.set_ylabel('MSE Loss')
ax.set_title('Divergence with lr = 0.5')
ax.set_yscale('log')
ax.axhline(y=mse_loss(y, X.ravel() * w_star + b_star),
           color=PALETTE['emerald'], linestyle='--',
           linewidth=1.5,
           label='Optimal MSE')
ax.legend()
```

```
plt.tight_layout()
plt.show()
```

2.6 Polynomial Regression

Linear regression can model non-linear relationships by applying **feature engineering**: we transform the input into polynomial features and then fit a linear model in this expanded space.

For a single feature x and degree d :

$$\phi(x) = [1, x, x^2, \dots, x^d]$$

As we increase the degree, the model becomes more flexible. But too much flexibility leads to **overfitting**: the model memorises the training noise instead of learning the underlying pattern.

```
# --- Generate noisy sine data for polynomial fitting ---
rng2 = np.random.default_rng(42)
N_poly = 30
X_poly = np.sort(rng2.uniform(-3, 3, N_poly))
y_poly = np.sin(X_poly) + rng2.normal(0, 0.3, N_poly)

def polynomial_features(x, degree):
    """Create polynomial feature matrix [1, x, x^2, ...,
    x^degree]."""
    return np.column_stack([x ** p for p in range(degree + 1)])

def fit_polynomial(x, y, degree):
    """Fit polynomial regression via normal equation."""
    Phi = polynomial_features(x, degree)
    # Regularised solve for numerical stability at high degree
    lam = 1e-10
    theta = np.linalg.solve(Phi.T @ Phi + lam * np.eye(degree +
    1), Phi.T @ y)
    return theta

# --- Overfitting Visualisation: 2x2 grid ---
degrees = [1, 3, 9, 15]
degree_colors = [PALETTE['royal_blue'], PALETTE['emerald'],
    PALETTE['amber'], PALETTE['coral']]

x_fine = np.linspace(-3.2, 3.2, 300)

fig, axes = plt.subplots(2, 2, figsize=(11, 9))

for ax, deg, color in zip(axes.ravel(), degrees, degree_colors):
    theta = fit_polynomial(X_poly, y_poly, deg)
    Phi_fine = polynomial_features(x_fine, deg)
```

```

y_fine = Phi_fine @ theta

# True function
ax.plot(x_fine, np.sin(x_fine), '--',
        color=PALETTE['warm_gray'],
        linewidth=1.5, label='True: sin(x)')

# Polynomial fit
ax.plot(x_fine, y_fine, color=color, linewidth=2.2,
        label=f'Degree {deg}')

# Data
ax.scatter(X_poly, y_poly, c=PALETTE['deep_blue'], s=35,
          edgecolors='white', linewidth=0.5, zorder=5)

# Training MSE
y_train_pred = polynomial_features(X_poly, deg) @ theta
train_mse = mse_loss(y_poly, y_train_pred)
ax.set_title(f'Degree {deg} (Train MSE = {train_mse:.4f})')
ax.set_ylim(-2.5, 2.5)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.legend(fontsize=8, loc='upper right')

fig.suptitle('Polynomial Regression: Underfitting to Overfitting',
            fontsize=14, fontweight='bold',
            color=PALETTE['deep_blue'], y=1.02)
plt.tight_layout()
plt.show()

```

2.7 Mini-batch Gradient Descent

In practice we rarely use all data at once (full batch). Instead we can:

Variant	Batch Size	Pro	Con
Batch GD	$\backslash(N\backslash)$	Stable gradient	Slow per step; memory heavy
Stochastic GD (SGD)	1	Fast per step	Very noisy
Mini-batch GD	$\backslash(B\backslash)$ (e.g., 16, 32)	Good balance	Needs batch-size tuning

Let's implement all three and compare their convergence.

```

def gradient_descent_variants(X, y, lr=0.05, epochs=100,
                             batch_size=None, seed=42):
    """Train linear regression with configurable batch size.

```

```

batch_size=None    -> full batch
batch_size=1      -> stochastic
batch_size=k      -> mini-batch of size k
"""

rng_gd = np.random.default_rng(seed)
x = X.ravel()
n = len(y)
w, b = 0.0, 0.0
loss_history = []

if batch_size is None:
    batch_size = n # full batch

for epoch in range(epochs):
    indices = rng_gd.permutation(n)
    for start in range(0, n, batch_size):
        idx = indices[start:start + batch_size]
        x_batch = x[idx]
        y_batch = y[idx]

        y_pred = x_batch * w + b
        error = y_batch - y_pred

        dw = -2.0 / len(idx) * np.sum(error * x_batch)
        db = -2.0 / len(idx) * np.sum(error)

        w -= lr * dw
        b -= lr * db

        # Record loss on full dataset at end of each epoch
        full_pred = x * w + b
        loss_history.append(mse_loss(y, full_pred))

    return w, b, loss_history

# Train all three variants
configs = [
    ('Batch GD',      None, PALETTE['royal_blue']),
    ('Mini-batch (16)', 16,  PALETTE['emerald']),
    ('SGD (1)',       1,   PALETTE['coral']),
]

fig, ax = plt.subplots(figsize=(9, 5))

for name, bs, color in configs:
    w_gd, b_gd, losses = gradient_descent_variants(
        X, y, lr=0.02, epochs=120, batch_size=bs)
    ax.plot(losses, color=color, linewidth=2, alpha=0.85,
            label=name)

ax.set_xlabel('Epoch')
ax.set_ylabel('MSE Loss')

```



```
ax.set_title('Convergence: Batch vs Mini-batch vs Stochastic GD')
ax.legend(fontsize=10)
ax.set_ylim(bottom=0)
plt.tight_layout()
plt.show()
```

2.8 Exercises

Exercise 1 – Regularisation.

Add an (L_2) (Ridge) regularisation term $(\lambda \|\mathbf{w}\|^2)$ to the MSE loss. Modify the `LinearRegression` class to accept a `reg` parameter. Re-run the polynomial regression experiment from Section 2.6 with degrees 9 and 15 using $(\lambda = 0.01)$. How does the fit change? Plot the results alongside the unregularised fits.

Exercise 2 – Multi-dimensional regression.

Generate a dataset with 3 input features using `make_regression` from `utils/data.py`. Extend the `LinearRegression` class to handle multiple features (replace scalar `w` with a weight vector). Train it with gradient descent and verify against the closed-form normal equation.

Exercise 3 – Learning rate schedules.

Implement a simple learning rate decay: $(\eta_t = \eta_0 / (1 + \alpha, t))$ where (t) is the epoch number and (α) is a decay rate. Compare the convergence of constant learning rate vs. decayed learning rate on the original dataset. Plot both loss curves on the same axes.

```
import sys
sys.path.insert(0, '..')

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from style.dl_style import PALETTE, apply_style

apply_style()
np.random.seed(42)
```

Chapter 3: Logistic Regression

In Chapter 2 we learned how linear regression maps inputs to continuous outputs. But what if the target is a **category** rather than a number? In this chapter, we bridge the gap from regression to classification by introducing **logistic regression** – the fundamental building block of neural network classifiers.

Learning Objectives

By the end of this chapter you will be able to:

1. Explain why linear regression fails for classification and how the **sigmoid function** solves the problem.
2. Derive and implement the **Binary Cross-Entropy (BCE) loss** from maximum likelihood principles.
3. Compute the gradient $\partial L / \partial \mathbf{w}$ and appreciate its elegant closed-form.
4. Extend binary logistic regression to multi-class problems via the **softmax function**.
5. Implement both LogisticRegression and SoftmaxRegression from scratch in NumPy.

3.1 From Regression to Classification

3.2 The Model

Logistic regression models the probability that input $\mathbf{x}$ belongs to class 1:

$$P(y=1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

The **decision boundary** is the set of points where $P(y=1 \mid \mathbf{x}) = 0.5$, which happens exactly when:

$$\frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} = 0.5 \quad \Longleftrightarrow \quad \mathbf{w}^T \mathbf{x} + b = 0$$

This is a **hyperplane** in the feature space – logistic regression can only learn

3.1 From Regression to Classification

linear decision boundaries.

3.3 Binary Cross-Entropy Loss

Derivation from Maximum Likelihood

Each sample $\mathbf{x}_i$ is a Bernoulli trial:

$$P(y_i \mid \mathbf{x}_i) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1 - y_i}$$

where $\hat{y}_i = \sigma(\mathbf{w}^{\text{top}} \mathbf{x}_i + b)$.

The **likelihood** over N i.i.d. samples is:

$$\mathcal{L} = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1 - y_i}$$

Taking the **negative log-likelihood** and averaging gives us the **Binary Cross-Entropy (BCE) loss**:

$$L = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i) \right]$$

Intuition: - When $y_i = 1$: the loss is $-\log \hat{y}_i$ – penalizes low predicted probability. - When $y_i = 0$: the loss is $-\log (1 - \hat{y}_i)$ – penalizes high predicted probability.

--- 3.3 BCE Loss Implementation ---

```
def bce_loss(y_true, y_pred):
    """Binary Cross-Entropy loss."""
    eps = 1e-8
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) *
                    np.log(1 - y_pred))

# Visualize BCE for a single sample
y_hat = np.linspace(0.001, 0.999, 500)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

# Left: loss curves for y=1 and y=0
ax = axes[0]
ax.plot(y_hat, -np.log(y_hat), color=PALETTE['royal_blue'],
        linewidth=2.5,
        label=r'$y=1$: $-\log(\hat{y})$')
ax.plot(y_hat, -np.log(1 - y_hat), color=PALETTE['coral'],
        linewidth=2.5,
        label=r'$y=0$: $-\log(1-\hat{y})$')
ax.set_xlabel(r'Predicted probability $\hat{y}$', fontsize=11)
```

```

ax.set_ylabel('Loss', fontsize=11)
ax.set_title('BCE Loss per Sample', fontsize=13)
ax.legend(fontsize=10)
ax.set_ylim(0, 5)
ax.set_xlim(0, 1)

# Right: BCE loss landscape (contour) for 2D logistic regression
ax = axes[1]

# Simple 2D dataset
rng = np.random.default_rng(42)
X_simple = rng.normal(0, 1, (100, 1))
y_simple = (X_simple[:, 0] > 0).astype(float)

w_range = np.linspace(-4, 8, 200)
b_range = np.linspace(-4, 4, 200)
W, B = np.meshgrid(w_range, b_range)
Loss = np.zeros_like(W)

for i in range(W.shape[0]):
    for j in range(W.shape[1]):
        z_val = X_simple * W[i, j] + B[i, j]
        pred = sigmoid(z_val).ravel()
        Loss[i, j] = bce_loss(y_simple, pred)

contour = ax.contourf(W, B, Loss, levels=30, cmap='Blues_r',
                      alpha=0.9)
ax.contour(W, B, Loss, levels=15, colors=PALETTE['deep_blue'],
           linewidths=0.5, alpha=0.4)
plt.colorbar(contour, ax=ax, label='BCE Loss')
ax.set_xlabel('Weight (w)', fontsize=11)
ax.set_ylabel('Bias (b)', fontsize=11)
ax.set_title('BCE Loss Landscape', fontsize=13)

# Mark the minimum
min_idx = np.unravel_index(np.argmin(Loss), Loss.shape)
ax.plot(W[min_idx], B[min_idx], '*', color=PALETTE['coral'],
        markersize=14, zorder=5,
        markeredgecolor='white', markeredgewidth=1)
ax.annotate('Minimum', xy=(W[min_idx], B[min_idx]),
            xytext=(W[min_idx]+1.5, B[min_idx]+1.5),
            fontsize=10, color=PALETTE['coral'],
            fontweight='bold',
            arrowprops=dict(arrowstyle='->',
                            color=PALETTE['coral'], lw=1.2))

plt.tight_layout()
plt.show()

```

3.4 Gradient Derivation and Implementation

3.5 Training on 2D Data

```python # — 3.5 Generate moon-shaped data and train

—

---

### ## 3.4 Gradient Derivation and Implementation

---

```
def make_moons(n=300, noise=0.15, seed=42):
 """Generate two interleaving half-moon shapes."""
 rng = np.random.default_rng(seed)
 n_each = n // 2
 theta_top = np.linspace(0, np.pi, n_each)
 theta_bot = np.linspace(0, np.pi, n - n_each)
 x_top = np.column_stack([np.cos(theta_top),
 np.sin(theta_top)])
 x_bot = np.column_stack([1 - np.cos(theta_bot),
 1 - np.sin(theta_bot) - 0.5])
 X = np.vstack([x_top, x_bot]) + rng.normal(0, noise, (n, 2))
 y = np.concatenate([np.zeros(n_each), np.ones(n - n_each)])
 return X, y

X_moon, y_moon = make_moons(n=300, noise=0.15, seed=42)

Train model
model = LogisticRegression(n_features=2, lr=0.5)
model.fit(X_moon, y_moon, epochs=1000)

Accuracy
y_pred = model.predict(X_moon)
acc = np.mean(y_pred == y_moon)
print(f"Training accuracy: {acc:.2%}")
print(f"Final loss: {model.loss_history[-1]:.4f}")

```python # — 3.5 Decision boundary + Loss curve —
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: Decision boundary with probability contours
ax = axes[0]
x_min, x_max = X_moon[:, 0].min() - 0.5, X_moon[:, 0].max() + 0.5
y_min, y_max = X_moon[:, 1].min() - 0.5, X_moon[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                     np.linspace(y_min, y_max, 300))
grid = np.column_stack([xx.ravel(), yy.ravel()])
Z = model.predict_proba(grid).reshape(xx.shape)

# Custom colormap from PALETTE
cmap_custom = mcolors.LinearSegmentedColormap.from_list('custom',
[PALETTE['royal_blue'], PALETTE['light_gray'], PALETTE['coral']])

contour = ax.contourf(xx, yy, Z, levels=50,
cmap=cmap_custom, alpha=0.8)
ax.contour(xx, yy, Z, levels=[0.5], colors=[PALETTE['charcoal']],
linewidths=2, linestyles='-')
plt.colorbar(contour, ax=ax, label=r'\(P(y=1 \mid \mathbf{x})\)')

# Scatter points
mask0 = y_moon == 0
mask1 = y_moon == 1
ax.scatter(X_moon[mask0, 0], X_moon[mask0, 1],
c=PALETTE['royal_blue'], edgecolors='white',
linewidth=0.6, s=35, label='Class 0', zorder=3)
ax.scatter(X_moon[mask1, 0], X_moon[mask1, 1],
c=PALETTE['coral'], edgecolors='white',
linewidth=0.6, s=35, label='Class 1', zorder=3)

ax.set_xlabel('(x_1)')
ax.set_ylabel('(x_2)')
ax.set_title('Decision Boundary (Logistic Regression)')
ax.legend(loc='upper right', fontsize=9)
```

3.4 Gradient Derivation and Implementation

```
# Right: Loss curve ax = axes[1]
ax.plot(model.loss_history,
color=PALETTE['royal_blue'], linewidth=2.2)
ax.set_xlabel('Epoch') ax.set_ylabel('BCE Loss')
ax.set_title('Training Loss Curve')
ax.axhline(y=model.loss_history[-1],
color=PALETTE['coral'], linestyle=':', alpha=0.6,
label=f'Final loss = {model.loss_history[-1]:.4f}')
ax.legend(fontsize=10)

plt.tight_layout() plt.show() ``
```

3.6 Decision Boundary Evolution

Let's watch how the decision boundary evolves during training. We retrain and snapshot the model at epochs 1, 50, 100, 200, 500, and 1000.

```
# --- 3.6 Decision Boundary Evolution (2x3 grid) ---

snapshot_epochs = [1, 50, 100, 200, 500, 1000]
snapshots = {} # epoch -> (w, b)

# Retrain and capture snapshots
model_evo = LogisticRegression(n_features=2, lr=0.5)
N = X_moon.shape[0]

for epoch in range(1, max(snapshot_epochs) + 1):
    y_hat = model_evo.predict_proba(X_moon)
    eps = 1e-8
    loss = -np.mean(y_moon * np.log(y_hat + eps) + (1 - y_moon) *
        np.log(1 - y_hat + eps))
    model_evo.loss_history.append(loss)

    error = y_hat - y_moon
    dw = (1 / N) * (X_moon.T @ error)
    db = (1 / N) * np.sum(error)
    model_evo.w -= model_evo.lr * dw
    model_evo.b -= model_evo.lr * db

    if epoch in snapshot_epochs:
        snapshots[epoch] = (model_evo.w.copy(), model_evo.b)

# Plot 2x3 grid
fig, axes = plt.subplots(2, 3, figsize=(15, 9))

cmap_custom = mcolors.LinearSegmentedColormap.from_list(
    'custom', [PALETTE['royal_blue'], PALETTE['light_gray'],
        PALETTE['coral']])

x_min, x_max = X_moon[:, 0].min() - 0.5, X_moon[:, 0].max() + 0.5
y_min, y_max = X_moon[:, 1].min() - 0.5, X_moon[:, 1].max() + 0.5
```

```

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                    np.linspace(y_min, y_max, 200))
grid = np.column_stack([xx.ravel(), yy.ravel()])

for idx, epoch in enumerate(snapshot_epochs):
    ax = axes[idx // 3, idx % 3]
    w_snap, b_snap = snapshots[epoch]

    # Compute decision boundary
    z_grid = grid @ w_snap + b_snap
    Z = sigmoid(z_grid).reshape(xx.shape)

    ax.contourf(xx, yy, Z, levels=50, cmap=cmap_custom, alpha=0.8)
    ax.contour(xx, yy, Z, levels=[0.5],
               colors=[PALETTE['charcoal']],
               linewidths=2, linestyles='--')

    mask0 = y_moon == 0
    mask1 = y_moon == 1
    ax.scatter(X_moon[mask0, 0], X_moon[mask0, 1],
               c=PALETTE['royal_blue'],
               edgecolors='white', linewidth=0.5, s=18, zorder=3)
    ax.scatter(X_moon[mask1, 0], X_moon[mask1, 1],
               c=PALETTE['coral'],
               edgecolors='white', linewidth=0.5, s=18, zorder=3)

    # Epoch title with loss
    loss_val = model_evo.loss_history[epoch - 1] if epoch <=
        len(model_evo.loss_history) else 0
    ax.set_title(f'Epoch {epoch} (loss={loss_val:.3f})',
                 fontsize=11)
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.set_xticks([])
    ax.set_yticks([])

fig.suptitle('Decision Boundary Evolution During Training',
             fontsize=14,
             fontweight='bold', color=PALETTE['deep_blue'],
             y=1.01)
plt.tight_layout()
plt.show()

```

3.7 Multi-Class: Softmax Regression

3.8 Comparison: Perceptron vs Logistic Regression

The **Perceptron** (a hard-threshold classifier from the 1950s) and **Logistic Regression** (a probabilistic classifier) both learn linear decision boundaries. Let's compare them side by side.

```python # — 3.8 Perceptron vs Logistic Regression —

class Perceptron: """Simple Perceptron classifier."""

---

### ## 3.7 Multi-Class: Softmax Regression

---

```
def init(self, n_features, lr=0.01): self.w =
np.zeros(n_features) self.b = 0.0 self.lr = lr

def predict(self, X): return (X @ self.w + self.b >=
0).astype(int)

def fit(self, X, y, epochs=1000): for epoch in
range(epochs): for i in range(len(X)): pred = 1 if (X[i] @
self.w + self.b >= 0) else 0 error = y[i] - pred self.w +=
self.lr * error * X[i] self.b += self.lr * error return self

Use moon data for comparison perceptron =
Perceptron(n_features=2, lr=0.1)
perceptron.fit(X_moon, y_moon, epochs=100)

logistic = LogisticRegression(n_features=2, lr=0.5)
logistic.fit(X_moon, y_moon, epochs=1000)

Side-by-side plots fig, axes = plt.subplots(1, 2,
figsize=(14, 5.5))

models_list = [('Perceptron', perceptron), ('Logistic
Regression', logistic)]

x_min, x_max = X_moon[:, 0].min() - 0.5, X_moon[:,
0].max() + 0.5 y_min, y_max = X_moon[:, 1].min() -
0.5, X_moon[:, 1].max() + 0.5 xx, yy =
np.meshgrid(np.linspace(x_min, x_max, 300),
np.linspace(y_min, y_max, 300)) grid =
np.column_stack([xx.ravel(), yy.ravel()])

cmap_custom =
mcolors.LinearSegmentedColormap.from_list('custom',
[PALETTE['royal_blue'], PALETTE['light_gray'],
PALETTE['coral']])

for i, (name, m) in enumerate(models_list): ax = axes[i]

if hasattr(m, 'predict_proba'): Z =
m.predict_proba(grid).reshape(xx.shape) else: Z =
m.predict(grid).reshape(xx.shape).astype(float)

ax.contourf(xx, yy, Z, levels=50, cmap=cmap_custom,
alpha=0.8) ax.contour(xx, yy, Z, levels=[0.5],
colors=[PALETTE['charcoal']], linewidths=2,
linestyles='--')

mask0 = y_moon == 0 mask1 = y_moon == 1
ax.scatter(X_moon[mask0, 0], X_moon[mask0, 1],
c=PALETTE['royal_blue'], edgecolors='white',
linewidth=0.6, s=30, label='Class 0', zorder=3)
ax.scatter(X_moon[mask1, 0], X_moon[mask1, 1],
c=PALETTE['coral'], edgecolors='white',
linewidth=0.6, s=30, label='Class 1', zorder=3)

acc = np.mean(m.predict(X_moon) == y_moon)
ax.set_title(f'{name} (acc={acc:.1%})', fontsize=12)
ax.set_xlabel('(x_1)') ax.set_ylabel('(x_2)')
ax.legend(loc='upper right', fontsize=9)

fig.suptitle('Perceptron vs Logistic Regression',
fontsize=14, fontweight='bold',
```



---

### ## 3.7 Multi-Class: Softmax Regression

---

```
color=PALETTE['deep_blue'], y=1.01)
plt.tight_layout() plt.show() ``
```

#### Key differences:

|                                                                        |                     |           |               |
|------------------------------------------------------------------------|---------------------|-----------|---------------|
| Perceptron                                                             | Logistic Regression | —   —   — | <b>Output</b> |
| Hard label (0 or 1)   Probability $(P(y=1 \mid \mathbf{x}))$           |                     |           |               |
| <b>Loss</b>   Misclassification count   Binary cross-entropy           |                     |           |               |
| <b>Boundary</b>   Linear   Linear (but with confidence gradient)       |                     |           |               |
| <b>Convergence</b>   Only if linearly separable   Always (convex loss) |                     |           |               |
| <b>Gradient</b>   Not well-defined at boundary   Smooth everywhere     |                     |           |               |

---

## 3.9 Exercises

### Exercise 1: Implementing the Log-Sum-Exp Trick

The **log-sum-exp** (LSE) function computes  $(\log \sum_k e^{z_k})$  in a numerically stable way:

$$[\text{LSE}(\mathbf{z}) = m + \log \sum_k e^{z_k - m}, \quad m = \max(\mathbf{z})]$$

**Task:** Implement a `log_sum_exp(z)` function and use it to write a numerically stable cross-entropy loss. Verify that it gives the same result as the naive implementation for moderate inputs but does not overflow for very large logits (e.g.,  $(z_k = 1000)$ ).

### Exercise 2: Regularized Logistic Regression

Add **L2 regularization** to the `LogisticRegression` class. The regularized loss is:

$$[L_{\text{reg}} = L_{\text{BCE}} + \frac{\lambda}{2N} \|\mathbf{w}\|^2]$$

**Tasks:** 1. Derive the modified gradient  $(\partial L_{\text{reg}} / \partial \mathbf{w})$ . 2. Implement `LogisticRegressionL2` with a `lam` parameter. 3. Train with  $(\lambda \in \{0, 0.1, 1, 10\})$  on the moon data and plot the decision boundaries side by side. How does regularization affect the boundary?

### Exercise 3: Multi-class Metrics

Implement a function `classification_report(y_true, y_pred, class_names)` that computes for each class: - **Precision** =  $TP / (TP + FP)$  - **Recall** =  $TP / (TP + FN)$  - **F1-score** =  $2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$

Apply it to the 3-class spiral predictions from Section 3.7. Which class does the softmax model find most difficult?

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
```

```
from style.dl_style import apply_style, PALETTE
apply_style()
np.random.seed(42)
```

## Chapter 4: Neural Network Foundations

In this chapter we build the core machinery of neural networks **entirely from scratch**. By the end you will understand:

- **Why** linear models fail on problems like XOR, and why we need non-linear transformations.
- The zoo of **activation functions** (sigmoid, tanh, ReLU and friends) and their derivatives.
- How to implement a **single-hidden-layer network** (forward pass, weight storage, predictions).
- How to **solve XOR** and visualize what the hidden layer actually does to the input space.
- The critical role of **weight initialization** (Xavier, He) in keeping signals healthy.
- How **network width** controls decision-boundary complexity.
- How the **choice of activation** affects learning dynamics.
- The beautiful geometric intuition: a hidden layer **warps space** to make data linearly separable.

---

### ## 4.1 Why Go Beyond Linear Models

---

#### ### 4.2 Activation Functions

Activation functions inject the **non-linearity** that makes neural networks powerful. Without them every layer would just be another matrix multiply, and the whole network would collapse to a single linear map.

Below we implement each activation *and* its derivative.

```
```python # ----- Activation
functions and their derivatives
```

```
def sigmoid(z): return 1.0 / (1.0
+ np.exp(-z))
```

```
def sigmoid_deriv(z): s =
sigmoid(z) return s * (1.0 - s)
```

```
def tanh_act(z): return
np.tanh(z)
```

4.1 Why Go Beyond Linear Models

```
def tanh_deriv(z): return 1.0 -
np.tanh(z) ** 2

def relu(z): return
np.maximum(0, z)

def relu_deriv(z): return (z >
0).astype(float)

def leaky_relu(z, alpha=0.01):
return np.where(z > 0, z, alpha
* z)

def leaky_relu_deriv(z,
alpha=0.01): return np.where(z
> 0, 1.0, alpha)

def elu(z, alpha=1.0): return
np.where(z > 0, z, alpha *
(np.exp(z) - 1))

def elu_deriv(z, alpha=1.0):
return np.where(z > 0, 1.0,
alpha * np.exp(z))

def swish(z): return z *
sigmoid(z)

def swish_deriv(z): s =
sigmoid(z) return s + z * s *
(1.0 - s)

def gelu(z): return 0.5 * z * (1.0
+ np.tanh(np.sqrt(2.0 / np.pi) *
(z + 0.044715 * z ** 3)))

def gelu_deriv(z): """Numerical
derivative for GELU (exact
form is unwieldy).""" h = 1e-7
return (gelu(z + h) - gelu(z -
h)) / (2 * h)

print('Activation functions
defined.') ``

``python # ----- Hero
visualization: activation
functions + derivatives -----

z = np.linspace(-5, 5, 500)

activations = [ ('Sigmoid',
sigmoid, sigmoid_deriv),
('Tanh', tanh_act, tanh_deriv),
('ReLU', relu, relu_deriv),
('Leaky ReLU', leaky_relu,
leaky_relu_deriv), ('ELU', elu,
elu_deriv), ('Swish', swish,
swish_deriv), ('GELU', gelu,
gelu_deriv),]
```

4.1 Why Go Beyond Linear Models

```
colors =
[ PALETTE['royal_blue'],
  PALETTE['coral'],
  PALETTE['emerald'],
  PALETTE['amber'],
  PALETTE['steel_blue'],
  PALETTE['deep_blue'],
  PALETTE['slate'],]

fig, axes = plt.subplots(2, 4,
figsize=(16, 7))

for idx, (name, fn, dfn) in
enumerate(activations): row, col
= divmod(idx, 4) ax = axes[row,
col] c = colors[idx] ax.plot(z,
fn(z), color=c, lw=2.2,
label=name) ax.plot(z, dfn(z),
color=c, lw=1.4, ls='-',
alpha=0.7, label=f'{name}\'')
ax.axhline(0,
color=PALETTE['warm_gray'],
lw=0.5, zorder=0) ax.axvline(0,
color=PALETTE['warm_gray'],
lw=0.5, zorder=0)
ax.set_title(name, fontsize=11)
ax.legend(fontsize=8,
loc='upper left') ax.set_xlim(-5,
5)

# hide the unused 8th subplot
axes[1, 3].set_visible(False)

fig.suptitle('Activation
Functions and Their
Derivatives', fontsize=14,
fontweight='bold',
color=PALETTE['deep_blue'],
y=1.01) plt.tight_layout()
plt.show() ``
```

Key observations:

Activation	Range	Derivative
Sigmoid	(0, 1)	(0, 0.25]
Tanh	(-1, 1)	(0, 1]
ReLU	[0, inf)	{0, 1}
Leaky ReLU	(-inf, inf)	{alpha, 1}

Smooth, probabilistic output | Saturates, vanishing gradients | | Zero-centered | Still saturates | | Fast, no saturation for $z > 0$ | Dead neurons ($z < 0$) | | Not zero-centered | |

4.1 Why Go Beyond Linear Models

ELU | (-alpha, inf) | smooth |
Smooth, closer to zero-centered
| Expensive exp() || Swish |
unbounded | smooth | Self-
gated, smooth | Slightly more
compute || GELU | unbounded |
smooth | Used in Transformers |
Approximation needed |

4.3 A Single Hidden Layer

The simplest non-trivial neural network has **one hidden layer**:

$$\mathbf{h} = \phi(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \quad \hat{y} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2$$

where ϕ is a non-linear activation function applied element-wise.

```
class SimpleNeuralNetwork:
    """Single-hidden-layer feedforward network for binary
        classification."""

    ACTIVATIONS = {
        'sigmoid': (sigmoid, sigmoid_deriv),
        'tanh':    (tanh_act, tanh_deriv),
        'relu':    (relu, relu_deriv),
    }

    def __init__(self, n_input, n_hidden, n_output,
                  activation='relu'):
        self.n_input = n_input
        self.n_hidden = n_hidden
        self.n_output = n_output

        # Activation
        self.act_fn, self.act_deriv = self.ACTIVATIONS[activation]

        # He initialization (works well with ReLU; reasonable for
        # others too)
        self.W1 = np.random.randn(n_input, n_hidden) *
            np.sqrt(2.0 / n_input)
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = np.random.randn(n_hidden, n_output) *
            np.sqrt(2.0 / n_hidden)
        self.b2 = np.zeros((1, n_output))

        # Cache for backprop
        self.cache = {}

    def forward(self, X):
```

```

        """Forward pass. Stores intermediates in self.cache."""
        self.cache['X'] = X
        self.cache['Z1'] = X @ self.W1 + self.b1          # pre-
        activation
        self.cache['H'] = self.act_fn(self.cache['Z1']) # hidden
        activations
        self.cache['Z2'] = self.cache['H'] @ self.W2 + self.b2 #
        output logits
        self.cache['Y_hat'] = sigmoid(self.cache['Z2']) # output
        probability
        return self.cache['Y_hat']

def predict(self, X):
    """Return class predictions (0 or 1)."""
    probs = self.forward(X)
    return (probs >= 0.5).astype(int).ravel()

def backward(self, y):
    """Backprop through the network. Returns gradients
    dict."""
    m = y.shape[0]
    Y = y.reshape(-1, 1)
    Y_hat = self.cache['Y_hat']
    H = self.cache['H']
    Z1 = self.cache['Z1']
    X = self.cache['X']

    # Output layer gradient (binary cross-entropy + sigmoid)
    dZ2 = Y_hat - Y                                # (m, n_output)
    dW2 = (H.T @ dZ2) / m
    db2 = np.mean(dZ2, axis=0, keepdims=True)

    # Hidden layer gradient
    dH = dZ2 @ self.W2.T                            # (m, n_hidden)
    dZ1 = dH * self.act_deriv(Z1)
    dW1 = (X.T @ dZ1) / m
    db1 = np.mean(dZ1, axis=0, keepdims=True)

    return {'dW1': dW1, 'db1': db1, 'dW2': dW2, 'db2': db2}

def train_step(self, X, y, lr=0.1):
    """One step of forward + backward + parameter update."""
    self.forward(X)
    grads = self.backward(y)
    self.W1 -= lr * grads['dW1']
    self.b1 -= lr * grads['db1']
    self.W2 -= lr * grads['dW2']
    self.b2 -= lr * grads['db2']

    # Binary cross-entropy loss
    eps = 1e-12
    yh = self.cache['Y_hat'].ravel()
    loss = -np.mean(y * np.log(yh + eps) + (1 - y) * np.log(1
    - yh + eps))

```

```

        return loss

print('SimpleNeuralNetwork class defined.')

# ----- Network architecture diagram -----
from matplotlib.patches import Circle, FancyArrowPatch

def draw_network(ax, layer_sizes, layer_labels=None):
    """Draw a feed-forward network diagram with circles and
    connections."""
    n_layers = len(layer_sizes)
    max_neurons = max(layer_sizes)
    x_spacing = 2.0
    y_spacing = 1.2
    radius = 0.3

    layer_colors = [PALETTE['steel_blue'], PALETTE['emerald'],
                    PALETTE['coral']]

    positions = [] # positions[layer][neuron] = (x, y)
    for i, size in enumerate(layer_sizes):
        x = i * x_spacing
        y_start = -(size - 1) * y_spacing / 2
        layer_pos = [(x, y_start + j * y_spacing) for j in
                     range(size)]
        positions.append(layer_pos)

    # Draw connections first (behind neurons)
    for i in range(n_layers - 1):
        for (x1, y1) in positions[i]:
            for (x2, y2) in positions[i + 1]:
                ax.plot([x1, x2], [y1, y2],
                        color=PALETTE['light_blue'],
                        lw=0.8, alpha=0.6, zorder=1)

    # Draw neurons
    for i, layer_pos in enumerate(positions):
        c = layer_colors[min(i, len(layer_colors) - 1)]
        for (x, y) in layer_pos:
            circle = Circle((x, y), radius, facecolor=c,
                            edgecolor='white', lw=2, zorder=3)
            ax.add_patch(circle)

    # Layer labels
    if layer_labels is None:
        layer_labels = ['Input', 'Hidden', 'Output']
    for i, label in enumerate(layer_labels):
        x = i * x_spacing
        y_top = (layer_sizes[i] - 1) * y_spacing / 2 + radius +
                0.45
        ax.text(x, y_top, label, ha='center', fontsize=10,
                fontweight='bold', color=PALETTE['deep_blue'])

    ax.set_xlim(-0.8, (n_layers - 1) * x_spacing + 0.8)

```

```

y_max = (max_neurons - 1) * y_spacing / 2 + 1.2
ax.set_ylim(-y_max, y_max)
ax.set_aspect('equal')
ax.axis('off')

fig, ax = plt.subplots(figsize=(8, 5))
draw_network(ax, [2, 4, 1],
             ['Input\n($x_1, x_2$)', 'Hidden\n$\mathbf{h} = \phi(W_1 x + b_1)$',
              'Output\n$\hat{y}$'])
ax.set_title('Single Hidden Layer Architecture', fontsize=13,
             fontweight='bold', color=PALETTE['deep_blue'],
             pad=20)
plt.tight_layout()
plt.show()

```

4.4 Solving XOR

4.5 Weight Initialization

How we initialize weights has a dramatic effect on whether training succeeds.

Why zeros fail If every weight is identical (e.g., all zeros), every neuron computes the same function and receives the same gradient. They can never differentiate themselves. This is the **symmetry problem**.

Common strategies

Strategy	Formula	Best with
random	$(W \sim \mathcal{N}(0, 0.01))$	Small
Xavier / Glorot	$(W \sim \mathcal{N}(0, \sqrt{2 / (n_{in} + n_{out})}))$	Shallow nets
Sigmoid / Tanh	He	
	$(W \sim \mathcal{N}(0, \sqrt{2 / n_{in}}))$	
ReLU family		

``python # ———— Activation distributions through 5 layers

```

def simulate_forward(init_fn,
                    activation_fn, n_layers=5,
                    n_neurons=256, n_samples=500):
    """Simulate forward pass through
    n_layers, return activations per
    layer.""" X =
    np.random.randn(n_samples,
                    n_neurons)
    activations = []
    for _
    in range(n_layers):
        W =
        init_fn(n_neurons, n_neurons)
        X
        = activation_fn(X @ W)

```

4.4 Solving XOR

```
activations.append(X.ravel())
return activations

init_strategies = { 'Zeros':
lambda ni, no: np.zeros((ni, no)),
'Small Random': lambda ni, no:
np.random.randn(ni, no) * 0.01,
'Xavier': lambda ni, no:
np.random.randn(ni, no) *
np.sqrt(2.0 / (ni + no)), 'He':
lambda ni, no:
np.random.randn(ni, no) *
np.sqrt(2.0 / ni), }

fig, axes = plt.subplots(4, 5,
figsize=(16, 10)) strategy_colors
= [PALETTE['warm_gray'],
PALETTE['coral'],
PALETTE['royal_blue'],
PALETTE['emerald']]

for row, (name, init_fn) in
enumerate(init_strategies.items()):
np.random.seed(0) acts =
simulate_forward(init_fn,
tanh_act) for col in range(5): ax =
axes[row, col] data = acts[col] #
Clip for display stability data =
data[np.isfinite(data)] if len(data)
== 0 or np.std(data) < 1e-15:
ax.text(0.5, 0.5, 'Collapsed',
transform=ax.transAxes,
ha='center', va='center',
fontsize=9,
color=PALETTE['coral']) else:
ax.hist(data, bins=50,
color=strategy_colors[row],
alpha=0.75, edgecolor='white',
linewidth=0.3) ax.set_xlim(-1.1,
1.1) ax.set_yticks([]) if row == 0:
ax.set_title(f'Layer {col + 1}',
fontsize=10) if col == 0:
ax.set_ylabel(name, fontsize=10,
fontweight='bold',
color=PALETTE['deep_blue'])

fig.suptitle('Activation
Distributions Across Layers (tanh
activation)', fontsize=14,
fontweight='bold',
color=PALETTE['deep_blue'],
y=1.01) plt.tight_layout()
plt.show() ``
```

Observations: - Zeros: All
activations collapse to zero

4.4 Solving XOR

immediately; learning is impossible. - **Small Random** ($\sigma = 0.01$): Activations quickly shrink to near-zero, meaning gradients also vanish. - **Xavier**: Activations maintain a healthy spread across layers with tanh, because the variance is calibrated to preserve signal. - **He**: Slightly wider spread than Xavier (designed for ReLU), but still healthy with tanh.

4.6 Hidden Layer Size Experiment

How does the number of hidden neurons affect the decision boundary? More neurons = more capacity = more complex boundaries.

```
# ----- Generate a non-linear dataset (moons) -----

def make_moons(n_samples=300, noise=0.2, seed=42):
    """Generate two interleaving half-moons."""
    rng = np.random.RandomState(seed)
    n = n_samples // 2
    theta1 = np.linspace(0, np.pi, n)
    theta2 = np.linspace(0, np.pi, n)
    X1 = np.c_[np.cos(theta1), np.sin(theta1)] + rng.randn(n, 2)
    * noise
    X2 = np.c_[1 - np.cos(theta2), 1 - np.sin(theta2) - 0.5] +
    rng.randn(n, 2) * noise
    X = np.vstack([X1, X2])
    y = np.hstack([np.zeros(n), np.ones(n)])
    return X, y

X_moons, y_moons = make_moons(300, noise=0.20)

hidden_sizes = [2, 4, 8, 16, 32, 64]

fig, axes = plt.subplots(2, 3, figsize=(14, 9))

for idx, h_size in enumerate(hidden_sizes):
    row, col = divmod(idx, 3)
    ax = axes[row, col]

    np.random.seed(42)
    net = SimpleNeuralNetwork(2, h_size, 1, activation='relu')
    for _ in range(2000):
        net.train_step(X_moons, y_moons, lr=0.3)

    plot_decision_boundary(ax, net, X_moons, y_moons,
```

```

        title=f'{h_size} hidden neurons')
acc = np.mean(net.predict(X_moons) == y_moons) * 100
ax.text(0.98, 0.02, f'{acc:.0f}% acc', transform=ax.transAxes,
        ha='right', va='bottom', fontsize=9,
        color=PALETTE['deep_blue'], fontweight='bold',
        bbox=dict(boxstyle='round,pad=0.3', fc='white',
        alpha=0.8))

fig.suptitle('Decision Boundaries vs. Hidden Layer Width',
            fontsize=14, fontweight='bold',
            color=PALETTE['deep_blue'], y=1.01)
plt.tight_layout()
plt.show()

```

4.7 Activation Function Comparison

4.8 How a Hidden Layer Warps Space

A hidden layer applies an **affine transformation** ($(Wx + b)$) followed by a **non-linear squash** ((ϕ)). The combined effect is to *warp* the input space so that previously tangled data becomes linearly separable.

Below we visualize this for the moons dataset: we pass a dense grid through the hidden layer and see how the space is reshaped.

```

```python # Use the ReLU network
from section 4.7 (16 hidden neurons)
net_relu = trained_nets['relu']

Create a grid in input space gx =
np.linspace(X_moons[:, 0].min() -
0.5, X_moons[:, 0].max() + 0.5, 30)
gy = np.linspace(X_moons[:, 1].min()
- 0.5, X_moons[:, 1].max() + 0.5, 30)
gxx, gyy = np.meshgrid(gx, gy)
grid_pts = np.c_[gxx.ravel(),
gyy.ravel()]

Forward pass to get hidden
activations net_relu.forward(grid_pts)
H_grid = net_relu.cache['H'] #
(n_grid, 16)

Also get hidden activations for the
data points
net_relu.forward(X_moons) H_data =
net_relu.cache['H']

Use PCA to project 16D hidden
space to 2D for visualization #
(simple PCA: center + SVD) H_all =

```

---

## ## 4.7 Activation Function Comparison

---

```
np.vstack([H_grid, H_data]) H_mean = H_all.mean(axis=0) H_centered = H_all - H_mean U, S, Vt = np.linalg.svd(H_centered, full_matrices=False) H_proj = H_centered @ Vt[:2].T # project onto top-2 PCs
```

```
H_grid_2d = H_proj[:len(grid_pts)]
H_data_2d = H_proj[len(grid_pts):]
```

```
fig, axes = plt.subplots(1, 2,
figsize=(13, 5.5))
```

```
— Input space —
```

```
axes[0].scatter(grid_pts[:, 0],
grid_pts[:, 1],
c=PALETTE['light_gray'], s=4,
alpha=0.4, zorder=1) for label,
marker, color in [(0, 'o',
PALETTE['steel_blue']), (1, 's',
PALETTE['coral'])]: mask =
y_moons == label
axes[0].scatter(X_moons[mask, 0],
X_moons[mask, 1], marker=marker,
c=color, s=40, edgecolors='white',
linewidths=0.8, zorder=5)
axes[0].set_title('Input Space',
fontsize=12) axes[0].set_xlabel('\n(x_1)'); axes[0].set_ylabel('\n(x_2)')
```

```
— Feature (hidden) space —
```

```
axes[1].scatter(H_grid_2d[:, 0],
H_grid_2d[:, 1],
c=PALETTE['light_gray'], s=4,
alpha=0.4, zorder=1) for label,
marker, color in [(0, 'o',
PALETTE['steel_blue']), (1, 's',
PALETTE['coral'])]: mask =
y_moons == label
axes[1].scatter(H_data_2d[mask, 0],
H_data_2d[mask, 1], marker=marker,
c=color, s=40, edgecolors='white',
linewidths=0.8, zorder=5)
axes[1].set_title('Hidden Layer
Feature Space (PCA)', fontsize=12)
axes[1].set_xlabel('PC 1');
axes[1].set_ylabel('PC 2')
```

```
Draw an arrow between the two
panels fig.text(0.50, 0.50, r'\n(\phi(W_1 x + b_1))', fontsize=13,
ha='center', va='center',
color=PALETTE['deep_blue'],
fontweight='bold',
```

---

## ## 4.7 Activation Function Comparison

---

```
bbox=dict(boxstyle='rarrow,pad=0.4',
fc=PALETTE['light_blue'],
alpha=0.5))

fig.suptitle('How a Hidden Layer
Warps Space', fontsize=14,
fontweight='bold',
color=PALETTE['deep_blue'],
y=1.02) plt.tight_layout() plt.show()
``
```

The hidden layer has learned to **pull the two moons apart** into distinct regions of feature space. The output layer then just needs to draw a single linear boundary in this warped space. This is the geometric essence of representation learning.

---

## 4.9 Exercises

### Exercise 1: Dead ReLU Exploration

Modify the weight initialization experiment (Section 4.5) to use ReLU instead of tanh. With very large initial weights, observe the **dead neuron** phenomenon: many neurons output exactly zero after the first layer and never recover. Count the fraction of dead neurons per layer for different initialization scales ( $\sigma = 0.01, 0.1, 1.0, 5.0$ ).

### Exercise 2: Three-Class Spiral

Generate a three-class spiral dataset (three interleaving spirals). Train a single-hidden-layer network with softmax output to classify it. You will need to: 1. Extend the `SimpleNeuralNetwork` class to support multi-class output with softmax and cross-entropy loss. 2. Experiment with hidden layer sizes (try 32, 64, 128) and plot the resulting decision boundaries. 3. Discuss: does a single hidden layer suffice, or does the spiral problem benefit from depth?

### Exercise 3: Batch Normalization Preview

Re-run the weight initialization experiment (Section 4.5) but add **batch normalization** after each linear layer (before the activation). Batch norm normalizes each layer's pre-activations to zero mean and unit variance:

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Show that with batch norm, even the “small random” and “zeros” strategies produce healthy activation distributions. Why does this make initialization less critical?

---

**Chapter summary:** We built a single-hidden-layer neural network from scratch and used it to understand the core ideas: non-linear activations, weight initialization, and the geometric intuition that hidden layers warp input space to make data linearly separable. In the next chapter we will tackle **backpropagation** in full detail, deriving every gradient and training deeper networks.

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
apply_style()
np.random.seed(42)
```

## Chapter 5: Backpropagation

Backpropagation is the **engine of deep learning** – the algorithm that makes it possible to train neural networks with millions of parameters. Without it, deep learning as we know it would not exist.

In this chapter, we will build a complete understanding of backpropagation from first principles:

### Learning Objectives:

1. **The Chain Rule** – Understand the calculus that makes backpropagation possible, for both single-variable and multivariate functions.
2. **Computational Graphs** – Visualize neural networks as directed acyclic graphs where forward and backward passes flow through nodes.
3. **Backprop by Hand** – Walk through every single computation in a tiny network to build deep intuition.
4. **Modular Implementation** – Build a Layer/Linear/ReLU/Sigmoid/Sequential system from scratch that automates backpropagation.
5. **Gradient Checking** – Verify our analytical gradients against numerical approximations to ensure correctness.

---

### ## 5.1 The Chain Rule

---

#### ### 5.2 Computational Graphs

A **computational graph** is a directed acyclic graph (DAG) that represents a mathematical expression: - **Nodes** represent operations (add, multiply, sigmoid, etc.) - **Edges** carry values (tensors) between operations

Every neural network computation can be expressed as a computational graph. This gives us two key algorithms:

1. **Forward pass:** Compute values from inputs to output, following the topological order.
2. **Backward pass:**

---

## ## 5.1 The Chain Rule

---

Propagate gradients from output to inputs, following the reverse topological order.

### Example:  $f = (x + y) \cdot z$

Let  $x = -2$ ,  $y = 5$ ,  $z = -4$ : -  
Forward:  $q = x + y = 3$ , then  $f = q \cdot z = -12$  - Backward:  $\frac{\partial f}{\partial f} = 1$ ,  $\frac{\partial f}{\partial q} = z = -4$ ,  $\frac{\partial f}{\partial z} = q = 3$ ,  $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot 1 = -4$ ,  $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot 1 = -4$

```
```python # Forward pass
computation x, y, z = -2, 5, -4 q = x +
y # q = 3 f = q * z # f = -12
```

```
# Backward pass (manual) df_df =
1.0 df_dq = z # -4 (partial of qz w.r.t.
q) df_dz = q # 3 (partial of qz w.r.t.
z) df_dx = df_dq * 1 # -4 (partial of
x+y w.r.t. x is 1) df_dy = df_dq * 1 #
-4 (partial of x+y w.r.t. y is 1)
```

```
print("Forward pass:") print(f" x =
{x}, y = {y}, z = {z}") print(f" q = x
+ y = {q}") print(f" f = q * z = {f}")
print() print("Backward pass:")
print(f" df/df = {df_df}") print(f" df/
dq = z = {df_dq}") print(f" df/dz = q
= {df_dz}") print(f" df/dx = df/dq *
dq/dx = {df_dq} * 1 = {df_dx}")
print(f" df/dy = df/dq * dq/dy =
{df_dq} * 1 = {df_dy}") ````
```

```
```python # Visualization:
Computational graph for f = (x + y) *
z fig, ax = plt.subplots(figsize=(11,
5.5)) ax.set_xlim(-0.5, 10.5)
ax.set_ylim(-1, 6)
ax.set_aspect('equal') ax.axis('off')

import matplotlib.patches as
mpatches from matplotlib.patches
import FancyBboxPatch,
FancyArrowPatch

— Node positions — # Variables
(circles) var_positions = {'x': (0.5,
4), 'y': (0.5, 2), 'z': (0.5, 0)} #
Operations (rounded rectangles)
op_positions = {'+': (3.5, 3), '*': (7,
1.5)} # Intermediate / output (circles)
```

---

## ## 5.1 The Chain Rule

---

```
val_positions = {'q': (5.3, 3), 'f':
(9.5, 1.5)}

Draw variable nodes (circles) for
name, (cx, cy) in
val_positions.items(): circle =
plt.Circle((cx, cy), 0.45, fill=True,
facecolor=PALETTE['light_blue'],
edgecolor=PALETTE['deep_blue'],
linewidth=2, zorder=5)
ax.add_patch(circle) ax.text(cx, cy,
f'\({name}\)', ha='center',
va='center', fontsize=14,
fontweight='bold',
color=PALETTE['deep_blue'],
zorder=6)

Draw operation nodes (rounded
rectangles) for name, (cx, cy) in
op_positions.items(): rect =
FancyBboxPatch((cx - 0.45, cy -
0.4), 0.9, 0.8,
boxstyle="round,pad=0.1",
facecolor=PALETTE['paper'],
edgecolor=PALETTE['deep_blue'],
linewidth=2, zorder=5)
ax.add_patch(rect) ax.text(cx, cy,
name, ha='center', va='center',
fontsize=16, fontweight='bold',
color=PALETTE['deep_blue'],
zorder=6)

Draw result nodes (circles) for
name, (cx, cy) in
val_positions.items(): circle =
plt.Circle((cx, cy), 0.45, fill=True,
facecolor=PALETTE['light_blue'],
edgecolor=PALETTE['deep_blue'],
linewidth=2, zorder=5)
ax.add_patch(circle) ax.text(cx, cy,
f'\({name}\)', ha='center',
va='center', fontsize=14,
fontweight='bold',
color=PALETTE['deep_blue'],
zorder=6)

— Arrows (edges) — arrow_kw =
dict(arrowstyle='->',
mutation_scale=20, linewidth=2,
color=PALETTE['slate'], zorder=3)

x -> + ax.annotate('x', xy=(3.05,
3.2), xytext=(0.95, 3.8),
arrowprops=arrow_kw) # y -> +
ax.annotate('y', xy=(3.05, 2.8),
xytext=(0.95, 2.2),
```



---

## ## 5.1 The Chain Rule

---

```
arrowprops=arrow_kw) # + -> q
ax.annotate(' ', xy=(4.85, 3),
xytext=(3.95, 3),
arrowprops=arrow_kw) # q ->
ax.annotate(' ', xy=(6.55, 2.0),
xytext=(5.75, 2.7),
arrowprops=arrow_kw) # z ->
ax.annotate(' ', xy=(6.55, 1.0),
xytext=(0.95, 0.2),
arrowprops=arrow_kw) # * -> f
ax.annotate(' ', xy=(9.05, 1.5),
xytext=(7.45, 1.5),
arrowprops=arrow_kw)

— Forward values (blue
annotations above edges) — fwd_kw
= dict(fontsize=11,
fontweight='bold',
color=PALETTE['royal_blue'],
bbox=dict(boxstyle='round,pad=0.2',
facecolor='white',
edgecolor=PALETTE['royal_blue'],
alpha=0.9))

ax.text(1.8, 3.85, '\\(-2\\)',
ha='center', fwd_kw) ax.text(1.8,
2.15, '\\(5\\)', ha='center', fwd_kw)
ax.text(4.4, 3.35, '\\(3\\)', ha='center',
fwd_kw) ax.text(3.5, 0.35, '\\(-4\\)',
ha='center', fwd_kw) ax.text(6.1,
2.65, '\\(3\\)', ha='center', fwd_kw)
ax.text(8.3, 1.85, '\\(-12\\)',
ha='center', fwd_kw)

— Backward gradients (coral
annotations below edges) —
bwd_kw = dict(fontsize=10,
fontweight='bold',
color=PALETTE['coral'],
bbox=dict(boxstyle='round,pad=0.2',
facecolor='white',
edgecolor=PALETTE['coral'],
alpha=0.9))

ax.text(1.8, 3.25, '\\(\\frac{\\partial f}{\\partial x}=-4\\)', ha='center',
bwd_kw) ax.text(1.8, 1.55, '\\(\\frac{\\partial f}{\\partial y}=-4\\)',
ha='center', bwd_kw) ax.text(4.4,
2.45, '\\(-4\\)', ha='center', bwd_kw)
ax.text(3.5, -0.35, '\\(\\frac{\\partial f}{\\partial z}=3\\)', ha='center',
bwd_kw) ax.text(6.1, 1.85, '\\(-4\\)',
ha='center', bwd_kw) ax.text(8.3,
1.05, '\\(1\\)', ha='center', bwd_kw)
```

---

## ## 5.1 The Chain Rule

---

```
Legend ax.text(5.0, 5.5,
'Computational Graph: \((f = (x + y) \cdot z)\)', ha='center', fontsize=15,
fontweight='bold',
color=PALETTE['deep_blue'])
ax.text(0.5, -0.8, 'Forward values',
fontsize=11, fontweight='bold',
color=PALETTE['royal_blue'])
ax.text(3.5, -0.8, 'Backward
gradients', fontsize=11,
fontweight='bold',
color=PALETTE['coral'])
plt.tight_layout() plt.show() ``
```

### Key observations from the computational graph:

1. **Local computation:** Each node only needs to know its own local gradient rule. The + node passes gradients through unchanged; the \* node swaps and multiplies. 2.

**Gradient flow:** Gradients flow backward (right to left), accumulating via multiplication at each step – this is the chain rule in action. 3. **Reuse:** The forward pass values ( $q = 3$ ), ( $z = -4$ ) are needed during the backward pass. This is why we **cache** intermediate results.

---

## 5.3 Backprop by Hand: A Tiny Example

Let us walk through **every computation** in a tiny neural network. This is the most important exercise in this entire book.

### Network Architecture

- **Input:** 2 neurons ( $i_1 = 0.05$ ), ( $i_2 = 0.10$ )
- **Hidden layer:** 2 neurons with **sigmoid** activation
- **Output layer:** 1 neuron with **sigmoid** activation
- **Loss:** MSE with target ( $t = 0.01$ )

### Weights

$$[W_1 = \begin{bmatrix} 0.15 & 0.20 \\ 0.25 & 0.30 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0.35 & 0.35 \end{bmatrix}]$$

$$[W_2 = \begin{bmatrix} 0.40 & 0.45 \end{bmatrix}, \quad b_2 = \begin{bmatrix} 0.60 \end{bmatrix}]$$

We will compute: 1. **Forward pass**: Every intermediate value 2. **Backward pass**: Every gradient, step by step

```
=====
5.3 Backprop by Hand
=====

Inputs and target
i1, i2 = 0.05, 0.10
target = 0.01
X_tiny = np.array([[i1, i2]])
y_tiny = np.array([[target]])

Weights
W1 = np.array([[0.15, 0.20],
 [0.25, 0.30]])
b1 = np.array([[0.35, 0.35]])
W2 = np.array([[0.40],
 [0.45]])
b2 = np.array([[0.60]])

--- FORWARD PASS ---
print("=" * 60)
print("FORWARD PASS")
print("=" * 60)

Hidden layer pre-activation
z1 = X_tiny @ W1 + b1
print(f"\nHidden pre-activation (z1 = X @ W1 + b1):")
print(f" z1_1 = {i1}*{W1[0,0]} + {i2}*{W1[1,0]} + {b1[0,0]} = {z1[0,0]:.6f}")
print(f" z1_2 = {i1}*{W1[0,1]} + {i2}*{W1[1,1]} + {b1[0,1]} = {z1[0,1]:.6f}")

Hidden layer activation (sigmoid)
def sigmoid(x):
 return 1.0 / (1.0 + np.exp(-x))

a1 = sigmoid(z1)
print(f"\nHidden activation (a1 = sigmoid(z1)):")
print(f" a1_1 = sigmoid({z1[0,0]:.6f}) = {a1[0,0]:.6f}")
print(f" a1_2 = sigmoid({z1[0,1]:.6f}) = {a1[0,1]:.6f}")

Output layer pre-activation
z2 = a1 @ W2 + b2
print(f"\nOutput pre-activation (z2 = a1 @ W2 + b2):")
print(f" z2 = {a1[0,0]:.6f}*{W2[0,0]} + {a1[0,1]:.6f}*{W2[1,0]} + {b2[0,0]} = {z2[0,0]:.6f}")

Output activation (sigmoid)
a2 = sigmoid(z2)
print(f"\nOutput activation (a2 = sigmoid(z2)):")
print(f" a2 = sigmoid({z2[0,0]:.6f}) = {a2[0,0]:.6f}")
```

```

Loss (MSE)
loss = 0.5 * (a2[0,0] - target) ** 2 # using 1/2 * MSE for
 cleaner gradients
print(f"\nLoss (L = 0.5 * (a2 - target)^2):")
print(f" L = 0.5 * ({a2[0,0]:.6f} - {target})^2 = {loss:.6f}")

Summary table
print(f"\n{'Variable':<12} {'Value':>12}")
print("-" * 26)
print(f"{'i1':<12} {i1:>12.6f}")
print(f"{'i2':<12} {i2:>12.6f}")
print(f"{'z1_1':<12} {z1[0,0]:>12.6f}")
print(f"{'z1_2':<12} {z1[0,1]:>12.6f}")
print(f"{'a1_1':<12} {a1[0,0]:>12.6f}")
print(f"{'a1_2':<12} {a1[0,1]:>12.6f}")
print(f"{'z2':<12} {z2[0,0]:>12.6f}")
print(f"{'a2':<12} {a2[0,0]:>12.6f}")
print(f"{'Loss':<12} {loss:>12.6f}")

--- BACKWARD PASS ---
print("=" * 60)
print("BACKWARD PASS")
print("=" * 60)

Step 1: dL/da2
dL_da2 = a2[0,0] - target
print(f"\nStep 1: dL/da2 = a2 - target = {a2[0,0]:.6f} - {target}
 = {dL_da2:.6f}")

Step 2: da2/dz2 (sigmoid derivative)
da2_dz2 = a2[0,0] * (1 - a2[0,0])
print(f"Step 2: da2/dz2 = a2*(1-a2) = {a2[0,0]:.6f}*(1-
 {a2[0,0]:.6f}) = {da2_dz2:.6f}")

Step 3: dL/dz2 (chain rule)
dL_dz2 = dL_da2 * da2_dz2
print(f"Step 3: dL/dz2 = dL/da2 * da2/dz2 = {dL_da2:.6f} *
 {da2_dz2:.6f} = {dL_dz2:.6f}")

Step 4: Gradients for W2 and b2
dL_dW2_0 = dL_dz2 * a1[0,0]
dL_dW2_1 = dL_dz2 * a1[0,1]
dL_db2 = dL_dz2
print(f"\nStep 4: Output layer parameter gradients")
print(f" dL/dW2[0] = dL/dz2 * a1_1 = {dL_dz2:.6f} *
 {a1[0,0]:.6f} = {dL_dW2_0:.6f}")
print(f" dL/dW2[1] = dL/dz2 * a1_2 = {dL_dz2:.6f} *
 {a1[0,1]:.6f} = {dL_dW2_1:.6f}")
print(f" dL/db2 = dL/dz2 = {dL_db2:.6f}")

Step 5: dL/da1 (backprop through W2)
dL_da1_1 = dL_dz2 * W2[0,0]
dL_da1_2 = dL_dz2 * W2[1,0]

```

```

print(f"\nStep 5: Hidden activation gradients")
print(f" dL/da1_1 = dL/dz2 * W2[0] = {dL_dz2:.6f} * {W2[0,0]} = {dL_da1_1:.6f}")
print(f" dL/da1_2 = dL/dz2 * W2[1] = {dL_dz2:.6f} * {W2[1,0]} = {dL_da1_2:.6f}")

Step 6: da1/dz1 (sigmoid derivative)
da1_dz1_1 = a1[0,0] * (1 - a1[0,0])
da1_dz1_2 = a1[0,1] * (1 - a1[0,1])
print(f"\nStep 6: Hidden sigmoid derivatives")
print(f" da1_1/dz1_1 = {a1[0,0]:.6f} * (1 - {a1[0,0]:.6f}) = {da1_dz1_1:.6f}")
print(f" da1_2/dz1_2 = {a1[0,1]:.6f} * (1 - {a1[0,1]:.6f}) = {da1_dz1_2:.6f}")

Step 7: dL/dz1
dL_dz1_1 = dL_da1_1 * da1_dz1_1
dL_dz1_2 = dL_da1_2 * da1_dz1_2
print(f"\nStep 7: dL/dz1 (chain rule)")
print(f" dL/dz1_1 = {dL_da1_1:.6f} * {da1_dz1_1:.6f} = {dL_dz1_1:.6f}")
print(f" dL/dz1_2 = {dL_da1_2:.6f} * {da1_dz1_2:.6f} = {dL_dz1_2:.6f}")

Step 8: Gradients for W1 and b1
dL_dW1 = np.array([[i1 * dL_dz1_1, i1 * dL_dz1_2],
 [i2 * dL_dz1_1, i2 * dL_dz1_2]])
dL_db1 = np.array([[dL_dz1_1, dL_dz1_2]])

print(f"\nStep 8: Hidden layer parameter gradients")
print(f" dL/dW1 = ")
print(f" [{i1}*{dL_dz1_1:.6f}, {i1}*{dL_dz1_2:.6f}]
 [{dL_dW1[0,0]:.8f}, {dL_dW1[0,1]:.8f}])")
print(f" [{i2}*{dL_dz1_1:.6f}, {i2}*{dL_dz1_2:.6f}] =
 [{dL_dW1[1,0]:.8f}, {dL_dW1[1,1]:.8f}])")
print(f" dL/db1 = [{dL_db1[0,0]:.8f}, {dL_db1[0,1]:.8f}])")

```

## Summary of All Partial Derivatives

Derivative	Formula	Value
$\frac{\partial L}{\partial a_2}$	$\frac{\partial L}{\partial a_2}$	see above
$\frac{\partial a_2}{\partial z_2}$	$a_2(1 - a_2)$	see above
$\frac{\partial L}{\partial z_2}$	$\frac{\partial L}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2}$	see above
$\frac{\partial L}{\partial W_2}$	$\frac{\partial L}{\partial z_2} \cdot a_1^T$	see above

Derivative	Formula	Value
$\frac{\partial L}{\partial b_2}$	$\frac{\partial L}{\partial z_2}$	see above
$\frac{\partial L}{\partial a_1}$	$\frac{\partial L}{\partial z_2} \cdot W_2^T$	see above
$\frac{\partial a_1}{\partial z_1}$	$a_1 \cdot (1 - a_1)$	see above
$\frac{\partial L}{\partial z_1}$	$\frac{\partial L}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1}$	see above
$\frac{\partial L}{\partial W_1}$	$X^T \cdot \frac{\partial L}{\partial z_1}$	see above
$\frac{\partial L}{\partial b_1}$	$\frac{\partial L}{\partial z_1}$	see above

**The pattern is clear:** at each layer, we compute  $\frac{\partial L}{\partial z}$  (the “delta”), then use it to get parameter gradients and propagate to the previous layer.

*# Verify our hand-computed gradients with numerical differentiation*

```
def tiny_network_loss(W1, b1, W2, b2, X, target):
```

```
 z1 = X @ W1 + b1
```

```
 a1 = sigmoid(z1)
```

```
 z2 = a1 @ W2 + b2
```

```
 a2 = sigmoid(z2)
```

```
 return 0.5 * (a2[0,0] - target) ** 2
```

*# Numerical gradient for W2*

```
eps = 1e-5
```

```
numerical_dW2 = np.zeros_like(W2)
```

```
for i in range(W2.shape[0]):
```

```
 for j in range(W2.shape[1]):
```

```
 W2_plus = W2.copy(); W2_plus[i,j] += eps
```

```
 W2_minus = W2.copy(); W2_minus[i,j] -= eps
```

```
 numerical_dW2[i,j] = (tiny_network_loss(W1, b1, W2_plus,
```

```
 b2, X_tiny, target) -
```

```
 tiny_network_loss(W1, b1, W2_minus,
```

```
 b2, X_tiny, target)) / (2 * eps)
```

```
analytical_dW2 = np.array([dL_dW2_0], [dL_dW2_1])
```

```
print("Gradient Verification (W2):")
```

```
print(f" Analytical: {analytical_dW2.ravel()}")
```

```
print(f" Numerical: {numerical_dW2.ravel()}")
```

```
print(f" Max error: {np.max(np.abs(analytical_dW2 -
 numerical_dW2)):.2e}")
```

*# Numerical gradient for W1*

```
numerical_dW1 = np.zeros_like(W1)
```

```
for i in range(W1.shape[0]):
```

```

 for j in range(W1.shape[1]):
 W1_plus = W1.copy(); W1_plus[i,j] += eps
 W1_minus = W1.copy(); W1_minus[i,j] -= eps
 numerical_dW1[i,j] = (tiny_network_loss(W1_plus, b1, W2,
 b2, X_tiny, target) -
 tiny_network_loss(W1_minus, b1, W2,
 b2, X_tiny, target)) / (2 * eps)

print(f"\nGradient Verification (W1):")
print(f" Analytical:\n {dL_dW1}")
print(f" Numerical:\n {numerical_dW1}")
print(f" Max error: {np.max(np.abs(dL_dW1 -
 numerical_dW1)):.2e}")
print("\nAll gradients verified!")

```

---

## ## 5.4 The General Algorithm

---

### ## 5.5 Modular Implementation

Let us build the modular backpropagation system from scratch. Each layer follows the same interface:

- forward(x) – compute output, cache values needed for backward
- backward(grad) – compute parameter gradients, return input gradient
- params() – return list of (parameter, gradient) tuples

``python #

=====

# Modular Neural Network from Scratch #

=====

```

class Layer: """Base class for all layers."""
 def forward(self, x): raise NotImplementedError
 def backward(self, grad): raise NotImplementedError
 def params(self): return []

class Linear(Layer): """Fully connected layer: y = x @ W + b"""
 def __init__(self, in_features, out_features):
 # He initialization scale = np.sqrt(2.0 / in_features)
 self.W = np.random.randn(in_features, out_features) * scale
 self.b = np.zeros((1, out_features))
 self.grad_W = None
 self.grad_b = None
 self._input = None # cached for backward

 def forward(self, x):
 self._input = x # cache input for backward pass
 return x @ self.W + self.b

 def backward(self, grad):
 # grad: dL/d(output), shape (N, out_features)
 # Param gradients
 self.grad_W = self._input.T @ grad # (in, N) @ (N, out) = (in, out)
 self.grad_b = np.sum(grad, axis=0, keepdims=True) # (1, out)
 # Input gradient to propagate
 return grad @ self.W.T # (N, out) @ (out, in) = (N, in)

 def params(self):
 return [(self.W, self.grad_W), (self.b, self.grad_b)]

class ReLU(Layer): """ReLU activation: y = max(0, x)"""
 def __init__(self):
 self._mask = None

 def forward(self, x):
 self._mask = (x > 0).astype(float) # binary mask: 1 where x > 0
 return x * self._mask

 def backward(self, grad):
 return grad * self._mask # gradient flows only where x was positive

```

---

## ## 5.4 The General Algorithm

---

```
class Sigmoid(Layer): """Sigmoid activation: $y = 1 / (1 + \exp(-x))$ """
def init(self): self._output = None
def forward(self, x): self._output = 1.0 / (1.0 + np.exp(-np.clip(x, -500, 500))) return self._output
def backward(self, grad): # sigmoid'(x) = sigmoid(x) * (1 - sigmoid(x)) return grad * self._output * (1 - self._output)
class Sequential: """Chains layers together, automating forward and backward passes."""
def init(self, *layers): self.layers = list(layers)
def forward(self, x): for layer in self.layers: x = layer.forward(x) return x
def backward(self, grad): for layer in reversed(self.layers): grad = layer.backward(grad) return grad
def params(self): p = [] for layer in self.layers: p.extend(layer.params()) return p
def predict(self, x): out = self.forward(x) return (out > 0.5).astype(int).ravel()
print("Layer classes defined:") print(" Layer – base class") print(" Linear – $y = x @ W + b$ ") print(" ReLU – $y = \max(0, x)$ ") print(" Sigmoid – $y = 1/(1+\exp(-x))$ ") print(" Sequential – chains layers") ``
``python # Quick verification: rebuild the tiny network from Section 5.3
np.random.seed(42) tiny_model = Sequential(Linear(2, 2), Sigmoid(), Linear(2, 1), Sigmoid())
Manually set weights to match our hand computation tiny_model.layers[0].W = W1.copy() tiny_model.layers[0].b = b1.copy() tiny_model.layers[2].W = W2.copy() tiny_model.layers[2].b = b2.copy()
Forward output = tiny_model.forward(X_tiny) loss_verify = 0.5 * (output[0,0] - target) ** 2 print(f"Model output: {output[0,0]:.6f} (hand-computed: {a2[0,0]:.6f})") print(f"Loss: {loss_verify:.6f} (hand-computed: {loss:.6f})")
Backward grad_output = (output - target) # dL/da2 for 0.5*MSE
tiny_model.backward(grad_output)
Compare gradients print(f" gradient (modular): {tiny_model.layers[2].grad_W.ravel()}") print(f"W2 gradient (by hand): {np.array([dL_dW2_0, dL_dW2_1])}") print(f"W1 gradient (modular):") print(f"W1 gradient (by hand):") print("implementation matches hand computation!") ``
``python # ===== # Train on the Moons Dataset #
===== from utils.data import make_moons, train_test_split
Generate data X_moons, y_moons = make_moons(n=500, noise=0.15, seed=42)
X_train, X_test, y_train, y_test = train_test_split(X_moons, y_moons, test_ratio=0.2, seed=42)
Reshape targets for BCE y_train_col = y_train.reshape(-1, 1).astype(float)
y_test_col = y_test.reshape(-1, 1).astype(float)
print(f"Training set: {X_train.shape[0]} samples") print(f"Test set: {X_test.shape[0]} samples") print(f"Features: {X_train.shape[1]}") ``
``python # Build and train the model np.random.seed(42) model = Sequential(Linear(2, 16), ReLU(), Linear(16, 8), ReLU(), Linear(8, 1), Sigmoid())
```



---

## ## 5.4 The General Algorithm

---

```
Binary cross-entropy loss def bce_loss(y_pred, y_true): y_pred = np.clip(y_pred,
1e-8, 1 - 1e-8) return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 -
y_pred))

def bce_grad(y_pred, y_true): y_pred = np.clip(y_pred, 1e-8, 1 - 1e-8) N =
y_pred.shape[0] return (y_pred - y_true) / (y_pred * (1 - y_pred) * N)

Training loop lr = 0.1 epochs = 300 losses = [] accuracies = []

for epoch in range(epochs): # Forward pass y_pred = model.forward(X_train)
loss_val = bce_loss(y_pred, y_train_col) losses.append(loss_val)

Accuracy preds = (y_pred > 0.5).astype(int).ravel() acc = np.mean(preds ==
y_train) accuracies.append(acc)

Backward pass grad = bce_grad(y_pred, y_train_col) model.backward(grad)

SGD update for param, grad_param in model.params(): param -= lr * grad_param

if (epoch + 1) % 50 == 0: print(f'Epoch {epoch+1:3d}/{epochs} - Loss:
{loss_val:.4f}, Accuracy: {acc:.4f}')

Test accuracy y_test_pred = model.predict(X_test) test_acc = np.mean(y_test_pred
== y_test) print(f'Accuracy: {test_acc:.4f}')

```python # Visualize training results from utils.plot import plot_decision_boundary
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

# Loss curve axes[0].plot(losses, color=PALETTE['royal_blue'], linewidth=2)
axes[0].set_xlabel('Epoch') axes[0].set_ylabel('BCE Loss')
axes[0].set_title('Training Loss')

# Accuracy curve axes[1].plot(accuracies, color=PALETTE['emerald'], linewidth=2)
axes[1].set_xlabel('Epoch') axes[1].set_ylabel('Accuracy')
axes[1].set_title('Training Accuracy') axes[1].set_ylim(0.4, 1.05)

# Decision boundary plot_decision_boundary(model, X_test, y_test, ax=axes[2])
axes[2].set_title(f'Decision Boundary (Test Acc: {test_acc:.1%})')
axes[2].set_xlabel('(x_1)') axes[2].set_ylabel('(x_2)')

fig.suptitle('Backpropagation in Action: Training on Moons Dataset', fontsize=14,
fontweight='bold', color=PALETTE['deep_blue']) plt.tight_layout() plt.show() ```

It works! Our from-scratch implementation of backpropagation successfully trains a
multi-layer neural network to learn a non-linear decision boundary. The key insight:
by implementing forward() and backward() for each layer independently, the
Sequential container can chain them together automatically.
```

5.6 Gradient Checking

How do we know our gradients are correct? **Gradient checking** compares our analytical (backprop) gradients against numerical gradients computed with finite differences.

Numerical Gradient

For each parameter θ_i , the numerical gradient is:

$$\left| \frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + \epsilon) - L(\theta_i - \epsilon)}{2\epsilon} \right|$$

This two-sided finite difference is $\mathcal{O}(\epsilon^2)$ accurate, much better than the one-sided version.

Relative Error Metric

We measure agreement with the relative error:

$$\text{error} = \frac{|g_{\text{analytical}} - g_{\text{numerical}}|}{\max(|g_{\text{analytical}}|, |g_{\text{numerical}}|, 10^{-8})}$$

- Error $< 10^{-7}$: **Excellent** – gradients are correct
- Error $< 10^{-5}$: **Acceptable** for most applications
- Error $> 10^{-3}$: **Bug alert** – something is wrong

```
def gradient_check(model, X, y, loss_fn, loss_grad_fn, eps=1e-5):
    """
    Compare analytical gradients (backprop) with numerical
    gradients.
    Returns max relative error and lists of all gradient pairs.
    """
    # Forward + backward to get analytical gradients
    y_pred = model.forward(X)
    loss = loss_fn(y_pred, y)
    grad = loss_grad_fn(y_pred, y)
    model.backward(grad)

    all_analytical = []
    all_numerical = []
    max_error = 0.0

    for param, grad_param in model.params():
        it = np.nditer(param, flags=['multi_index'])
        while not it.finished:
            idx = it.multi_index
            old_val = param[idx]

            # f(theta + eps)
            param[idx] = old_val + eps
            loss_plus = loss_fn(model.forward(X), y)

            # f(theta - eps)
            param[idx] = old_val - eps
            loss_minus = loss_fn(model.forward(X), y)

            # Restore
            param[idx] = old_val

            numerical_grad = (loss_plus - loss_minus) / (2 * eps)
            analytical_grad = grad_param[idx]
```

```

        all_analytical.append(analytical_grad)
        all_numerical.append(numerical_grad)

        denom = max(abs(numerical_grad),
abs(analytical_grad), 1e-8)
        error = abs(numerical_grad - analytical_grad) / denom
        max_error = max(max_error, error)

    it.iternext()

    return max_error, np.array(all_analytical),
        np.array(all_numerical)

# Build a fresh small model for gradient checking
np.random.seed(42)
check_model = Sequential(
    Linear(2, 4),
    ReLU(),
    Linear(4, 3),
    ReLU(),
    Linear(3, 1),
    Sigmoid()
)

# Use a small batch for checking
X_check = X_train[:5]
y_check = y_train_col[:5]

max_err, analytical_grads, numerical_grads = gradient_check(
    check_model, X_check, y_check, bce_loss, bce_grad
)

print(f"Number of parameters checked: {len(analytical_grads)}")
print(f"Maximum relative error: {max_err:.2e}")
if max_err < 1e-5:
    print("PASSED: Gradients are correct!")
else:
    print("WARNING: Gradient error is large -- check
        implementation!")

# Scatter plot: Numerical vs Analytical gradients
fig, ax = plt.subplots(figsize=(7, 7))

ax.scatter(numerical_grads, analytical_grads, alpha=0.6, s=40,
    color=PALETTE['royal_blue'], edgecolors='white',
    linewidth=0.5,
    zorder=3, label='Gradient pairs')

# y = x reference line
lims = [min(numerical_grads.min(), analytical_grads.min()) * 1.1,
    max(numerical_grads.max(), analytical_grads.max()) * 1.1]
ax.plot(lims, lims, '--', color=PALETTE['coral'], linewidth=2,
    label='$y = x$ (perfect agreement)', zorder=2)

```

```

ax.set_xlabel('Numerical Gradient', fontsize=12)
ax.set_ylabel('Analytical Gradient (Backprop)', fontsize=12)
ax.set_title('Gradient Checking: Analytical vs Numerical',
             fontsize=13,
             fontweight='bold', color=PALETTE['deep_blue'])
ax.legend(fontsize=11)
ax.set_aspect('equal')
ax.set_xlim(lims)
ax.set_ylim(lims)

# Annotation
ax.text(0.05, 0.92, f'Max relative error: {max_err:.2e}',
       transform=ax.transAxes, fontsize=11,
       bbox=dict(boxstyle='round,pad=0.4',
                 facecolor=PALETTE['light_blue'], alpha=0.7))

plt.tight_layout()
plt.show()

```

All points lie directly on the $(y = x)$ line, confirming that our backpropagation implementation computes the correct gradients. Gradient checking is an essential debugging tool – **always run it when implementing a new layer**.

5.7 Gradient Flow Visualization

5.8 Computational Graph Visualization (Hero Figure)

Let us create a detailed, publication-quality visualization of the full computational graph for our tiny example from Section 5.3. This figure shows every operation, forward value, and backward gradient.

```
``python #
```

```
=====
# Hero Figure: Full Computational Graph for the Tiny Network (Section 5.3) #
=====
```

```

fig, ax = plt.subplots(figsize=(18, 10)) ax.set_xlim(-1, 21) ax.set_ylim(-2.5, 9) ax.set_aspect('equal')
ax.axis('off')

from matplotlib.patches import FancyBboxPatch, Circle

# --- Colors --- c_input = PALETTE['light_blue'] c_op = '#F0F0F5' c_edge =
PALETTE['deep_blue'] c_fwd = PALETTE['royal_blue'] c_bwd = PALETTE['coral'] c_result =
PALETTE['light_blue']

# --- Reuse values from Section 5.3 --- # Forward v_i1, v_i2 = 0.05, 0.10 v_z1_1, v_z1_2 = z1[0,
0], z1[0, 1] v_a1_1, v_a1_2 = a1[0, 0], a1[0, 1] v_z2 = z2[0, 0] v_a2 = a2[0, 0] v_loss = loss
# Backward v_dL_da2 = dL_da2 v_dL_dz2 = dL_dz2 v_dL_da1_1 = dL_da1_1 v_dL_da1_2 =
dL_da1_2 v_dL_dz1_1 = dL_dz1_1 v_dL_dz1_2 = dL_dz1_2

# --- Helper functions --- def draw_var_node(ax, cx, cy, label, size=0.5): circle = Circle((cx, cy),
size, fill=True, facecolor=c_input, edgecolor=c_edge, linewidth=2, zorder=5) ax.add_patch(circle)
ax.text(cx, cy, label, ha='center', va='center', fontsize=11, fontweight='bold', color=c_edge,
zorder=6)

def draw_op_node(ax, cx, cy, label, w=1.2, h=0.8): rect = FancyBboxPatch((cx - w/2, cy - h/2), w, h,
boxstyle="round,pad=0.12", facecolor=c_op, edgecolor=c_edge, linewidth=2, zorder=5)
ax.add_patch(rect) ax.text(cx, cy, label, ha='center', va='center', fontsize=10, fontweight='bold',
color=c_edge, zorder=6)

```

5.7 Gradient Flow Visualization

```
def draw_arrow(ax, x1, y1, x2, y2): ax.annotate(' ', xy=(x2, y2), xytext=(x1, y1),
arrowprops=dict(arrowstyle='->', mutation_scale=15, linewidth=1.5, color=PALETTE['slate']),
zorder=3)

def fwd_label(ax, x, y, text): ax.text(x, y, text, ha='center', va='center', fontsize=8.5,
fontweight='bold', color=c_fwd, bbox=dict(boxstyle='round,pad=0.15', facecolor='white',
edgecolor=c_fwd, alpha=0.9), zorder=7)

def bwd_label(ax, x, y, text): ax.text(x, y, text, ha='center', va='center', fontsize=8,
fontweight='bold', color=c_bwd, bbox=dict(boxstyle='round,pad=0.15', facecolor='white',
edgecolor=c_bwd, alpha=0.9), zorder=7)

# ===== # Column 0: Inputs #
===== draw_var_node(ax, 0, 7, '\(i_1\)') draw_var_node(ax, 0,
3, '\(i_2\)') fwd_label(ax, 0, 8.0, '0.05') fwd_label(ax, 0, 1.9, '0.10')
# ===== # Column 1: Linear (W1, b1) ops #
===== draw_op_node(ax, 3.5, 6.5, '\(\Sigma\) + \(\mathbf{b}\)', w=1.4,
h=0.9) draw_op_node(ax, 3.5, 3.5, '\(\Sigma\) + \(\mathbf{b}\)', w=1.4, h=0.9)
# Arrows: inputs -> linear draw_arrow(ax, 0.5, 7, 2.8, 6.7) draw_arrow(ax, 0.5, 3, 2.8, 3.7)
draw_arrow(ax, 0.5, 6.8, 2.8, 3.9) draw_arrow(ax, 0.5, 3.2, 2.8, 6.3)
# Weight labels on arrows fwd_label(ax, 1.5, 7.2, 'w=0.15') fwd_label(ax, 1.5, 5.2, 'w=0.25')
fwd_label(ax, 1.5, 4.8, 'w=0.20') fwd_label(ax, 1.5, 2.8, 'w=0.30')
# Forward values after linear fwd_label(ax, 5.0, 7.2, f'z={v_z1_1:.4f}') fwd_label(ax, 5.0, 2.8,
f'z={v_z1_2:.4f}')
# ===== # Column 2: Sigmoid (hidden) #
===== draw_op_node(ax, 6.5, 6.5, '\(\sigma\)') w=1.0, h=0.8)
draw_op_node(ax, 6.5, 3.5, '\(\sigma\)') w=1.0, h=0.8)
draw_arrow(ax, 4.2, 6.5, 6.0, 6.5) draw_arrow(ax, 4.2, 3.5, 6.0, 3.5)
fwd_label(ax, 8.0, 7.2, f'a={v_a1_1:.4f}') fwd_label(ax, 8.0, 2.8, f'a={v_a1_2:.4f}')
# ===== # Column 3: Linear (W2, b2) op #
===== draw_op_node(ax, 10, 5, '\(\Sigma\) + \(\mathbf{b}\)', w=1.4,
h=0.9)
draw_arrow(ax, 7.0, 6.5, 9.3, 5.3) draw_arrow(ax, 7.0, 3.5, 9.3, 4.7)
fwd_label(ax, 8.3, 6.1, 'w=0.40') fwd_label(ax, 8.3, 3.9, 'w=0.45')
fwd_label(ax, 11.6, 5.7, f'z={v_z2:.4f}')
# ===== # Column 4: Sigmoid (output) #
===== draw_op_node(ax, 13.2, 5, '\(\sigma\)') w=1.0, h=0.8)
draw_arrow(ax, 10.7, 5, 12.7, 5)
fwd_label(ax, 14.7, 5.7, f'a={v_a2:.4f}')
# ===== # Column 5: Loss #
===== draw_op_node(ax, 16.5, 5, 'MSE', w=1.2, h=0.8)
draw_arrow(ax, 13.7, 5, 15.9, 5)
draw_var_node(ax, 16.5, 3, '\(t\)') size=0.45) draw_arrow(ax, 16.5, 3.45, 16.5, 4.6) fwd_label(ax,
16.5, 2.0, '0.01')
# Loss output draw_var_node(ax, 19, 5, '\(L\)') size=0.5) draw_arrow(ax, 17.1, 5, 18.5, 5)
fwd_label(ax, 19, 6.0, f'{v_loss:.6f}')
# ===== # Backward gradient annotations #
===== bwd_label(ax, 19, 4.0, '\(\frac{\partial L}{\partial L}\)')
=1\)) bwd_label(ax, 17.8, 3.8, f'\(\Delta={v_dL_da2:.4f}\)') bwd_label(ax, 15.0, 4.0, f'\(
```

5.7 Gradient Flow Visualization

```
\delta={v_dL_dz2:.6f}\') bwd_label(ax, 11.6, 4.0, f'\(\delta={v_dL_dz2:.6f}\') bwd_label(ax, 8.5,
5.5, f'\(\delta_1={v_dL_da1_1:.6f}\') bwd_label(ax, 8.5, 4.2, f'\(\delta_2={v_dL_da1_2:.6f}\')
bwd_label(ax, 5.0, 6.0, f'\(\delta={v_dL_dz1_1:.6f}\') bwd_label(ax, 5.0, 4.0, f'\(
\delta={v_dL_dz1_2:.6f}\')
# ===== # Title and legend #
===== ax.text(10, 8.5, 'Full Computational Graph: 2-2-1 Neural
Network', ha='center', fontsize=16, fontweight='bold', color=PALETTE['deep_blue']) ax.text(10,
8.0, 'Inputs: (i_1)=0.05, (i_2)=0.10) | Target: (t)=0.01 | Loss: MSE', ha='center', fontsize=11,
color=PALETTE['slate'])
# Legend boxes legend_y = -1.5 fwd_label(ax, 6, legend_y, 'Forward values (blue)') bwd_label(ax,
11, legend_y, 'Backward gradients (coral)') ax.text(16, legend_y, 'Arrows show data flow direction',
ha='center', va='center', fontsize=10, color=PALETTE['slate']),
bbox=dict(boxstyle='round,pad=0.3', facecolor=PALETTE['light_gray'], alpha=0.8))
plt.tight_layout() plt.show() ``
```

This hero figure captures the essence of backpropagation: **forward values flow left to right**, computing the loss, while **gradients flow right to left**, telling each parameter how to adjust to reduce the loss.

5.9 Exercises

Exercise 1: Extend the Tiny Example

Modify the tiny network from Section 5.3 to have **2 output neurons** instead of 1 (making it suitable for 2-class classification). Use the same input values and similar weights. Compute the full forward and backward pass by hand, then verify with the modular implementation.

Hints: - (W_2) becomes a $((2 \times 2))$ matrix - Use softmax + cross-entropy loss instead of sigmoid + MSE - The gradient at the output simplifies to $((\hat{y} - y_{\text{one-hot}}))$

Exercise 2: Implement a New Activation

Implement the **Tanh** activation as a `Layer` subclass: - Forward: $((\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}))$ - Backward: $((\frac{\partial}{\partial x} \tanh(x) = 1 - \tanh^2(x)))$

Then: 1. Run gradient checking to verify your implementation. 2. Train on the moons dataset and compare with ReLU and Sigmoid. 3. Visualize the gradient flow – where does Tanh fall on the vanishing gradient spectrum?

Exercise 3: Gradient Checkpointing

Implement a simple version of **gradient checkpointing** for the Sequential container: - During the forward pass, only store activations at every $((k))$ -th layer (the “checkpoints”). - During the backward pass, recompute the missing activations from the nearest checkpoint. - Compare memory usage (number of cached tensors) for $((k = 1))$ (standard) vs $((k = 2))$ and $((k = 4))$.

This technique is critical for training very large models (e.g., GPT-scale) where memory is the bottleneck.

Summary

In this chapter, we built a complete understanding of backpropagation:

Concept	Key Takeaway
Chain Rule	Decompose complex derivatives into products of simple local derivatives
Computational Graphs	Visualize computation as a DAG; forward computes values, backward propagates gradients
Backprop by Hand	Every gradient is a chain of local derivatives – no magic involved
General Algorithm	Cache forward values, then sweep backward computing parameter gradients
Modular Design	<code>forward()</code> + <code>backward()</code> per layer enables composable, maintainable code
Gradient Checking	Always verify analytical gradients against numerical ones – this catches bugs
Gradient Flow	Activation choice profoundly affects gradient propagation; ReLU enables deep networks

Backpropagation is not just an algorithm – it is the **computational backbone** that makes all of deep learning possible. In the next chapter, we will use this foundation to explore optimization algorithms that determine *how* we use these gradients to update parameters.

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
from utils.nn import Linear, ReLU, Sigmoid, Tanh, Sequential,
    MSELoss, BinaryCrossEntropyLoss, CrossEntropyLoss
from utils.data import make_moons, make_circles, make_spirals,
    make_blobs, train_test_split
from utils.plot import plot_decision_boundary, plot_loss_curve
apply_style()
np.random.seed(42)
```

Chapter 6: Multi-Layer Networks

In this chapter we go beyond a single hidden layer and build **deep** neural networks from scratch. We will see how stacking layers creates a hierarchy of learned features, implement a clean training loop, explore different loss functions, and tackle increasingly complex datasets.

Learning objectives:

1. Understand why depth matters – how stacking layers creates feature hierarchies
2. Implement a reusable training loop (forward, loss, backward, update)
3. Compare MSE, Binary Cross-Entropy, and Cross-Entropy loss functions
4. Train multi-layer networks on four synthetic datasets
5. Observe the effect of network depth on learning capacity
6. Diagnose and visualize the **vanishing gradient problem**
7. Explore width vs. depth tradeoffs
8. Perform multi-class classification with softmax + cross-entropy

6.1 Going Deep

In Chapter 5 we built a `Sequential` container that chains layers together. Now we put it to use by stacking **multiple** linear + activation blocks.

Why depth?

A single hidden layer can theoretically approximate any function (Universal Approximation Theorem), but it may require an **exponentially** wide layer to do so. Deep networks learn a **hierarchy of features**:

Layer	What it learns
1	Simple boundaries (lines, half-planes)
2	Compositions of boundaries (corners, curves)
3	Complex regions (spirals, rings)
4+	Fine-grained decision surfaces

Depth vs. Width

- **Wide & shallow:** many neurons per layer, few layers. Good at memorisation, less parameter-efficient.
- **Deep & narrow:** fewer neurons per layer, many layers. Better at learning composable features, but can be harder to train (vanishing gradients).

Building a deep network with Sequential

```
model = Sequential(  
    Linear(2, 32), ReLU(),  
    Linear(32, 32), ReLU(),
```



```

        Linear(32, 1), Sigmoid()
    )

```

Each Linear + ReLU block is one “layer” of representation. The final Sigmoid squashes the output to $\in [0, 1]$ for binary classification.

6.2 The Training Loop

Every neural network training procedure follows the same four-step cycle:

1. **Forward pass** – compute predictions
2. **Loss** – measure how wrong we are
3. **Backward pass** – compute gradients of the loss with respect to every parameter
4. **Parameter update** – nudge each parameter in the direction that reduces the loss

We encapsulate this in a reusable train function.

```

def train(model, loss_fn, X, y, lr=0.01, epochs=1000,
         verbose=True):
    """Full training loop: forward -> loss -> backward ->
       update."""
    losses = []
    for epoch in range(epochs):
        # 1. Forward pass
        y_pred = model.forward(X)
        # 2. Compute loss
        loss = loss_fn.forward(y_pred, y)
        # 3. Backward pass
        grad = loss_fn.backward()
        model.backward(grad)
        # 4. Parameter update (vanilla SGD)
        for param, grad_p in model.params():
            param -= lr * grad_p
        losses.append(loss)
        if verbose and (epoch + 1) % 200 == 0:
            print(f"Epoch {epoch+1:4d}/{epochs} Loss:
                  {loss:.4f}")
    return losses

```

The function returns the loss history so we can plot learning curves later. Let us verify it works on a quick example.

```

# Quick sanity check: train a small network on the moons dataset
np.random.seed(42)
X_demo, y_demo = make_moons(n=200, noise=0.15)

demo_model = Sequential(
    Linear(2, 16), ReLU(),
    Linear(16, 16), ReLU(),
    Linear(16, 1), Sigmoid()
)
demo_losses = train(demo_model, BinaryCrossEntropyLoss(), X_demo,
                    y_demo,

```

```

lr=0.1, epochs=600, verbose=True)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
plot_loss_curve(demo_losses, ax=axes[0], title="Training Loss
                (Moons demo)")
plot_decision_boundary(demo_model, X_demo, y_demo, ax=axes[1])
axes[1].set_title("Decision Boundary")
plt.tight_layout()
plt.show()

```

6.3 Loss Functions

The choice of loss function depends on the task:

Task	Loss	Output activation
Regression	MSELoss	None (identity)
Binary classification	BinaryCrossEntropyLoss	Sigmoid
Multi-class classification	CrossEntropyLoss	Softmax (built-in)

All three are implemented in `utils.nn`. Let us briefly review their formulas and see their gradients in action.

```

# --- MSELoss ---
mse = MSELoss()
y_pred_demo = np.array([[0.8], [0.3], [0.6]])
y_true_demo = np.array([[1.0], [0.0], [1.0]])

loss_val = mse.forward(y_pred_demo, y_true_demo)
grad_val = mse.backward()
print(f"MSELoss:  loss = {loss_val:.4f}")
print(f"  grad = {grad_val.ravel()}")

# --- BinaryCrossEntropyLoss ---
bce = BinaryCrossEntropyLoss()
loss_val = bce.forward(y_pred_demo, y_true_demo)
grad_val = bce.backward()
print(f"\nBCELoss:  loss = {loss_val:.4f}")
print(f"  grad = {grad_val.ravel()}")

# --- CrossEntropyLoss ---
ce = CrossEntropyLoss()
logits_demo = np.array([[2.0, 0.5, -1.0],
                        [0.1, 2.5, 0.3]])
labels_demo = np.array([0, 1]) # integer class labels

loss_val = ce.forward(logits_demo, labels_demo)
grad_val = ce.backward()
print(f"\nCrossEntropyLoss:  loss = {loss_val:.4f}")
print(f"  grad =\n{grad_val}")

```

Key points:

- **MSELoss**: Gradient is proportional to the error $(\hat{y} - y)$. Simple but can be slow to converge for classification.
- **BinaryCrossEntropyLoss**: Gradient pushes harder when the prediction is confidently wrong. Pairs with Sigmoid output.
- **CrossEntropyLoss**: Combines softmax and negative log-likelihood. The gradient is elegantly $(\text{softmax}(z) - \mathbf{e}_y)$, where $\mathbf{e}_y$ is the one-hot target. No separate Softmax layer needed.

6.4 Experiments on Synthetic Datasets

We now train 3-layer ReLU networks on four different synthetic datasets to see how multi-layer networks handle varying geometry: - **Moons** – two interleaving half-circles - **Circles** – concentric rings - **Spirals** – intertwined spirals (the hardest) - **Blobs** – separated clusters

```
# Generate datasets
datasets = {
    "Moons": make_moons(n=300, noise=0.15),
    "Circles": make_circles(n=300, noise=0.08),
    "Spirals": make_spirals(n=300, n_classes=2, noise=0.3),
    "Blobs": make_blobs(n=300, centers=2, std=0.8),
}

fig, axes = plt.subplots(2, 2, figsize=(11, 10))
axes = axes.ravel()

for idx, (name, (X, y)) in enumerate(datasets.items()):
    np.random.seed(42)
    model = Sequential(
        Linear(2, 32), ReLU(),
        Linear(32, 32), ReLU(),
        Linear(32, 32), ReLU(),
        Linear(32, 1), Sigmoid()
    )
    losses = train(model, BinaryCrossEntropyLoss(), X, y,
                    lr=0.1, epochs=1000, verbose=False)

    plot_decision_boundary(model, X, y, ax=axes[idx])
    axes[idx].set_title(f"{name} (final loss: {losses[-1]:.3f})")

fig.suptitle("3-Layer ReLU Networks on Four Datasets",
             fontsize=14, fontweight='bold',
             color=PALETTE['deep_blue'], y=1.02)
plt.tight_layout()
plt.show()
```

The 3-layer network handles moons, circles, and blobs easily. Spirals are harder – the decision boundary needs to curve tightly, which requires either more capacity or more training.

6.5 Depth Experiment

How does the number of layers affect learning? We fix the dataset (spirals – the most challenging) and vary the network depth from 1 to 5 hidden layers.

```
X_spiral, y_spiral = make_spirals(n=400, n_classes=2, noise=0.25)
```

```
def build_model(n_hidden_layers, width=32):
    """Build a binary classifier with variable depth."""
    layers = []
    in_dim = 2
    for _ in range(n_hidden_layers):
        layers.append(Linear(in_dim, width))
        layers.append(ReLU())
        in_dim = width
    layers.append(Linear(in_dim, 1))
    layers.append(Sigmoid())
    return Sequential(*layers)

depths = [1, 2, 3, 5]
depth_results = {}

for d in depths:
    np.random.seed(42)
    model = build_model(d)
    losses = train(model, BinaryCrossEntropyLoss(), X_spiral,
                    y_spiral,
                    lr=0.1, epochs=1500, verbose=False)
    depth_results[d] = (model, losses)

# Decision boundaries for each depth
fig, axes = plt.subplots(2, 2, figsize=(11, 10))
axes = axes.ravel()

for idx, d in enumerate(depths):
    model, losses = depth_results[d]
    plot_decision_boundary(model, X_spiral, y_spiral,
                           ax=axes[idx])
    axes[idx].set_title(f"{d} hidden layer{'s' if d > 1 else ''}"
                       f"(loss: {losses[-1]:.3f})")

fig.suptitle("Effect of Network Depth on Spiral Classification",
             fontsize=14, fontweight='bold',
             color=PALETTE['deep_blue'], y=1.02)
plt.tight_layout()
plt.show()

# Loss curves overlaid
fig, ax = plt.subplots(figsize=(9, 5))
colors = [PALETTE['royal_blue'], PALETTE['coral'],
          PALETTE['emerald'], PALETTE['amber']]

for (d, (_, losses)), c in zip(depth_results.items(), colors):
```

```

ax.plot(losses, label=f"{d} hidden layer{'s' if d > 1 else ''}", color=c, alpha=0.85)

ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.set_title("Training Loss by Network Depth (Spirals)")
ax.legend()
plt.tight_layout()
plt.show()

```

Observations: - A 1-layer network (linear boundary + sigmoid) cannot capture the spiral structure at all. - 2 layers start to approximate it but struggle with the tightest curves. - 3 and 5 layers learn progressively better boundaries. - Deeper networks converge faster *up to a point* – beyond that, training can become unstable without techniques like batch normalisation or careful initialization.

6.6 The Vanishing Gradient Problem

When we stack many layers with **Sigmoid** activations, gradients get multiplied by the sigmoid derivative $\sigma'(z) \leq 0.25$ at each layer. After 10 layers the gradient has been multiplied by $(0.25^{10} \approx 10^{-6})$ – it effectively **vanishes**.

This is the **vanishing gradient problem**, and it was a major obstacle to training deep networks before ReLU became the default activation.

We will visualize this by recording gradient norms at every layer throughout training.

```

def build_deep_model(n_layers, width=16, activation='sigmoid'):
    """Build a deep binary classifier with the given
    activation."""
    act_class = Sigmoid if activation == 'sigmoid' else ReLU
    layers = []
    in_dim = 2
    for _ in range(n_layers):
        layers.append(Linear(in_dim, width))
        layers.append(act_class())
        in_dim = width
    layers.append(Linear(in_dim, 1))
    layers.append(Sigmoid())
    return Sequential(*layers)

def train_with_grad_tracking(model, loss_fn, X, y, lr=0.01,
                             epochs=2000,
                             track_every=100):
    """Train and record gradient norms at Linear layers every
    `track_every` epochs."""
    losses = []
    grad_norms = [] # list of lists: one per tracked epoch

    for epoch in range(epochs):

```

```

y_pred = model.forward(X)
loss = loss_fn.forward(y_pred, y)
grad = loss_fn.backward()
model.backward(grad)

# Track gradient norms at Linear layers
if epoch % track_every == 0:
    norms = []
    for layer in model.layers:
        if isinstance(layer, Linear) and layer.grad_W is
not None:
            norms.append(np.linalg.norm(layer.grad_W))
    grad_norms.append(norms)

    for param, grad_p in model.params():
        param -= lr * grad_p
    losses.append(loss)

return losses, grad_norms

X_vg, y_vg = make_moons(n=300, noise=0.15)
n_deep_layers = 10
track_interval = 100
n_epochs_vg = 2000

# Train with Sigmoid
np.random.seed(42)
model_sig = build_deep_model(n_deep_layers, width=16,
                             activation='sigmoid')
losses_sig, grads_sig = train_with_grad_tracking(
    model_sig, BinaryCrossEntropyLoss(), X_vg, y_vg,
    lr=0.5, epochs=n_epochs_vg, track_every=track_interval
)

# Train with ReLU
np.random.seed(42)
model_relu = build_deep_model(n_deep_layers, width=16,
                              activation='relu')
losses_relu, grads_relu = train_with_grad_tracking(
    model_relu, BinaryCrossEntropyLoss(), X_vg, y_vg,
    lr=0.05, epochs=n_epochs_vg, track_every=track_interval
)

print(f"Sigmoid final loss: {losses_sig[-1]:.4f}")
print(f"ReLU final loss: {losses_relu[-1]:.4f}")

# === HERO VISUALIZATION: Gradient Magnitude Heatmaps ===

def plot_gradient_heatmap(grad_norms, ax, title, track_every,
                          vmin=None, vmax=None):
    """Plot a gradient magnitude heatmap.

```



```
cbar.set_label(r"$\log_{10}$(gradient norm)", fontsize=11)

fig.suptitle("The Vanishing Gradient Problem: Sigmoid vs ReLU",
             fontsize=15, fontweight='bold',
             color=PALETTE['deep_blue'], y=1.02)
plt.tight_layout()
plt.show()
```

Reading the Heatmaps

Left (Sigmoid): The early layers (L1, L2, L3) are *dark* throughout training – their gradient norms are orders of magnitude smaller than the later layers. These layers barely learn anything, which is why deep sigmoid networks train so poorly.

Right (ReLU): Gradient magnitudes are much more *uniform* across layers. ReLU has a derivative of exactly 1 for positive inputs, so the gradient signal propagates through without exponential decay.

This is why **ReLU** (and its variants) became the default activation for deep networks. It was one of the key insights that made training deep networks practical.

6.7 Width vs. Depth

Given a roughly fixed parameter budget, how should we allocate capacity?

Configuration	Layers	Width	Approx. params
Shallow-wide	2	128	~17k
Deep-narrow	8	16	~2k
Balanced	4	32	~4k

We compare all three on the spirals dataset.

```
X_wd, y_wd = make_spirals(n=400, n_classes=2, noise=0.25)

configs = {
    "Shallow-Wide\n(2 layers, 128 neurons)": (2, 128),
    "Balanced\n(4 layers, 32 neurons)": (4, 32),
    "Deep-Narrow\n(8 layers, 16 neurons)": (8, 16),
}

wd_results = {}
for name, (depth, width) in configs.items():
    np.random.seed(42)
    model = build_model(depth, width)
    n_params = sum(p.size for p, _ in model.params())
    losses = train(model, BinaryCrossEntropyLoss(), X_wd, y_wd,
                   lr=0.1, epochs=1500, verbose=False)
    wd_results[name] = (model, losses, n_params)
    print(f"{name.replace(chr(10), ' ')}:
          {n_params:,} parameters, final loss = {losses[-1]:.4f}")
```



```

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for idx, (name, (model, losses, n_params)) in
    enumerate(wd_results.items()):
    plot_decision_boundary(model, X_wd, y_wd, ax=axes[idx])
    axes[idx].set_title(f"{name}\n({n_params:,} params)",
        fontsize=10)

fig.suptitle("Width vs Depth: Decision Boundaries",
    fontsize=14, fontweight='bold',
    color=PALETTE['deep_blue'], y=1.05)
plt.tight_layout()
plt.show()

# Loss curves
fig, ax = plt.subplots(figsize=(9, 5))
colors_wd = [PALETTE['royal_blue'], PALETTE['emerald'],
    PALETTE['coral']]
for (name, (_, losses, _)), c in zip(wd_results.items(),
    colors_wd):
    label = name.replace('\n', ' ')
    ax.plot(losses, label=label, color=c, alpha=0.85)
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.set_title("Width vs Depth: Training Loss")
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()

```

Takeaways: - The **shallow-wide** network has the most parameters and can memorize well, but its boundary is less smooth. - The **deep-narrow** network has very few parameters and may underfit, but it forces compositional feature learning. - The **balanced** network often strikes the best tradeoff – enough depth for feature hierarchy, enough width for expressiveness.

6.8 Multi-Class Classification

So far we have tackled binary (2-class) problems. For multi-class classification we:

1. Output one value per class (the **logits**)
2. Use **CrossEntropyLoss** which internally applies softmax and computes the negative log-likelihood
3. The predicted class is simply argmax of the output

```

# 3-class spiral dataset
X_mc, y_mc = make_spirals(n=450, n_classes=3, noise=0.25)

np.random.seed(42)
mc_model = Sequential(
    Linear(2, 64), ReLU(),
    Linear(64, 64), ReLU(),
    Linear(64, 32), ReLU(),
    Linear(32, 3) # 3 output logits, no softmax --
    CrossEntropyLoss handles it
)

```

```

mc_loss_fn = CrossEntropyLoss()
mc_losses = []
mc_accs = []
lr_mc = 0.05
epochs_mc = 2000

for epoch in range(epochs_mc):
    y_pred = mc_model.forward(X_mc)
    loss = mc_loss_fn.forward(y_pred, y_mc)
    grad = mc_loss_fn.backward()
    mc_model.backward(grad)
    for param, grad_p in mc_model.params():
        param -= lr_mc * grad_p
    mc_losses.append(loss)

    # Track accuracy
    preds = np.argmax(y_pred, axis=1)
    acc = np.mean(preds == y_mc.astype(int))
    mc_accs.append(acc)

    if (epoch + 1) % 500 == 0:
        print(f"Epoch {epoch+1:4d}/{epochs_mc} Loss: {loss:.4f}
              Acc: {acc:.3f}")

print(f"\nFinal accuracy: {mc_accs[-1]:.1%}")

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Decision regions
plot_decision_boundary(mc_model, X_mc, y_mc, ax=axes[0])
axes[0].set_title("3-Class Decision Regions")

# Loss curve
axes[1].plot(mc_losses, color=PALETTE['royal_blue'])
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Cross-Entropy Loss")
axes[1].set_title("Training Loss")

# Accuracy curve
axes[2].plot(mc_accs, color=PALETTE['emerald'])
axes[2].set_xlabel("Epoch")
axes[2].set_ylabel("Accuracy")
axes[2].set_title("Training Accuracy")
axes[2].set_ylim(0, 1.05)
axes[2].axhline(y=1.0, color=PALETTE['warm_gray'],
                linestyle='--', alpha=0.5)

fig.suptitle("Multi-Class Classification: 3-Class Spirals",
             fontsize=14, fontweight='bold',
             color=PALETTE['deep_blue'], y=1.02)
plt.tight_layout()
plt.show()

```

The network successfully carves out three distinct regions for the three spiral classes. The accuracy curve shows rapid learning in the first few hundred epochs, then gradual refinement.

6.9 Exercises

Exercise 1: Activation function showdown

Train 4-layer networks on the spirals dataset using three different activations: ReLU, Sigmoid, and Tanh. Plot their loss curves on the same axes and compare convergence speed and final loss. Which activation works best and why?

Exercise 2: Learning rate sensitivity

Using the 3-layer ReLU architecture from Section 6.4, train on the moons dataset with learning rates $\{0.001, 0.01, 0.1, 1.0\}$. For each, plot the loss curve and the final decision boundary in a 2x4 grid. What happens when the learning rate is too small? Too large?

Exercise 3: Your own gradient experiment

Reproduce the vanishing gradient heatmap from Section 6.6 but with **Tanh** activations instead of Sigmoid. Does Tanh suffer from the same problem? (Hint: the derivative of tanh at $z=0$ is 1, compared to 0.25 for sigmoid.) Try a 10-layer Tanh network and compare the heatmap to the Sigmoid and ReLU results.

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
from utils.nn import Linear, ReLU, Sequential, CrossEntropyLoss
from utils.data import make_spirals, train_test_split
from utils.plot import plot_decision_boundary, plot_loss_curve
apply_style()
np.random.seed(42)
```

Chapter 7: Optimizers

How neural networks learn – choosing the right path downhill

Learning Objectives

By the end of this chapter, you will:

- Understand the optimization landscape: loss surfaces, local minima, saddle points, and plateaus
- Implement **SGD** from scratch and understand its batch, mini-batch, and stochastic variants

- Build **Momentum SGD** and grasp exponential moving averages as the key intuition
- Implement **RMSProp** and understand adaptive per-parameter learning rates
- Build the **Adam** optimizer with bias correction
- Design and implement **learning rate schedules**: step decay, exponential decay, cosine annealing, and warmup
- Compare all optimizers on the same task and interpret the differences in convergence
- Visualize optimizer trajectories on a 2D loss landscape – the signature figure of this chapter

7.1 The Optimization Landscape

Training a neural network means minimizing a loss function $L(\theta)$ with respect to the parameters θ . The shape of this function – the **loss surface** – determines how difficult the optimization problem is.

Convex vs Non-Convex

A **convex** loss surface has a single global minimum: any downhill step eventually reaches it. Linear regression with MSE loss is convex. Deep neural networks, however, produce **non-convex** loss surfaces with:

- **Local minima** – points that are lower than all nearby points but not the global lowest
- **Saddle points** – points where the gradient is zero but the surface curves up in some directions and down in others
- **Plateaus** – flat regions where the gradient is near zero, slowing learning to a crawl

In high-dimensional spaces (millions of parameters), saddle points vastly outnumber local minima and are the primary challenge for optimization. Let us visualize what these look like.

--- 7.1 Visualization: 2D Loss Surfaces ---

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
```

--- (a) Convex surface: $f(x, y) = x^2 + y^2$ ---

```
x = np.linspace(-3, 3, 200)
```

```
y = np.linspace(-3, 3, 200)
```

```
X, Y = np.meshgrid(x, y)
```

```
Z_convex = X**2 + Y**2
```

```
ax = axes[0]
```

```
cf = ax.contourf(X, Y, Z_convex, levels=30, cmap='Blues')
```

```
ax.contour(X, Y, Z_convex, levels=15,
           colors=PALETTE['deep_blue'], linewidths=0.5, alpha=0.5)
```

```
ax.plot(0, 0, 'o', color=PALETTE['emerald'], markersize=10,
        markeredgecolor='white', zorder=5)
```

```
ax.annotate('Global\nminimum', xy=(0, 0), xytext=(1.0, 1.5),
```

```

        fontsize=9, color=PALETTE['emerald'],
        fontweight='bold',
        arrowprops=dict(arrowstyle='->',
            color=PALETTE['emerald'], lw=1.5))
ax.set_title('(a) Convex:  $f = x^2 + y^2$ ')
ax.set_xlabel('$x$'); ax.set_ylabel('$y$')
ax.grid(False)

# --- (b) Saddle point:  $f(x, y) = x^2 - y^2$  ---
Z_saddle = X**2 - Y**2

ax = axes[1]
cf = ax.contourf(X, Y, Z_saddle, levels=30, cmap='RdBu_r')
ax.contour(X, Y, Z_saddle, levels=15, colors=PALETTE['charcoal'],
    linewidths=0.5, alpha=0.4)
ax.plot(0, 0, 's', color=PALETTE['coral'], markersize=12,
    markeredgecolor='white', zorder=5)
ax.annotate('Saddle\npoint', xy=(0, 0), xytext=(1.2, 1.5),
    fontsize=9, color=PALETTE['coral'], fontweight='bold',
    arrowprops=dict(arrowstyle='->',
        color=PALETTE['coral'], lw=1.5))
ax.set_title('(b) Saddle:  $f = x^2 - y^2$ ')
ax.set_xlabel('$x$'); ax.set_ylabel('$y$')
ax.grid(False)

# --- (c) Beale's function: complex non-convex landscape ---
x_b = np.linspace(-4.5, 4.5, 300)
y_b = np.linspace(-4.5, 4.5, 300)
XB, YB = np.meshgrid(x_b, y_b)
Z_beale = ((1.5 - XB + XB * YB)**2 +
    (2.25 - XB + XB * YB**2)**2 +
    (2.625 - XB + XB * YB**3)**2)
Z_beale_log = np.log(Z_beale + 1) # log scale for better
    visualization

ax = axes[2]
cf = ax.contourf(XB, YB, Z_beale_log, levels=30, cmap='magma_r')
ax.contour(XB, YB, Z_beale_log, levels=15, colors='white',
    linewidths=0.3, alpha=0.4)
ax.plot(3.0, 0.5, '*', color=PALETTE['emerald'], markersize=15,
    markeredgecolor='white', zorder=5)
ax.annotate('Global min\n(3, 0.5)', xy=(3.0, 0.5), xytext=(0.0,
    2.5),
    fontsize=9, color=PALETTE['emerald'],
    fontweight='bold',
    arrowprops=dict(arrowstyle='->',
        color=PALETTE['emerald'], lw=1.5))
ax.set_title("(c) Beale's Function (non-convex)")
ax.set_xlabel('$x$'); ax.set_ylabel('$y$')
ax.grid(False)

plt.tight_layout()
plt.savefig('../figures/ch07_loss_surfaces.png', dpi=200,

```

```

        facecolor=PALETTE['paper'], bbox_inches='tight')
plt.show()

```

7.2 Stochastic Gradient Descent (SGD)

The most fundamental optimizer is **gradient descent**: repeatedly step in the direction of steepest descent.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}$$

where α is the **learning rate**.

Variants by Batch Size

Variant	Batch Size	Gradient Estimate	Properties
Batch GD	Entire dataset	Exact	Stable but slow; memory-heavy
Mini-batch GD	B samples (e.g., 32, 64)	Approximate	Best trade-off; most common
Stochastic GD	1 sample	Very noisy	Fast updates; high variance

Mini-batch SGD is the standard in practice. The noise from random batches actually helps escape shallow local minima and saddle points.

--- 7.2 SGD Implementation ---

```

class SGD:
    """Vanilla Stochastic Gradient Descent.

    Parameters
    -----
    params_fn : callable
        Function returning list of (param, grad) tuples from the
        model.
    lr : float
        Learning rate (step size).
    """
    def __init__(self, params_fn, lr=0.01):
        self.params_fn = params_fn
        self.lr = lr

    def step(self):
        """Update parameters: theta -= lr * grad."""
        for param, grad in self.params_fn():
            param -= self.lr * grad

    def zero_grad(self):
        """Reset all gradients to zero."""

```

```

        for param, grad in self.params_fn():
            if grad is not None:
                grad[:] = 0

# Quick test: single linear layer, one SGD step
np.random.seed(42)
layer = Linear(2, 3)
x_test = np.random.randn(4, 2)
y_test = np.array([0, 1, 2, 1])
loss_fn = CrossEntropyLoss()

# Forward + backward
logits = layer.forward(x_test)
loss_before = loss_fn.forward(logits, y_test)
grad = loss_fn.backward()
layer.backward(grad)

# One SGD step
optimizer = SGD(layer.params, lr=0.1)
optimizer.step()

# Verify loss decreased
logits_after = layer.forward(x_test)
loss_after = loss_fn.forward(logits_after, y_test)
print(f"Loss before SGD step: {loss_before:.4f}")
print(f"Loss after SGD step: {loss_after:.4f}")
print(f"Loss decreased: {loss_after < loss_before}")

```

7.3 Momentum

Vanilla SGD has a problem: it zigzags on elongated loss surfaces, making slow progress toward the minimum. **Momentum** fixes this by accumulating a velocity vector that smooths out the gradient direction.

Intuition: The Ball Rolling Downhill

Imagine a ball rolling down a hilly landscape. It does not stop at every tiny bump – its accumulated velocity carries it through small obstacles and keeps it rolling toward the valley floor.

Mathematics

Momentum uses an **exponential moving average** of past gradients:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla \theta_t$$

$$\theta_{t+1} = \theta_t - \alpha \mathbf{v}_t$$

where $\beta \in [0, 1)$ is the momentum coefficient (typically 0.9). The velocity $\mathbf{v}_t$ accumulates past gradients with exponential decay.

Nesterov Momentum

Nesterov's "look-ahead" variant computes the gradient at the anticipated future position:

$$\begin{aligned} \mathbf{v}_t &= \beta \mathbf{v}_{t-1} + \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t - \alpha \beta \mathbf{v}_{t-1}) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \alpha \mathbf{v}_t \end{aligned}$$

In practice, Nesterov momentum is often implemented with a rearranged update that avoids computing the gradient at a shifted position.

--- 7.3 Momentum SGD Implementation ---

```
class MomentumSGD:
    """SGD with Momentum (optionally Nesterov).

    Parameters
    -----
    params_fn : callable
        Function returning list of (param, grad) tuples.
    lr : float
        Learning rate.
    momentum : float
        Momentum coefficient (beta), typically 0.9.
    nesterov : bool
        If True, use Nesterov accelerated gradient.
    """
    def __init__(self, params_fn, lr=0.01, momentum=0.9,
                 nesterov=False):
        self.params_fn = params_fn
        self.lr = lr
        self.momentum = momentum
        self.nesterov = nesterov
        self.velocities = None # lazily initialized

    def step(self):
        params_list = list(self.params_fn())
        # Lazy initialization of velocity buffers
        if self.velocities is None:
            self.velocities = [np.zeros_like(p) for p, _ in
                               params_list]

        for i, (param, grad) in enumerate(params_list):
            # v_t = beta * v_{t-1} + grad
            self.velocities[i] = self.momentum *
            self.velocities[i] + grad
            if self.nesterov:
                # Nesterov: use look-ahead gradient
                param -= self.lr * (self.momentum *
                                    self.velocities[i] + grad)
            else:
                param -= self.lr * self.velocities[i]
```



```

def zero_grad(self):
    for param, grad in self.params_fn():
        if grad is not None:
            grad[:] = 0

print("MomentumSGD class defined.")

# --- 7.3 Visualization: SGD vs Momentum on Elongated Quadratic
---

def elongated_quadratic(x, y):
    """ $f(x, y) = 10x^2 + y^2$  -- very elongated elliptical
    contours."""
    return 10 * x**2 + y**2

def elongated_grad(pos):
    """Gradient of  $f(x, y) = 10x^2 + y^2$ ."""
    return np.array([20 * pos[0], 2 * pos[1]])

def run_sgd_path(start, grad_fn, lr, n_steps):
    """Run vanilla SGD and record the path."""
    path = [start.copy()]
    pos = start.copy()
    for _ in range(n_steps):
        g = grad_fn(pos)
        pos -= lr * g
        path.append(pos.copy())
    return np.array(path)

def run_momentum_path(start, grad_fn, lr, momentum, n_steps):
    """Run Momentum SGD and record the path."""
    path = [start.copy()]
    pos = start.copy()
    vel = np.zeros_like(start)
    for _ in range(n_steps):
        g = grad_fn(pos)
        vel = momentum * vel + g
        pos -= lr * vel
        path.append(pos.copy())
    return np.array(path)

start = np.array([3.0, 3.0])
n_steps = 30

path_sgd = run_sgd_path(start, elongated_grad, lr=0.04,
                        n_steps=n_steps)
path_mom = run_momentum_path(start, elongated_grad, lr=0.01,
                             momentum=0.9, n_steps=n_steps)

# --- Plot ---

```

```

fig, ax = plt.subplots(figsize=(8, 5))

x = np.linspace(-4, 4, 200)
y = np.linspace(-4, 4, 200)
X, Y = np.meshgrid(x, y)
Z = elongated_quadratic(X, Y)

ax.contourf(X, Y, Z, levels=30, cmap='Blues', alpha=0.5)
ax.contour(X, Y, Z, levels=15, colors=PALETTE['steel_blue'],
           linewidths=0.6, alpha=0.6)

# SGD path
ax.plot(path_sgd[:, 0], path_sgd[:, 1], '-o',
        color=PALETTE['coral'],
        markersize=4, linewidth=1.8, markeredgecolor='white',
        markeredgewidth=0.5,
        label='SGD (lr=0.04)', zorder=4)

# Momentum path
ax.plot(path_mom[:, 0], path_mom[:, 1], '-s',
        color=PALETTE['emerald'],
        markersize=4, linewidth=1.8, markeredgecolor='white',
        markeredgewidth=0.5,
        label='Momentum (lr=0.01,  $\beta=0.9$ )', zorder=4)

# Start point
ax.plot(start[0], start[1], '*', color=PALETTE['amber'],
        markersize=18,
        markeredgecolor='white', zorder=5, label='Start')

# Minimum
ax.plot(0, 0, 'P', color=PALETTE['deep_blue'], markersize=12,
        markeredgecolor='white', zorder=5, label='Minimum')

ax.set_title('SGD vs Momentum on Elongated Quadratic  $f = 10x^2 + y^2$ ')
ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
ax.legend(loc='upper right', fontsize=9)
ax.set_xlim(-4, 4)
ax.set_ylim(-4, 4)
ax.grid(False)

plt.tight_layout()
plt.savefig('../figures/ch07_sgd_vs_momentum.png', dpi=200,
           facecolor=PALETTE['paper'], bbox_inches='tight')
plt.show()

```

7.4 RMSProp

A different problem with SGD: all parameters share the same learning rate, even though some dimensions of the loss surface are much steeper than others. **RMSProp** (Root Mean Square Propagation, Hinton 2012) fixes this by maintaining a per-parameter running average of squared gradients.

Mathematics

$$s_t = \rho s_{t-1} + (1 - \rho) \|\nabla_{\theta} \mathcal{L}\|^2$$
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{s_t + \epsilon}} \nabla_{\theta} \mathcal{L}$$

where: s_t is the exponential moving average of squared gradients - ρ is the decay rate (typically 0.99) - ϵ is a small constant for numerical stability (typically 10^{-8})

Key insight: Parameters with consistently large gradients get their effective learning rate *reduced* (the denominator is large), while parameters with small gradients get *amplified*. This equalizes the step sizes across dimensions.

--- 7.4 RMSProp Implementation ---

```
class RMSProp:
    """RMSProp optimizer -- adaptive learning rates per parameter.

    Parameters
    -----
    params_fn : callable
        Function returning list of (param, grad) tuples.
    lr : float
        Learning rate.
    rho : float
        Decay rate for running average of squared gradients.
    eps : float
        Small constant for numerical stability.
    """
    def __init__(self, params_fn, lr=0.001, rho=0.99, eps=1e-8):
        self.params_fn = params_fn
        self.lr = lr
        self.rho = rho
        self.eps = eps
        self.sq_avg = None # lazily initialized

    def step(self):
        params_list = list(self.params_fn())
        if self.sq_avg is None:
            self.sq_avg = [np.zeros_like(p) for p, _ in
                           params_list]

        for i, (param, grad) in enumerate(params_list):
            # s_t = rho * s_{t-1} + (1 - rho) * grad^2
```

```

        self.sq_avg[i] = self.rho * self.sq_avg[i] + (1 -
self.rho) * grad**2
        # theta -= lr / sqrt(s_t + eps) * grad
        param -= self.lr * grad / (np.sqrt(self.sq_avg[i]) +
self.eps)

def zero_grad(self):
    for param, grad in self.params_fn():
        if grad is not None:
            grad[:] = 0

print("RMSProp class defined.")

```

7.5 Adam

Adam (Adaptive Moment Estimation, Kingma & Ba 2014) combines the best of both worlds: **momentum** (first moment) and **RMSProp** (second moment). It is the default optimizer in most deep learning projects today.

Mathematics

Adam maintains two exponential moving averages:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta})^2$$
 (first moment -- mean of gradients) (second moment -- variance of gradients)

Bias Correction

Since $(m_0 = 0)$ and $(v_0 = 0)$, the estimates are biased toward zero in early steps. Adam corrects this:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The update rule:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Default Hyperparameters

The original paper recommends $(\alpha = 0.001)$, $(\beta_1 = 0.9)$, $(\beta_2 = 0.999)$, $(\epsilon = 10^{-8})$. These defaults work remarkably well across a wide range of problems.

--- 7.5 Adam Implementation ---

class Adam:

"""Adam optimizer -- adaptive learning rates with momentum.

Combines first-moment (mean) and second-moment (variance) estimates of the gradients with bias correction.

Parameters

params_fn : callable

Function returning list of (param, grad) tuples.

lr : float

Learning rate (alpha).

beta1 : float

Decay rate for first moment estimate.

beta2 : float

Decay rate for second moment estimate.

eps : float

Small constant for numerical stability.

"""

def __init__(self, params_fn, lr=0.001, beta1=0.9,

beta2=0.999, eps=1e-8):

self.params_fn = params_fn

self.lr = lr

self.beta1 = beta1

self.beta2 = beta2

self.eps = eps

self.m = None # first moment (mean)

self.v = None # second moment (variance)

self.t = 0 # time step counter

def step(self):

params_list = list(self.params_fn())

Lazy initialization

if self.m **is** None:

self.m = [np.zeros_like(p) **for** p, _ **in** params_list]

self.v = [np.zeros_like(p) **for** p, _ **in** params_list]

self.t += 1

for i, (param, grad) **in** enumerate(params_list):

Update biased first and second moment estimates

self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1)

* grad

self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2)

* grad**2

Bias correction

m_hat = self.m[i] / (1 - self.beta1**self.t)

v_hat = self.v[i] / (1 - self.beta2**self.t)

Update parameters

param -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

```

def zero_grad(self):
    for param, grad in self.params_fn():
        if grad is not None:
            grad[:] = 0

# Quick verification: Adam on a simple problem
np.random.seed(42)
layer_test = Linear(2, 3)
loss_fn_test = CrossEntropyLoss()

optimizer_adam = Adam(layer_test.params, lr=0.01)

losses = []
for step in range(50):
    logits = layer_test.forward(x_test)
    loss = loss_fn_test.forward(logits, y_test)
    losses.append(loss)
    grad = loss_fn_test.backward()
    layer_test.backward(grad)
    optimizer_adam.step()
    optimizer_adam.zero_grad()

print(f"Adam: loss {losses[0]:.4f} -> {losses[-1]:.4f} in 50
      steps")

```

7.6 Learning Rate Schedules

The learning rate α does not need to stay constant during training. In fact, **decaying the learning rate** over time is one of the most effective ways to improve final performance:

- **High learning rate early:** explore broadly, escape bad regions quickly
- **Low learning rate late:** fine-tune near a good minimum

Here are four popular schedules.

```

# --- 7.6 Learning Rate Schedules ---

def step_decay(initial_lr, epoch, drop_factor=0.5, drop_every=30):
    """Step decay: cut lr by drop_factor every drop_every
    epochs."""
    return initial_lr * (drop_factor ** (epoch // drop_every))

def exponential_decay(initial_lr, epoch, decay_rate=0.02):
    """Exponential decay: lr = lr_0 * exp(-decay_rate * epoch)."""
    return initial_lr * np.exp(-decay_rate * epoch)

def cosine_annealing(initial_lr, epoch, total_epochs, min_lr=0.0):
    """Cosine annealing: smooth decay following a cosine curve."""

```

```

    return min_lr + 0.5 * (initial_lr - min_lr) * (1 +
        np.cos(np.pi * epoch / total_epochs))

def warmup_cosine(initial_lr, epoch, total_epochs,
    warmup_epochs=10, min_lr=0.0):
    """Linear warmup followed by cosine annealing."""
    if epoch < warmup_epochs:
        return initial_lr * (epoch + 1) / warmup_epochs
    else:
        progress = (epoch - warmup_epochs) / (total_epochs -
            warmup_epochs)
        return min_lr + 0.5 * (initial_lr - min_lr) * (1 +
            np.cos(np.pi * progress))

# --- Visualization: 2x2 grid of LR schedules ---
total_epochs = 150
epochs = np.arange(total_epochs)
initial_lr = 0.01

schedules = {
    'Step Decay\n($\\times 0.5$ every 30 epochs)':
        [step_decay(initial_lr, e) for e in epochs],
    'Exponential Decay\n($\\gamma = 0.02$)':
        [exponential_decay(initial_lr, e) for e in epochs],
    'Cosine Annealing': [cosine_annealing(initial_lr, e,
        total_epochs) for e in epochs],
    'Warmup + Cosine\n(10 epoch warmup)':
        [warmup_cosine(initial_lr, e, total_epochs) for e in
        epochs],
}

colors_sched = [PALETTE['royal_blue'], PALETTE['coral'],
    PALETTE['emerald'], PALETTE['amber']]

fig, axes = plt.subplots(2, 2, figsize=(11, 7))
for ax, (name, lr_values), color in zip(axes.flat,
    schedules.items(), colors_sched):
    ax.plot(epochs, lr_values, color=color, linewidth=2.5)
    ax.fill_between(epochs, lr_values, alpha=0.08, color=color)
    ax.set_title(name, fontsize=11)
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Learning Rate')
    ax.set_ylim(-0.0005, initial_lr * 1.15)
    ax.axhline(y=0, color=PALETTE['warm_gray'], linewidth=0.5,
        linestyle='--', alpha=0.5)

fig.suptitle('Learning Rate Schedules', fontsize=14,
    fontweight='bold',
    color=PALETTE['deep_blue'], y=1.02)
plt.tight_layout()
plt.savefig('../figures/ch07_lr_schedules.png', dpi=200,
    facecolor=PALETTE['paper'], bbox_inches='tight')
plt.show()

```

7.7 Optimizer Comparison – Loss Curves

Now let us put all four optimizers head-to-head on the same task: classifying the spirals dataset. We will train identical network architectures with each optimizer and compare their loss curves.

```
# --- 7.7 Optimizer Comparison: Training on Spirals ---

# Generate data
X_all, y_all = make_spirals(n=300, n_classes=3, noise=0.3,
                           seed=42)
X_train, X_test, y_train, y_test = train_test_split(X_all, y_all,
                                                    test_ratio=0.2, seed=42)

def make_model(seed=42):
    """Create a fresh model with the same random
    initialization."""
    np.random.seed(seed)
    return Sequential(
        Linear(2, 64), ReLU(),
        Linear(64, 32), ReLU(),
        Linear(32, 3)
    )

def train_model(model, optimizer, X, y, epochs=200):
    """Train model and return loss history."""
    loss_fn = CrossEntropyLoss()
    losses = []
    for epoch in range(epochs):
        # Forward
        logits = model.forward(X)
        loss = loss_fn.forward(logits, y)
        losses.append(loss)
        # Backward
        grad = loss_fn.backward()
        model.backward(grad)
        # Update
        optimizer.step()
        optimizer.zero_grad()
    return losses

# Train with each optimizer
n_epochs = 200

optimizers_config = {
    'SGD (lr=0.5)': Lambda(m: SGD(m.params, lr=0.5),
    'Momentum (lr=0.1)': Lambda(m: MomentumSGD(m.params, lr=0.1,
    momentum=0.9),
    'RMSProp (lr=0.005)': Lambda(m: RMSProp(m.params, lr=0.005),
    'Adam (lr=0.005)': Lambda(m: Adam(m.params, lr=0.005),
```



```

}

results = {}
for name, opt_fn in optimizers_config.items():
    model = make_model(seed=42)
    opt = opt_fn(model)
    losses = train_model(model, opt, X_train, y_train,
                          epochs=n_epochs)
    results[name] = losses
    print(f"{name:25s} | Final loss: {losses[-1]:.4f}")

print("\nTraining complete.")

# --- 7.7 Visualization: Loss Curves + Convergence Speed ---

opt_colors = [PALETTE['coral'], PALETTE['emerald'],
              PALETTE['amber'], PALETTE['royal_blue']]

fig, axes = plt.subplots(1, 2, figsize=(14, 5),
                        gridspec_kw={'width_ratios': [2, 1]})

# --- Left: Loss curves ---
ax = axes[0]
for (name, losses), color in zip(results.items(), opt_colors):
    ax.plot(losses, label=name, color=color, linewidth=2.2,
            alpha=0.9)

ax.set_xlabel('Epoch')
ax.set_ylabel('Cross-Entropy Loss')
ax.set_title('Optimizer Comparison: Loss Curves')
ax.legend(fontsize=9, loc='upper right')
ax.set_ylim(bottom=0)

# --- Right: Bar chart of epochs to reach threshold ---
threshold = 0.5
ax = axes[1]
epochs_to_threshold = []
names = []
for name, losses in results.items():
    reached = [i for i, l in enumerate(losses) if l < threshold]
    epochs_to_threshold.append(reached[0] if reached else
                              n_epochs)
    names.append(name.split(' ')[0]) # short name

bars = ax.barh(range(len(names)), epochs_to_threshold,
               color=opt_colors,
               edgecolor='white', linewidth=1.2, height=0.6)
ax.set_yticks(range(len(names)))
ax.set_yticklabels(names, fontsize=10)
ax.set_xlabel('Epochs to Reach Loss < 0.5')
ax.set_title('Convergence Speed')
ax.invert_yaxis()

# Add value labels on bars

```

```

for bar, val in zip(bars, epochs_to_threshold):
    ax.text(bar.get_width() + 1.5, bar.get_y() +
            bar.get_height() / 2,
            f'{val}', va='center', fontsize=10,
            color=PALETTE['charcoal'], fontweight='bold')

plt.tight_layout()
plt.savefig('../figures/ch07_optimizer_comparison.png', dpi=200,
            facecolor=PALETTE['paper'], bbox_inches='tight')
plt.show()

```

7.8 Optimizer Trajectories – Hero Figure

The most informative way to compare optimizers is to watch them navigate a 2D loss landscape. We use **Beale’s function** – a classic non-convex test function with a single global minimum at $((3, 0.5))$ but a complex, ravine-filled surface that exposes each optimizer’s personality.

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$$

--- 7.8 Hero Figure: Optimizer Trajectories on Beale's Function ---

```

def beale(pos):
    """Beale's function value."""
    x, y = pos[0], pos[1]
    return ((1.5 - x + x*y)**2 +
            (2.25 - x + x*y**2)**2 +
            (2.625 - x + x*y**3)**2)

def beale_grad(pos):
    """Gradient of Beale's function."""
    x, y = pos[0], pos[1]
    t1 = 1.5 - x + x*y
    t2 = 2.25 - x + x*y**2
    t3 = 2.625 - x + x*y**3
    dfdx = (2 * t1 * (-1 + y) +
            2 * t2 * (-1 + y**2) +
            2 * t3 * (-1 + y**3))
    dfdy = (2 * t1 * x +
            2 * t2 * 2*x*y +
            2 * t3 * 3*x*y**2)
    return np.array([dfdx, dfdy])

# --- Optimizer trajectory runners (2D parameter space) ---

def run_2d_sgd(start, grad_fn, lr, n_steps):
    path = [start.copy()]
    pos = start.copy()
    for _ in range(n_steps):

```

```

        g = grad_fn(pos)
        # Clip gradients for stability
        g = np.clip(g, -10, 10)
        pos = pos - lr * g
        path.append(pos.copy())
    return np.array(path)

def run_2d_momentum(start, grad_fn, lr, beta, n_steps):
    path = [start.copy()]
    pos = start.copy()
    vel = np.zeros(2)
    for _ in range(n_steps):
        g = grad_fn(pos)
        g = np.clip(g, -10, 10)
        vel = beta * vel + g
        pos = pos - lr * vel
        path.append(pos.copy())
    return np.array(path)

def run_2d_rmsprop(start, grad_fn, lr, rho, eps, n_steps):
    path = [start.copy()]
    pos = start.copy()
    sq = np.zeros(2)
    for _ in range(n_steps):
        g = grad_fn(pos)
        g = np.clip(g, -10, 10)
        sq = rho * sq + (1 - rho) * g**2
        pos = pos - lr * g / (np.sqrt(sq) + eps)
        path.append(pos.copy())
    return np.array(path)

def run_2d_adam(start, grad_fn, lr, beta1, beta2, eps, n_steps):
    path = [start.copy()]
    pos = start.copy()
    m = np.zeros(2)
    v = np.zeros(2)
    for t in range(1, n_steps + 1):
        g = grad_fn(pos)
        g = np.clip(g, -10, 10)
        m = beta1 * m + (1 - beta1) * g
        v = beta2 * v + (1 - beta2) * g**2
        m_hat = m / (1 - beta1**t)
        v_hat = v / (1 - beta2**t)
        pos = pos - lr * m_hat / (np.sqrt(v_hat) + eps)
        path.append(pos.copy())
    return np.array(path)

# --- Run all four optimizers from the same starting point ---
start = np.array([0.0, 2.0])

```

```

n_steps = 120

path_sgd_b = run_2d_sgd(start, beale_grad, lr=0.0001,
                        n_steps=n_steps)
path_mom_b = run_2d_momentum(start, beale_grad, lr=0.00005,
                             beta=0.9, n_steps=n_steps)
path_rms_b = run_2d_rmsprop(start, beale_grad, lr=0.01,
                             rho=0.9, eps=1e-8, n_steps=n_steps)
path_adam_b = run_2d_adam(start, beale_grad, lr=0.1, beta1=0.9,
                          beta2=0.999, eps=1e-8, n_steps=n_steps)

# --- Hero Figure ---
fig, ax = plt.subplots(figsize=(10, 8))

# Background: Beale's function contours (log scale)
x_grid = np.linspace(-1.5, 4.5, 400)
y_grid = np.linspace(-1.5, 3.5, 400)
XG, YG = np.meshgrid(x_grid, y_grid)
ZG = ((1.5 - XG + XG*YG)**2 +
      (2.25 - XG + XG*YG**2)**2 +
      (2.625 - XG + XG*YG**3)**2)
ZG_log = np.log(ZG + 1)

# Filled contours
cf = ax.contourf(XG, YG, ZG_log, levels=40, cmap='magma_r',
                alpha=0.85)
ax.contour(XG, YG, ZG_log, levels=20, colors='white',
          linewidths=0.3, alpha=0.3)

# --- Plot optimizer paths ---
paths = [
    ('SGD', path_sgd_b, PALETTE['coral'], 'o'),
    ('Momentum', path_mom_b, PALETTE['emerald'], 's'),
    ('RMSProp', path_rms_b, PALETTE['amber'], 'D'),
    ('Adam', path_adam_b, PALETTE['royal_blue'], '^'),
]

for name, path, color, marker in paths:
    ax.plot(path[:, 0], path[:, 1], '-', color=color,
            linewidth=2.0, alpha=0.85, zorder=3)
    # Plot markers at intervals for clarity
    step_interval = max(1, len(path) // 12)
    ax.plot(path[:, 0], path[:, 1], path[:, 0], path[:, 1],
            marker=marker, color=color, markersize=6,
            linestyle='None',
            markeredgewidth=0.8,
            zorder=4, label=name)

# Start point
ax.plot(start[0], start[1], '*', color='white', markersize=20,
        markeredgewidth=1.5,
        zorder=6)
ax.annotate('Start', xy=(start[0], start[1]),

```

```

        xytext=(start[0] - 1.0, start[1] + 0.4),
        fontsize=10, color='white', fontweight='bold',
        arrowprops=dict(arrowstyle='->', color='white',
lw=1.5))

# Global minimum
ax.plot(3.0, 0.5, '*', color=PALETTE['emerald'], markersize=18,
        markeredgecolor='white', markeredgewidth=1.5, zorder=6)
ax.annotate('Global min\n(3, 0.5)', xy=(3.0, 0.5),
        xytext=(3.5, 1.8),
        fontsize=10, color='white', fontweight='bold',
        arrowprops=dict(arrowstyle='->', color='white',
lw=1.5))

ax.set_title("Optimizer Trajectories on Beale's Function",
        fontsize=14, pad=15)
ax.set_xlabel('$x$', fontsize=12)
ax.set_ylabel('$y$', fontsize=12)
ax.legend(loc='lower left', fontsize=10, frameon=True,
        facecolor='white',
        framealpha=0.85, edgecolor=PALETTE['warm_gray'])
ax.set_xlim(-1.5, 4.5)
ax.set_ylim(-1.5, 3.5)
ax.grid(False)

plt.tight_layout()
plt.savefig('../figures/ch07_optimizer_trajectories.png', dpi=200,
        facecolor=PALETTE['paper'], bbox_inches='tight')
plt.show()

# Print final positions
print("\nFinal positions after {} steps:".format(n_steps))
print(f" SGD:      ({path_sgd_b[-1, 0]:.3f}, {path_sgd_b[-1,
1]:.3f})  loss = {beale(path_sgd_b[-1]):.4f}")
print(f" Momentum: ({path_mom_b[-1, 0]:.3f}, {path_mom_b[-1,
1]:.3f})  loss = {beale(path_mom_b[-1]):.4f}")
print(f" RMSProp:  ({path_rms_b[-1, 0]:.3f}, {path_rms_b[-1,
1]:.3f})  loss = {beale(path_rms_b[-1]):.4f}")
print(f" Adam:     ({path_adam_b[-1, 0]:.3f}, {path_adam_b[-1,
1]:.3f})  loss = {beale(path_adam_b[-1]):.4f}")
print(f" Target:   (3.000, 0.500)  loss = 0.0000")

```

7.9 Exercises

Exercise 7.1: Implement AdaGrad

AdaGrad (Duchi et al., 2011) was the precursor to RMSProp. It accumulates *all* past squared gradients without decay:

$$\mathbf{G}_t = \mathbf{G}_{t-1} + (\nabla_{\boldsymbol{\theta}} \mathcal{L})^2, \text{ \quad } \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\mathbf{G}_t + \epsilon}} \nabla_{\boldsymbol{\theta}} \mathcal{L}$$

Implement an AdaGrad class following the same interface as our other optimizers. Then run it on Beale's function alongside Adam and RMSProp. What happens to AdaGrad's effective learning rate over time? Why does RMSProp solve this problem?

Exercise 7.2: Learning Rate Finder

Implement a **learning rate range test** (Smith, 2017): start with a very small learning rate and increase it exponentially over one epoch, recording the loss at each step. Plot loss vs. learning rate on a log scale. The optimal learning rate is typically just before the point where the loss starts to diverge.

Write a function `lr_range_test(model, X, y, lr_min=1e-6, lr_max=1.0, n_steps=100)` that returns `(lr_values, loss_values)`. Apply it to the spirals dataset and find the best learning rate for SGD.

Exercise 7.3: Optimizer Sensitivity Analysis

How sensitive is each optimizer to its hyperparameters? For Adam, train the spirals model with $(\beta_1 \in \{0.5, 0.9, 0.99\})$ and $(\beta_2 \in \{0.9, 0.999, 0.9999\})$ (9 combinations). Create a 3x3 heatmap showing the final loss for each combination. Repeat the exercise for Momentum SGD varying $(\beta \in \{0.5, 0.9, 0.99\})$ and $lr \in \{0.01, 0.1, 0.5\})$. Which optimizer is more robust to hyperparameter choices?

Next: [Chapter 8 – Regularization](#)

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
from utils.nn import Linear, ReLU, Sequential, Dropout,
    CrossEntropyLoss
from utils.data import make_moons, make_spirals, train_test_split
from utils.plot import plot_decision_boundary, plot_loss_curve
apply_style()
np.random.seed(42)
```

Chapter 8: Regularization

Taming the Beast: Preventing Neural Networks from Memorizing

Learning Objectives

By the end of this chapter, you will:

- Understand **overfitting** and the bias-variance tradeoff

- Implement **L1 and L2 regularization** as gradient modifications and see their effects on weight distributions
- Build a **Dropout** layer from scratch using inverted dropout
- Implement **early stopping** with a patience parameter
- Preview **data augmentation** techniques for simple 2D data
- Compare all regularization methods side-by-side on the same dataset
- Find the optimal regularization strength through grid search

8.1 Overfitting

A model **overfits** when it learns the noise in the training data rather than the underlying pattern. It achieves near-perfect training accuracy but performs poorly on unseen data.

Overfitting is especially likely when: - The model has **many more parameters** than training examples - The training data is **noisy** - Training continues for **too many epochs**

Let us generate a small, noisy dataset and train an overly large network to see this phenomenon in action.

```
# Generate a small, noisy dataset -- the perfect recipe for overfitting
```

```
np.random.seed(42)
X, y = make_moons(n=120, noise=0.30, seed=42)
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_ratio=0.3, seed=42)
```

```
print(f"Training samples: {len(X_train)}")
print(f"Test samples:      {len(X_test)}")
print(f"Features:           {X_train.shape[1]}")
```

```
# Visualize the data
```

```
fig, ax = plt.subplots(figsize=(7, 5))
for c in [0, 1]:
    mask = y_train == c
    ax.scatter(X_train[mask, 0], X_train[mask, 1],
               c=PALETTE["royal_blue"] if c == 0 else
               PALETTE["coral"],
               label=f"Class {c} (train)", edgecolors="white",
               linewidth=0.5, s=50)
for c in [0, 1]:
    mask = y_test == c
    ax.scatter(X_test[mask, 0], X_test[mask, 1],
               c=PALETTE["royal_blue"] if c == 0 else
               PALETTE["coral"],
               marker="x", label=f"Class {c} (test)", s=40,
               alpha=0.6)
ax.set_title("Small Noisy Moon Dataset")
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

```

def build_large_network():
    """An overly large network for our small dataset: 2 -> 64 ->
    64 -> 64 -> 2."""
    return Sequential(
        Linear(2, 64), ReLU(),
        Linear(64, 64), ReLU(),
        Linear(64, 64), ReLU(),
        Linear(64, 2)
    )

def train_model(model, X_tr, y_tr, X_te, y_te, epochs=500,
                lr=0.01,
                l2_lambda=0.0, l1_lambda=0.0, dropout_model=False,
                early_stopping=False, patience=20):
    """General training loop with optional regularization."""
    loss_fn = CrossEntropyLoss()
    train_losses, test_losses = [], []
    best_test_loss = float('inf')
    patience_counter = 0
    stopped_epoch = epochs

    for epoch in range(epochs):
        # --- Forward (training mode) ---
        model.train()
        logits = model.forward(X_tr)
        loss = loss_fn.forward(logits, y_tr)
        train_losses.append(loss)

        # --- Backward ---
        grad = loss_fn.backward()
        model.backward(grad)

        # --- Apply regularization to gradients ---
        for param, grad_param in model.params():
            if grad_param is not None:
                if l2_lambda > 0:
                    grad_param += l2_lambda * param # L2:
                    gradient += lambda * w
                if l1_lambda > 0:
                    grad_param += l1_lambda * np.sign(param) #
                    L1: gradient += lambda * sign(w)

        # --- Parameter update (SGD) ---
        for param, grad_param in model.params():
            if grad_param is not None:
                param -= lr * grad_param

        # --- Evaluate on test set (eval mode) ---
        model.eval()
        test_logits = model.forward(X_te)
        test_loss = loss_fn.forward(test_logits, y_te)
        test_losses.append(test_loss)

```



```

    # --- Early stopping check ---
    if early_stopping:
        if test_loss < best_test_loss - 1e-4:
            best_test_loss = test_loss
            patience_counter = 0
        else:
            patience_counter += 1
        if patience_counter >= patience:
            stopped_epoch = epoch + 1
            break

    model.eval()
    return train_losses, test_losses, stopped_epoch

def compute_accuracy(model, X, y):
    """Compute classification accuracy."""
    model.eval()
    logits = model.forward(X)
    preds = np.argmax(logits, axis=1)
    return np.mean(preds == y.astype(int))

# Train the large network with no regularization
np.random.seed(42)
model_overfit = build_large_network()
train_losses, test_losses, _ = train_model(
    model_overfit, X_train, y_train, X_test, y_test, epochs=500,
    lr=0.05
)

train_acc = compute_accuracy(model_overfit, X_train, y_train)
test_acc = compute_accuracy(model_overfit, X_test, y_test)
print(f"Train accuracy: {train_acc:.1%}")
print(f"Test accuracy: {test_acc:.1%}")
print(f"Gap: {train_acc - test_acc:.1%}")

# Visualization: Train vs Test loss curves showing the overfitting
gap
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(train_losses, color=PALETTE["royal_blue"], label="Train
Loss", linewidth=2.2)
ax.plot(test_losses, color=PALETTE["coral"], label="Test Loss",
linewidth=2.2)
ax.set_xlabel("Epoch")
ax.set_ylabel("Cross-Entropy Loss")
ax.set_title("Overfitting: Train Loss Drops While Test Loss
Diverges")
ax.legend(fontsize=11)

# Annotate the gap
gap_epoch = 400
ax.annotate("Overfitting gap",

```

```

        xy=(gap_epoch, test_losses[gap_epoch]),
        xytext=(gap_epoch - 120, test_losses[gap_epoch] +
0.15),
        fontsize=10, color=PALETTE["coral"],
        arrowprops=dict(arrowstyle="->",
color=PALETTE["coral"], lw=1.5))

plt.tight_layout()
plt.savefig("../figures/ch08_overfitting_curves.png", dpi=200,
facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

```

The Bias-Variance Tradeoff

Every model's prediction error can be decomposed into three components:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- **Bias** measures how far the model's average prediction is from the truth. High bias means the model is too simple (underfitting).
- **Variance** measures how much predictions change across different training sets. High variance means the model is too sensitive to training data (overfitting).
- **Irreducible noise** is the inherent randomness in the data that no model can capture.

Our large network has **low bias** (it can fit complex patterns) but **high variance** (it memorizes noise). Regularization techniques reduce variance at the cost of slightly increasing bias, finding a better overall tradeoff.

The rest of this chapter introduces five regularization strategies: 1. L1 and L2 regularization (weight penalties) 2. Dropout (random neuron deactivation) 3. Early stopping (halt before overfitting) 4. Data augmentation (expand the training set)

Each attacks overfitting from a different angle. Let us explore them one by one.

8.2 L1 and L2 Regularization

The simplest way to fight overfitting is to **penalize large weights**. The idea: if the loss function rewards both accuracy *and* small weights, the model cannot afford to memorize noise with extreme weight values.

L2 Regularization (Weight Decay)

Add the sum of squared weights to the loss:

$$L_{\text{total}} = L + \frac{\lambda}{2} \sum_i w_i^2$$

The gradient contribution from the penalty term is:

$$\frac{\partial L_{\text{total}}}{\partial w_i} = \frac{\partial L}{\partial w_i} + \lambda w_i$$

This means at each update, every weight is **shrunk toward zero** by a factor proportional to its current value. Larger weights get penalized more.

L1 Regularization (Lasso)

Add the sum of absolute weight values:

$$[L_{\text{total}} = L + \lambda \sum_i |w_i|]$$

The gradient contribution:

$$[\frac{\partial L_{\text{total}}}{\partial w_i} = \frac{\partial L}{\partial w_i} + \lambda \cdot \text{sign}(w_i)]$$

L1 pushes weights toward **exactly zero**, producing sparse weight matrices. This acts as a form of automatic feature selection.

Implementation

Both are implemented as simple gradient modifications in the training loop – see the `train_model` function above. The key lines:

```
# L2: gradient += lambda * w
grad_param += l2_lambda * param

# L1: gradient += lambda * sign(w)
grad_param += l1_lambda * np.sign(param)

# Train with L2 regularization
np.random.seed(42)
model_l2 = build_large_network()
train_losses_l2, test_losses_l2, _ = train_model(
    model_l2, X_train, y_train, X_test, y_test,
    epochs=500, lr=0.05, l2_lambda=0.01
)

# Train with L1 regularization
np.random.seed(42)
model_l1 = build_large_network()
train_losses_l1, test_losses_l1, _ = train_model(
    model_l1, X_train, y_train, X_test, y_test,
    epochs=500, lr=0.05, l1_lambda=0.005
)

print("=== L2 Regularization (lambda=0.01) ===")
print(f" Train acc: {compute_accuracy(model_l2, X_train, y_train):.1%}")
print(f" Test acc: {compute_accuracy(model_l2, X_test, y_test):.1%}")
print()
print("=== L1 Regularization (lambda=0.005) ===")
```

```

print(f" Train acc: {compute_accuracy(model_l1, X_train,
                                     y_train):.1%}")
print(f" Test acc:  {compute_accuracy(model_l1, X_test,
                                     y_test):.1%}")

# Weight distribution histograms: with vs without L2
# regularization
def collect_weights(model):
    """Collect all weight values from Linear layers."""
    weights = []
    for layer in model.layers:
        if isinstance(layer, Linear):
            weights.append(layer.W.ravel())
    return np.concatenate(weights)

weights_no_reg = collect_weights(model_overfit)
weights_l2 = collect_weights(model_l2)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), sharey=True)

# Without regularization
axes[0].hist(weights_no_reg, bins=60,
             color=PALETTE["royal_blue"], alpha=0.8,
             edgecolor="white", linewidth=0.3)
axes[0].set_title("No Regularization")
axes[0].set_xlabel("Weight Value")
axes[0].set_ylabel("Count")
axes[0].axvline(0, color=PALETTE["warm_gray"], linestyle="--",
               alpha=0.6)
axes[0].text(0.95, 0.92, f"std = {weights_no_reg.std():.3f}",
            transform=axes[0].transAxes, ha="right", fontsize=10,
            color=PALETTE["charcoal"])

# With L2
axes[1].hist(weights_l2, bins=60, color=PALETTE["emerald"],
             alpha=0.8,
             edgecolor="white", linewidth=0.3)
axes[1].set_title("With L2 Regularization ( $\lambda=0.01$ )")
axes[1].set_xlabel("Weight Value")
axes[1].axvline(0, color=PALETTE["warm_gray"], linestyle="--",
               alpha=0.6)
axes[1].text(0.95, 0.92, f"std = {weights_l2.std():.3f}",
            transform=axes[1].transAxes, ha="right", fontsize=10,
            color=PALETTE["charcoal"])

fig.suptitle("L2 Regularization Shrinks Weight Magnitudes",
            fontsize=13, fontweight="bold",
            color=PALETTE["deep_blue"], y=1.02)
plt.tight_layout()
plt.savefig("../figures/ch08_weight_histograms.png", dpi=200,
            facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

```

```

# L1 sparsity visualization: bar chart of weight magnitudes
weights_l1_vals = collect_weights(model_l1)

# Sort by magnitude and take a sample for visibility
sorted_mags_no_reg = np.sort(np.abs(weights_no_reg))[:, :-1]
sorted_mags_l1 = np.sort(np.abs(weights_l1_vals))[:, :-1]

# Show first 100 weights (sorted by magnitude)
n_show = 100
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), sharey=True)

axes[0].bar(range(n_show), sorted_mags_no_reg[:n_show],
            color=PALETTE["royal_blue"], alpha=0.8, width=1.0)
axes[0].set_title("No Regularization")
axes[0].set_xlabel("Weight Index (sorted by magnitude)")
axes[0].set_ylabel("|Weight|")
axes[0].axhline(0.01, color=PALETTE["coral"], linestyle="--",
               alpha=0.6, label="Near-zero threshold")
n_near_zero_noreg = np.sum(np.abs(weights_no_reg) < 0.01)
axes[0].text(0.95, 0.85, f"Near-zero: {n_near_zero_noreg}/\n{len(weights_no_reg)}",
            transform=axes[0].transAxes, ha="right", fontsize=10,
            color=PALETTE["charcoal"])

axes[1].bar(range(n_show), sorted_mags_l1[:n_show],
            color=PALETTE["amber"], alpha=0.8, width=1.0)
axes[1].set_title("With L1 Regularization ($\\lambda=0.005$)")
axes[1].set_xlabel("Weight Index (sorted by magnitude)")
axes[1].axhline(0.01, color=PALETTE["coral"], linestyle="--",
               alpha=0.6, label="Near-zero threshold")
n_near_zero_l1 = np.sum(np.abs(weights_l1_vals) < 0.01)
axes[1].text(0.95, 0.85, f"Near-zero: {n_near_zero_l1}/\n{len(weights_l1_vals)}",
            transform=axes[1].transAxes, ha="right", fontsize=10,
            color=PALETTE["charcoal"])

fig.suptitle("L1 Regularization Drives Many Weights to Zero\n(Sparsity)", fontsize=13,
            fontweight="bold", color=PALETTE["deep_blue"],
            y=1.02)
plt.tight_layout()
plt.savefig("../figures/ch08_l1_sparsity.png", dpi=200,
            facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

```

8.3 Dropout

Dropout is arguably the most popular regularization technique specific to neural networks. The idea is beautifully simple:

During training, randomly set each neuron's output to zero with probability p .

This forces the network to be redundant – no single neuron can become a “master feature detector” that the entire network relies on. Each neuron must learn to be useful *in combination with random subsets* of other neurons.

Inverted Dropout

We use **inverted dropout**: during training, we scale the surviving activations by $\frac{1}{1-p}$ so that the expected value of each activation remains the same. This means **at test time, we do nothing** – no scaling needed.

Training: $\tilde{h}_i = \frac{m_i \cdot h_i}{1-p}, \quad m_i \sim \text{Bernoulli}(1-p)$

Test time: $\tilde{h}_i = h_i$

Implementation

Here is our Dropout layer (already in `utils/nn.py`):

```
class Dropout:
    def __init__(self, p=0.5):
        self.p = p          # probability of dropping
        self._mask = None

    def forward(self, x, training=True):
        if training:
            # Create binary mask: 1 with prob (1-p), 0 with prob p
            # Scale by 1/(1-p) for inverted dropout
            self._mask = (np.random.rand(*x.shape) > self.p) / (1 - self.p)
            return x * self._mask
        return x # No dropout at test time

    def backward(self, grad):
        return grad * self._mask # Same mask applied to gradients
```

The key insight: the same mask used in the forward pass is reused in the backward pass. Dropped neurons contribute zero gradient – they are truly “absent” for that training step.

```
# Network visualization: show some neurons "dropped" (grayed out)
import matplotlib.patches as mpatches
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
layer_sizes = [4, 6, 6, 3]
layer_labels = ["Input", "Hidden 1", "Hidden 2", "Output"]
```

```
# Dropout masks for hidden layers (some neurons dropped)
np.random.seed(7)
dropout_masks = {
    1: np.random.rand(6) > 0.5, # ~50% dropped in hidden 1
    2: np.random.rand(6) > 0.5, # ~50% dropped in hidden 2
```

```

}

for panel, (ax, title, use_dropout) in enumerate(zip(
    axes, ["Standard Network (All Active)", "With Dropout
          (p=0.5)"], [False, True]
)):
    ax.set_xlim(-0.5, 4.5)
    ax.set_ylim(-1, max(layer_sizes) + 0.5)
    ax.set_aspect("equal")
    ax.axis("off")
    ax.set_title(title, fontsize=12, fontweight="bold",
                 color=PALETTE["deep_blue"])

    # Compute neuron positions
    positions = []
    for i, n in enumerate(layer_sizes):
        x = i * 1.3
        y_offset = (max(layer_sizes) - n) / 2
        layer_pos = [(x, y_offset + j) for j in range(n)]
        positions.append(layer_pos)

    # Draw connections first
    for i in range(len(layer_sizes) - 1):
        for j, (x1, y1) in enumerate(positions[i]):
            # Check if source neuron is dropped
            src_active = True
            if use_dropout and i in dropout_masks:
                src_active = dropout_masks[i][j]
            for k, (x2, y2) in enumerate(positions[i + 1]):
                # Check if target neuron is dropped
                dst_active = True
                if use_dropout and (i + 1) in dropout_masks:
                    dst_active = dropout_masks[i + 1][k]
                if use_dropout and (not src_active or not
                    dst_active):
                    ax.plot([x1, x2], [y1, y2],
                            color=PALETTE["light_gray"],
                            alpha=0.3, linewidth=0.5, zorder=1)
                else:
                    ax.plot([x1, x2], [y1, y2],
                            color=PALETTE["slate"],
                            alpha=0.3, linewidth=0.8, zorder=1)

    # Draw neurons
    for i, layer_pos in enumerate(positions):
        for j, (x, y) in enumerate(layer_pos):
            active = True
            if use_dropout and i in dropout_masks:
                active = dropout_masks[i][j]

            if active:
                color = PALETTE["royal_blue"] if i <
len(layer_sizes) - 1 else PALETTE["emerald"]

```

```

        circle = plt.Circle((x, y), 0.25, color=color,
zorder=3,
                                edgecolor="white",
linewidth=1.5)
    else:
        circle = plt.Circle((x, y), 0.25,
color=PALETTE["light_gray"], zorder=3,

edgecolor=PALETTE["warm_gray"], linewidth=1.5,
                                linestyle="--")
        ax.text(x, y, "X", ha="center", va="center",
fontsize=8,
                color=PALETTE["warm_gray"],
fontweight="bold", zorder=4)
        ax.add_patch(circle)

ax.text(positions[i][0][0], -0.7, layer_labels[i],
        ha="center", fontsize=9,
color=PALETTE["charcoal"])

# Legend
legend_elements = [
    mpatches.Patch(facecolor=PALETTE["royal_blue"],
        edgecolor="white", label="Active neuron"),
    mpatches.Patch(facecolor=PALETTE["light_gray"],
        edgecolor=PALETTE["warm_gray"],
        label="Dropped neuron", linestyle="--"),
]
axes[1].legend(handles=legend_elements, loc="upper right",
    fontsize=8, frameon=True)

plt.tight_layout()
plt.savefig("../figures/ch08_dropout_network.png", dpi=200,
    facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

# Train with dropout
def build_dropout_network(p=0.3):
    """Same large network but with Dropout after each hidden
    layer."""
    return Sequential(
        Linear(2, 64), ReLU(), Dropout(p),
        Linear(64, 64), ReLU(), Dropout(p),
        Linear(64, 64), ReLU(), Dropout(p),
        Linear(64, 2)
    )

np.random.seed(42)
model_dropout = build_dropout_network(p=0.3)
train_losses_do, test_losses_do, _ = train_model(
    model_dropout, X_train, y_train, X_test, y_test,
    epochs=500, lr=0.05, dropout_model=True
)

```



```

print("=== Dropout (p=0.3) ===")
print(f"  Train acc: {compute_accuracy(model_dropout, X_train,
                                     y_train):.1%}")
print(f"  Test acc: {compute_accuracy(model_dropout, X_test,
                                     y_test):.1%}")

```

8.4 Early Stopping

Early stopping is the simplest regularization technique: **stop training when the validation loss stops improving**.

The reasoning is straightforward from the overfitting plot in Section 8.1: - Early in training, both train and test loss decrease (the model is learning the true pattern). - After some point, train loss keeps dropping but test loss starts increasing (the model begins memorizing noise). - The optimal model is at the **turning point** where test loss is minimized.

Patience

We do not stop at the very first epoch where test loss increases – that could be a temporary fluctuation. Instead, we use a **patience** parameter: we stop only after the test loss has failed to improve for patience consecutive epochs.

```

if test_loss < best_test_loss - 1e-4:
    best_test_loss = test_loss
    patience_counter = 0      # Reset: we saw improvement
else:
    patience_counter += 1    # No improvement

if patience_counter >= patience:
    break                  # Stop training

# Train with early stopping
np.random.seed(42)
model_early = build_large_network()
train_losses_es, test_losses_es, stopped_epoch = train_model(
    model_early, X_train, y_train, X_test, y_test,
    epochs=500, lr=0.05, early_stopping=True, patience=30
)

print(f"=== Early Stopping (patience=30) ===")
print(f"  Stopped at epoch: {stopped_epoch}")
print(f"  Train acc: {compute_accuracy(model_early, X_train,
                                     y_train):.1%}")
print(f"  Test acc: {compute_accuracy(model_early, X_test,
                                     y_test):.1%}")

# Visualization: Loss curve with vertical line where early
#                 stopping triggers
fig, ax = plt.subplots(figsize=(8, 5))

```

```

# Also show what would have happened without early stopping (use
  overfit curves)
ax.plot(train_losses, color=PALETTE["royal_blue"], alpha=0.3,
        linewidth=1.5,
        label="Train (no early stop)", linestyle="--")
ax.plot(test_losses, color=PALETTE["coral"], alpha=0.3,
        linewidth=1.5,
        label="Test (no early stop)", linestyle="--")

# Early-stopped curves
ax.plot(train_losses_es, color=PALETTE["royal_blue"],
        linewidth=2.2,
        label="Train (early stopping)")
ax.plot(test_losses_es, color=PALETTE["coral"], linewidth=2.2,
        label="Test (early stopping)")

# Vertical line at stop point
ax.axvline(stopped_epoch - 1, color=PALETTE["emerald"],
          linestyle="-",
          linewidth=2.5, alpha=0.8, label=f"Early stop (epoch
            {stopped_epoch})")

ax.set_xlabel("Epoch")
ax.set_ylabel("Cross-Entropy Loss")
ax.set_title("Early Stopping: Halt Before Overfitting")
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig("../figures/ch08_early_stopping.png", dpi=200,
          facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

```

8.5 Data Augmentation Preview

Another powerful way to fight overfitting is to **expand the training set** with transformed copies of existing data. The model sees more variation during training, which reduces its tendency to memorize specific examples.

For image data, common augmentations include flipping, rotation, cropping, and color jitter. For our simple 2D data, we can:

- **Add noise:** perturb each point slightly
- **Rotate:** apply small random rotations around the origin
- **Scale:** slightly stretch or compress

Let us visualize how a single point looks after several augmentations.

```

# Data augmentation for 2D data
def augment_2d(X, y, n_augmented=5, noise_std=0.1,
              max_rotation_deg=15, seed=42):
    """Generate augmented copies of 2D data points."""
    rng = np.random.default_rng(seed)
    X_aug_all, y_aug_all = [X.copy()], [y.copy()]

```

```

for _ in range(n_augmented):
    X_copy = X.copy()
    # Add Gaussian noise
    X_copy += rng.normal(0, noise_std, X_copy.shape)
    # Apply small random rotation
    angle = rng.uniform(-max_rotation_deg, max_rotation_deg)
    * np.pi / 180
    cos_a, sin_a = np.cos(angle), np.sin(angle)
    R = np.array([[cos_a, -sin_a], [sin_a, cos_a]])
    X_copy = X_copy @ R.T
    X_aug_all.append(X_copy)
    y_aug_all.append(y.copy())

return np.vstack(X_aug_all), np.concatenate(y_aug_all)

```

```

# Show original + 5 augmented versions for a few sample points
fig, axes = plt.subplots(2, 3, figsize=(13, 8))
axes = axes.ravel()

```

```

# Pick 6 sample points (3 from each class)
np.random.seed(0)
sample_indices = [0, 5, 10, 45, 50, 55]

```

```

for idx, (ax, si) in enumerate(zip(axes, sample_indices)):
    original = X_train[si:si+1]
    label = y_train[si]
    color = PALETTE["royal_blue"] if label == 0 else
        PALETTE["coral"]

```

```

# Generate augmented versions
augmented_points = []
for aug_seed in range(5):
    rng = np.random.default_rng(aug_seed + idx * 100)
    pt = original.copy()
    pt += rng.normal(0, 0.12, pt.shape)
    angle = rng.uniform(-20, 20) * np.pi / 180
    cos_a, sin_a = np.cos(angle), np.sin(angle)
    R = np.array([[cos_a, -sin_a], [sin_a, cos_a]])
    pt = pt @ R.T
    augmented_points.append(pt[0])

```

```

# Plot all augmented versions
for i, ap in enumerate(augmented_points):
    ax.scatter(ap[0], ap[1], c=color, alpha=0.4, s=80,
        edgecolors="white",
        linewidth=0.5, zorder=2)

```

```

# Plot original (larger, on top)
ax.scatter(original[0, 0], original[0, 1], c=color, s=150,
    edgecolors=PALETTE["charcoal"], linewidth=2,
    zorder=3, marker="*")

```

```

ax.set_title(f"Sample {si} (Class {label})", fontsize=10)
ax.set_xlim(original[0, 0] - 0.6, original[0, 0] + 0.6)
ax.set_ylim(original[0, 1] - 0.6, original[0, 1] + 0.6)
ax.set_aspect("equal")

# Add legend to the first subplot
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], marker="*", color="w",
            markerfacecolor=PALETTE["slate"],
            markersize=12, label="Original"),
    Line2D([0], [0], marker="o", color="w",
            markerfacecolor=PALETTE["slate"],
            markersize=8, alpha=0.5, label="Augmented"),
]
axes[0].legend(handles=legend_elements, fontsize=8, loc="upper
left")

fig.suptitle("Data Augmentation: Original Points and Their
Augmented Versions",
             fontsize=13, fontweight="bold",
             color=PALETTE["deep_blue"], y=1.01)
plt.tight_layout()
plt.savefig("../figures/ch08_augmentation_grid.png", dpi=200,
            facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

```

8.6 Regularization Comparison (Hero Figure)

Now let us put everything together. We train the **same architecture** on the **same noisy dataset** using six different strategies:

1. **No regularization** (baseline)
2. **L1 regularization** ($\lambda = 0.005$)
3. **L2 regularization** ($\lambda = 0.01$)
4. **Dropout** ($p = 0.3$)
5. **Early stopping** (patience = 30)
6. **L2 + Dropout** (combined)

For each, we plot the decision boundary and report test accuracy.

=== Hero Figure: 2x3 Grid of Decision Boundaries ===

```

configs = [
    {"name": "No Regularization", "l2": 0.0, "l1": 0.0,
     "dropout": False, "p": 0.0, "early": False},
    {"name": "L1 ( $\lambda=0.005$ )", "l2": 0.0, "l1": 0.005,
     "dropout": False, "p": 0.0, "early": False},
    {"name": "L2 ( $\lambda=0.01$ )", "l2": 0.01, "l1": 0.0,
     "dropout": False, "p": 0.0, "early": False},
    {"name": "Dropout (p=0.3)", "l2": 0.0, "l1": 0.0,
     "dropout": True, "p": 0.3, "early": False},

```

```

        {"name": "Early Stopping",          "l2": 0.0,   "l1": 0.0,
         "dropout": False, "p": 0.0,   "early": True},
        {"name": "L2 + Dropout",           "l2": 0.01,  "l1": 0.0,
         "dropout": True,  "p": 0.3,   "early": False},
    ]

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.ravel()

for i, cfg in enumerate(configs):
    np.random.seed(42)

    if cfg["dropout"]:
        model = Sequential(
            Linear(2, 64), ReLU(), Dropout(cfg["p"]),
            Linear(64, 64), ReLU(), Dropout(cfg["p"]),
            Linear(64, 64), ReLU(), Dropout(cfg["p"]),
            Linear(64, 2)
        )
    else:
        model = build_large_network()

    _, _, stopped = train_model(
        model, X_train, y_train, X_test, y_test,
        epochs=500, lr=0.05,
        l2_lambda=cfg["l2"], l1_lambda=cfg["l1"],
        dropout_model=cfg["dropout"],
        early_stopping=cfg["early"], patience=30
    )

    acc = compute_accuracy(model, X_test, y_test)

    # Plot decision boundary
    plot_decision_boundary(model, X_test, y_test, ax=axes[i],
        resolution=150)

    title = cfg["name"]
    if cfg["early"]:
        title += f" (epoch {stopped})"
    axes[i].set_title(f"{title}\nTest Acc: {acc:.1%}",
        fontsize=11,
        fontweight="bold",
        color=PALETTE["deep_blue"])

fig.suptitle("Regularization Comparison: Same Network, Same Data,
    Different Strategies",
        fontsize=15, fontweight="bold",
        color=PALETTE["deep_blue"], y=1.02)
plt.tight_layout()
plt.savefig("../figures/ch08_regularization_comparison.png",
    dpi=200,
    facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

```

What to Observe

- **No regularization:** The decision boundary is highly irregular, capturing noise in the training data.
- **L1:** The boundary is smoother because many weights are driven to zero, simplifying the model.
- **L2:** The boundary is smoother because all weights are kept small, limiting the model's expressiveness.
- **Dropout:** The boundary is smoother because the network must learn redundant representations.
- **Early stopping:** The boundary is simpler because training halted before the model could memorize noise.
- **L2 + Dropout:** Combining techniques often gives the smoothest boundary and best generalization.

In practice, **combining multiple regularization methods** (e.g., weight decay + dropout + early stopping) is common and often gives the best results.

8.7 Finding Optimal Lambda

How do we choose the regularization strength λ ? Too small and we still overfit. Too large and we underfit (the penalty dominates and the model cannot learn).

The standard approach: **grid search** over candidate λ values and evaluate on a validation set.

```
# Grid search over lambda values for L2 regularization
lambdas = [0, 0.001, 0.01, 0.1, 1.0]
train_accs = []
val_accs = []

for lam in lambdas:
    np.random.seed(42)
    model = build_large_network()
    train_model(
        model, X_train, y_train, X_test, y_test,
        epochs=400, lr=0.05, l2_lambda=lam
    )
    tr_acc = compute_accuracy(model, X_train, y_train)
    te_acc = compute_accuracy(model, X_test, y_test)
    train_accs.append(tr_acc)
    val_accs.append(te_acc)
    print(f"lambda={lam:<6.3f}  Train: {tr_acc:.1%}  Val: {te_acc:.1%}")

best_idx = np.argmax(val_accs)
print(f"\nBest lambda: {lambdas[best_idx]} (Val acc: {val_accs[best_idx]:.1%}")

# Plot train and validation accuracy vs lambda
fig, ax = plt.subplots(figsize=(8, 5))
```

```

x_positions = range(len(lambdas))
ax.plot(x_positions, train_accs, 'o-',
        color=PALETTE["royal_blue"],
        label="Train Accuracy", linewidth=2.2, markersize=8)
ax.plot(x_positions, val_accs, 's-', color=PALETTE["coral"],
        label="Validation Accuracy", linewidth=2.2, markersize=8)

# Mark the best lambda
ax.axvline(best_idx, color=PALETTE["emerald"], linestyle="--",
          linewidth=2, alpha=0.7,
          label=f"Best  $\lambda$  = {lambdas[best_idx]}")

# Annotate regions
ax.annotate("Overfitting\n(too little reg.)", xy=(0.3, 0.5),
           xycoords="axes fraction", fontsize=10, ha="center",
           color=PALETTE["warm_gray"], style="italic")
ax.annotate("Underfitting\n(too much reg.)", xy=(0.85, 0.5),
           xycoords="axes fraction", fontsize=10, ha="center",
           color=PALETTE["warm_gray"], style="italic")

ax.set_xticks(x_positions)
ax.set_xticklabels([str(l) for l in lambdas])
ax.set_xlabel("Regularization Strength ( $\lambda$ )")
ax.set_ylabel("Accuracy")
ax.set_title("Finding the Optimal  $\lambda$ : The Sweet Spot")
ax.legend(fontsize=10)
ax.set_ylim(0.4, 1.05)

plt.tight_layout()
plt.savefig("../figures/ch08_optimal_lambda.png", dpi=200,
           facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

```

Key Takeaway

The plot reveals the classic U-shaped validation curve:

- **λ too small** (left): Training accuracy is near-perfect but validation lags behind. The model overfits.
- **λ too large** (right): Both accuracies are low. The penalty overwhelms the learning signal. The model underfits.
- **Sweet spot** (middle): Validation accuracy is maximized. The model generalizes well.

In practice, you would typically search over a finer grid (or use logarithmic spacing) and use proper cross-validation rather than a single train/test split.

8.8 Exercises

Exercise 1: Dropout Rate Sensitivity

Train the same network with dropout rates $(p \in \{0.1, 0.2, 0.3, 0.5, 0.7, 0.9\})$. Plot test accuracy vs. dropout rate. At what value of (p) does the network start to underfit? Why does $(p=0.9)$ (dropping 90% of neurons) hurt performance?

Exercise 2: L1 vs L2 Weight Analysis

Train two networks on the `make_spirals` dataset: one with L1 regularization ($(\lambda = 0.01)$) and one with L2 regularization ($(\lambda = 0.01)$). After training: - Count the number of weights with $(|w| < 0.001)$ in each model. - Plot overlapping histograms of weight distributions for both models. - Explain why L1 produces more exact zeros than L2 by examining the gradient of each penalty at $(w = 0)$.

Exercise 3: Combining Everything

Implement a training pipeline that combines L2 regularization, dropout, early stopping, and data augmentation. Use the `make_spirals` dataset (which is harder than moons). Experiment with hyperparameters to achieve the best possible test accuracy. Report: - Your best hyperparameter combination - The train and test accuracy - A plot of the decision boundary

Hint: Data augmentation means generating new augmented batches *each epoch*, not once at the start.

Next: *Chapter 9 – Convolutional Networks*

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
from utils.nn import Linear, ReLU, Sequential, BatchNorm,
    LayerNorm, CrossEntropyLoss
from utils.data import make_spirals, train_test_split
from utils.plot import plot_loss_curve
apply_style()
np.random.seed(42)
```

Chapter 9: Batch Normalization

Taming the Internal Chaos of Deep Networks

Learning Objectives

By the end of this chapter, you will:

- Understand **internal covariate shift** and why it makes deep networks hard to train

- Derive and implement **Batch Normalization** from scratch, including the full backward pass
- Verify your gradient computation with **numerical gradient checking**
- Visualize how BatchNorm stabilizes **activation distributions** across layers
- See how BatchNorm enables **higher learning rates** and faster convergence
- Analyze **gradient flow** through deep networks with and without normalization
- Understand **Layer Normalization** as an alternative for small-batch and sequence settings
- Know when to choose BatchNorm vs LayerNorm vs other normalization schemes

9.1 Internal Covariate Shift

Training deep networks is fundamentally a game of **coordination**. Each layer learns to transform its input into a useful representation for the next layer. But here is the problem:

As the parameters of earlier layers change during training, the **distribution of inputs** to every subsequent layer shifts. Each layer must constantly re-adapt to a moving target.

This phenomenon is called **internal covariate shift** (Ioffe & Szegedy, 2015). It has several practical consequences:

- **Slower convergence**: Layers spend optimization effort tracking shifting distributions rather than learning useful features.
- **Requires careful initialization**: If activations drift too far (saturating sigmoids, exploding ReLUs), gradients vanish or explode.
- **Demands small learning rates**: Large updates amplify the distribution shift, causing instability.

The Core Idea

What if we could **guarantee** that each layer receives inputs with a stable distribution? Specifically, what if we normalized each layer's inputs to have zero mean and unit variance?

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

This is exactly what **Batch Normalization** does. It normalizes the inputs to each layer using statistics computed over the current mini-batch, then adds learnable parameters to recover expressive power.

The result: we can use **higher learning rates**, worry less about initialization, and train deeper networks more reliably.

9.2 The Batch Normalization Transform

Given a mini-batch $\mathcal{B} = \{x_1, x_2, \dots, x_N\}$ of N samples, Batch Normalization operates in two stages:

Stage 1: Normalize

Compute the mean and variance of each feature across the batch:

$$\begin{aligned}\mu_{\mathcal{B}} &= \frac{1}{N} \sum_{i=1}^N x_i \\ \sigma^2_{\mathcal{B}} &= \frac{1}{N} \sum_{i=1}^N (x_i - \mu_{\mathcal{B}})^2\end{aligned}$$

Then normalize:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}$$

where ϵ is a small constant (e.g., 10^{-5}) for numerical stability.

Stage 2: Scale and Shift

If we only normalized, we would limit the representation power of each layer (e.g., forcing all activations near zero would break ReLU). So we introduce two **learnable parameters** per feature:

$$y_i = \gamma \cdot \hat{x}_i + \beta$$

- γ (scale) is initialized to 1
- β (shift) is initialized to 0

The network can learn $\gamma = \sigma_{\mathcal{B}}$ and $\beta = \mu_{\mathcal{B}}$ to recover the original activations if that is optimal. So BatchNorm never *reduces* representational power.

Training vs Inference

During **training**, we use batch statistics ($\mu_{\mathcal{B}}$, $\sigma^2_{\mathcal{B}}$).

During **inference**, we use **running averages** accumulated during training:

$$\begin{aligned}\mu_{\text{run}} &\leftarrow (1 - m) \cdot \mu_{\text{run}} + m \cdot \mu_{\mathcal{B}} \\ \sigma^2_{\text{run}} &\leftarrow (1 - m) \cdot \sigma^2_{\text{run}} + m \cdot \sigma^2_{\mathcal{B}}\end{aligned}$$

where m is the momentum (typically 0.1). This ensures deterministic predictions at test time.

```
class BatchNormScratch:
```

```
    """
```

```
    Batch Normalization implemented from scratch.
```

```
    Normalizes across the batch dimension (axis=0), then applies  
    learnable scale (gamma) and shift (beta) parameters.
```

```
    """
```

```
    def __init__(self, n_features, momentum=0.1, eps=1e-5):
```

```

# Learnable parameters
self.gamma = np.ones((1, n_features))    # scale
self.beta = np.zeros((1, n_features))    # shift
self.eps = eps
self.momentum = momentum

# Running statistics for inference
self.running_mean = np.zeros((1, n_features))
self.running_var = np.ones((1, n_features))

# Gradients
self.grad_gamma = None
self.grad_beta = None

# Cache for backward pass
self._cache = None
self.training = True

def forward(self, x, training=True):
    if training:
        # Step 1: Compute batch statistics
        mean = np.mean(x, axis=0, keepdims=True)    # (1,
        D)
        var = np.var(x, axis=0, keepdims=True)    # (1,
        D)

        # Step 2: Update running statistics (for inference
        later)
        self.running_mean = (1 - self.momentum) *
self.running_mean + self.momentum * mean
        self.running_var = (1 - self.momentum) *
self.running_var + self.momentum * var

        # Step 3: Normalize
        x_norm = (x - mean) / np.sqrt(var + self.eps)    # (N,
        D)

        # Save for backward pass
        self._cache = (x, x_norm, mean, var)
    else:
        # Use running statistics at inference time
        x_norm = (x - self.running_mean) /
np.sqrt(self.running_var + self.eps)

        # Step 4: Scale and shift
        return self.gamma * x_norm + self.beta

def backward(self, grad_output):
    """
    Full backward pass through BatchNorm.

    Given dL/dy (grad_output), compute:
    - dL/dgamma, dL/dbeta (parameter gradients)
    - dL/dx (gradient to propagate further)

```

```

"""
x, x_norm, mean, var = self._cache
N = x.shape[0]

# --- Parameter gradients ---
# dL/dgamma = sum_i (dL/dy_i * x_hat_i)
self.grad_gamma = np.sum(grad_output * x_norm, axis=0,
keepdims=True)

# dL/dbeta = sum_i (dL/dy_i)
self.grad_beta = np.sum(grad_output, axis=0,
keepdims=True)

# --- Input gradient (the hard part) ---
# dL/dx_hat = dL/dy * gamma
dx_norm = grad_output * self.gamma

# dL/dvar = sum_i (dL/dx_hat_i * (x_i - mu) * -0.5 * (var
+ eps)^(-3/2))
dvar = np.sum(dx_norm * (x - mean) * -0.5 * (var +
self.eps) ** (-1.5),
axis=0, keepdims=True)

# dL/dmean = sum_i (dL/dx_hat_i * -1/sqrt(var+eps)) + dvar
* mean(-2(x-mu))
dmean = np.sum(dx_norm * -1.0 / np.sqrt(var + self.eps),
axis=0, keepdims=True) + \
dvar * np.mean(-2.0 * (x - mean), axis=0,
keepdims=True)

# dL/dx_i = dL/dx_hat_i / sqrt(var+eps) + dvar * 2(x_i -
mu)/N + dmean/N
dx = dx_norm / np.sqrt(var + self.eps) + dvar * 2.0 * (x
- mean) / N + dmean / N

return dx

def params(self):
    return [(self.gamma, self.grad_gamma), (self.beta,
self.grad_beta)]

# Quick test: verify shapes
bn_test = BatchNormScratch(n_features=4)
x_test = np.random.randn(8, 4) # batch of 8, 4 features
y_test = bn_test.forward(x_test, training=True)
print(f"Input shape: {x_test.shape}")
print(f"Output shape: {y_test.shape}")
print(f"Output mean (per feature):
{y_test.mean(axis=0).round(6)}")
print(f"Output std (per feature): {y_test.std(axis=0).round(4)}")

```

9.3 The Backward Pass – Derivation

The BatchNorm backward pass is one of the trickiest gradient derivations in deep learning. Let us walk through it step by step.

Forward pass recap (introducing named intermediate variables):

1. $\mu = \frac{1}{N} \sum_i x_i$
2. $\sigma^2 = \frac{1}{N} \sum_i (x_i - \mu)^2$
3. $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$
4. $y_i = \gamma \hat{x}_i + \beta$

Backward pass – given $\frac{\partial L}{\partial y_i}$, find $\frac{\partial L}{\partial x_i}$, $\frac{\partial L}{\partial \gamma}$, $\frac{\partial L}{\partial \beta}$:

Step 1: Parameter gradients (easy): $\frac{\partial L}{\partial \gamma} = \sum_i \frac{\partial L}{\partial y_i} \hat{x}_i$
 $\frac{\partial L}{\partial \beta} = \sum_i \frac{\partial L}{\partial y_i}$

Step 2: Gradient through scale: $\frac{\partial L}{\partial y_i} \frac{\partial \hat{x}_i}{\partial \gamma} = \frac{\partial L}{\partial y_i}$

Step 3: Gradient through the normalization. Since $\hat{x}_i$ depends on x_i both directly and through μ and σ^2 , we need to apply the chain rule carefully:

$$\frac{\partial L}{\partial \sigma^2} = \sum_i \frac{\partial L}{\partial \hat{x}_i} \frac{\partial \hat{x}_i}{\partial \sigma^2} = \sum_i \frac{\partial L}{\partial \hat{x}_i} \frac{-1}{2(\sigma^2 + \epsilon)^{3/2}}$$

$$\frac{\partial L}{\partial \mu} = \sum_i \frac{\partial L}{\partial \hat{x}_i} \frac{\partial \hat{x}_i}{\partial \mu} = \sum_i \frac{\partial L}{\partial \hat{x}_i} \frac{-1}{\sqrt{\sigma^2 + \epsilon}} + \frac{\partial L}{\partial \sigma^2} \frac{\partial \sigma^2}{\partial \mu} = \sum_i \frac{\partial L}{\partial \hat{x}_i} \frac{-1}{\sqrt{\sigma^2 + \epsilon}} + \frac{\partial L}{\partial \sigma^2} \frac{2}{N} \sum_i (x_i - \mu)$$

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial \hat{x}_i} \frac{\partial \hat{x}_i}{\partial x_i} = \frac{\partial L}{\partial \hat{x}_i} \frac{1}{\sqrt{\sigma^2 + \epsilon}} + \frac{\partial L}{\partial \sigma^2} \frac{\partial \sigma^2}{\partial x_i} = \frac{\partial L}{\partial \hat{x}_i} \frac{1}{\sqrt{\sigma^2 + \epsilon}} + \frac{\partial L}{\partial \sigma^2} \frac{2(x_i - \mu)}{N} + \frac{\partial L}{\partial \mu} \frac{\partial \mu}{\partial x_i} = \frac{\partial L}{\partial \hat{x}_i} \frac{1}{\sqrt{\sigma^2 + \epsilon}} + \frac{\partial L}{\partial \sigma^2} \frac{2(x_i - \mu)}{N} + \frac{\partial L}{\partial \mu} \frac{1}{N}$$

The key insight: every x_i influences μ and σ^2 , which in turn affect *all* $\hat{x}_j$. This coupling makes the gradient non-trivial.

```
def gradient_check_bn(bn_layer, x, eps=1e-5):
    """
    Numerical gradient check for our BatchNorm implementation.
    Compare analytical gradients (backward) with numerical
    gradients (finite differences).
    """
    # Forward pass
    y = bn_layer.forward(x, training=True)

    # Use sum as a simple loss so dL/dy = ones
    grad_output = np.ones_like(y)

    # Analytical backward pass
    dx_analytical = bn_layer.backward(grad_output)
```

```

# Numerical gradient for dx
dx_numerical = np.zeros_like(x)
for i in range(x.shape[0]):
    for j in range(x.shape[1]):
        x_plus = x.copy()
        x_plus[i, j] += eps
        y_plus = bn_layer.forward(x_plus, training=True)
        loss_plus = np.sum(y_plus)

        x_minus = x.copy()
        x_minus[i, j] -= eps
        y_minus = bn_layer.forward(x_minus, training=True)
        loss_minus = np.sum(y_minus)

        dx_numerical[i, j] = (loss_plus - loss_minus) / (2 *
eps)

# Restore cache by running forward again with original x
bn_layer.forward(x, training=True)

# Compare
diff = np.abs(dx_analytical - dx_numerical)
denom = np.maximum(np.abs(dx_analytical) +
    np.abs(dx_numerical), 1e-8)
relative_error = np.max(diff / denom)

return relative_error, dx_analytical, dx_numerical

# Run gradient check
np.random.seed(42)
bn_check = BatchNormScratch(n_features=3)
x_check = np.random.randn(5, 3)

rel_err, dx_ana, dx_num = gradient_check_bn(bn_check, x_check)

print("Gradient Check for BatchNorm backward pass")
print("=" * 48)
print(f"Max relative error: {rel_err:.2e}")
print(f"Status: {'PASS' if rel_err < 1e-5 else 'FAIL'}")
print()
print("Analytical gradient (sample):")
print(dx_ana.round(6))
print("\nNumerical gradient (sample):")
print(dx_num.round(6))

```

9.4 Activation Distributions – The Hero Figure

The most compelling way to understand BatchNorm is to **see** its effect on activations. We will build a 10-layer network, pass data through it, and record the activation distribution at each layer.

Without BatchNorm: activations tend to collapse toward zero (vanishing) or spread wildly (exploding), depending on the initialization and depth.

With BatchNorm: activations maintain a healthy distribution at every layer.

```
# -----  
# Build two 10-layer networks: one with BN, one without  
# -----  
np.random.seed(42)  
  
n_samples = 512  
input_dim = 20  
hidden_dim = 64  
n_layers = 10  
  
X_hero = np.random.randn(n_samples, input_dim)  
  
# --- Network WITHOUT BatchNorm ---  
layers_no_bn = []  
for i in range(n_layers):  
    d_in = input_dim if i == 0 else hidden_dim  
    layers_no_bn.append(Linear(d_in, hidden_dim))  
    layers_no_bn.append(ReLU())  
  
# --- Network WITH BatchNorm ---  
layers_with_bn = []  
for i in range(n_layers):  
    d_in = input_dim if i == 0 else hidden_dim  
    layers_with_bn.append(Linear(d_in, hidden_dim))  
    layers_with_bn.append(BatchNorm(hidden_dim))  
    layers_with_bn.append(ReLU())  
  
# Use the same weights in both networks for a fair comparison  
np.random.seed(123)  
for i in range(n_layers):  
    d_in = input_dim if i == 0 else hidden_dim  
    W = np.random.randn(d_in, hidden_dim) * np.sqrt(2.0 / d_in)  
    b = np.zeros((1, hidden_dim))  
    layers_no_bn[i * 2].W = W.copy()  
    layers_no_bn[i * 2].b = b.copy()  
    layers_with_bn[i * 3].W = W.copy()  
    layers_with_bn[i * 3].b = b.copy()  
  
# --- Record activations ---  
def record_activations(layers, x):  
    """Pass data through layers and record output after each  
    ReLU."""  
    activations = []  
    h = x.copy()  
    for layer in layers:  
        h = layer.forward(h)  
        if isinstance(layer, ReLU):  
            activations.append(h.copy())
```

```

    return activations

acts_no_bn = record_activations(layers_no_bn, X_hero)
acts_with_bn = record_activations(layers_with_bn, X_hero)

# -----
# Hero Figure: 2 rows x 5 columns of activation histograms
# -----
display_layers = [0, 2, 4, 6, 8] # layers 1, 3, 5, 7, 9

fig, axes = plt.subplots(2, 5, figsize=(16, 6))
fig.patch.set_facecolor(PALETTE["paper"])

for col, layer_idx in enumerate(display_layers):
    # Top row: WITHOUT BatchNorm
    ax_top = axes[0, col]
    data_no = acts_no_bn[layer_idx].ravel()
    ax_top.hist(data_no, bins=50, color=PALETTE["coral"],
                alpha=0.85, edgecolor="white", linewidth=0.3)
    ax_top.set_title(f"Layer {layer_idx + 1}", fontsize=10)
    ax_top.set_xlim(-1, 5)
    ax_top.set_yticks([])
    if col == 0:
        ax_top.set_ylabel("Without BN", fontsize=11,
                          fontweight="bold", color=PALETTE["coral"])

    # Bottom row: WITH BatchNorm
    ax_bot = axes[1, col]
    data_bn = acts_with_bn[layer_idx].ravel()
    ax_bot.hist(data_bn, bins=50, color=PALETTE["royal_blue"],
                alpha=0.85, edgecolor="white", linewidth=0.3)
    ax_bot.set_xlim(-1, 5)
    ax_bot.set_yticks([])
    if col == 0:
        ax_bot.set_ylabel("With BN", fontsize=11,
                          fontweight="bold", color=PALETTE["royal_blue"])

fig.suptitle("Activation Distributions: Without vs With Batch
Normalization",
             fontsize=14, fontweight="bold",
             color=PALETTE["deep_blue"], y=1.02)

# Add annotation
fig.text(0.5, -0.02,
        "Without BN, activations collapse toward zero in deeper\n"
        "layers. "\n
        "With BN, activations maintain healthy distributions\n"
        "throughout.",
        ha="center", fontsize=10, color=PALETTE["slate"],
        style="italic")

plt.tight_layout()
plt.savefig("../figures/ch09_activation_distributions.png",
            dpi=200,

```



```

        facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

# Print summary statistics
print("\nActivation statistics (mean / std):")
print(f"{'Layer':<10} {'Without BN':>20} {'With BN':>20}")
print("-" * 52)
for i in display_layers:
    m_no, s_no = acts_no_bn[i].mean(), acts_no_bn[i].std()
    m_bn, s_bn = acts_with_bn[i].mean(), acts_with_bn[i].std()
    print(f"Layer {i+1:<4} {m_no:>8.4f} / {s_no:<8.4f}
          {m_bn:>8.4f} / {s_bn:<8.4f}")

```

9.5 Training with BatchNorm

Now let us see BatchNorm in action on a real training task. We will train two networks on the spirals dataset – one with BatchNorm and one without – and compare their loss curves.

The key claim: **BatchNorm enables higher learning rates and faster convergence.**

```

# -----
# Prepare data
# -----
np.random.seed(42)
X, y = make_spirals(n=600, n_classes=3, noise=0.25, seed=42)
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_ratio=0.2, seed=42)

# -----
# Training helper
# -----
def train_network(model, X_train, y_train, loss_fn, lr=0.01,
                  epochs=200, batch_size=64):
    """Train a model and return loss history."""
    losses = []
    n = len(X_train)

    for epoch in range(epochs):
        # Shuffle
        idx = np.random.permutation(n)
        X_shuf = X_train[idx]
        y_shuf = y_train[idx]

        epoch_loss = 0.0
        n_batches = 0

        for start in range(0, n, batch_size):
            end = min(start + batch_size, n)
            X_batch = X_shuf[start:end]
            y_batch = y_shuf[start:end]

```

```

        # Forward
        model.train()
        logits = model.forward(X_batch)
        loss = loss_fn.forward(logits, y_batch)
        epoch_loss += loss
        n_batches += 1

        # Backward
        grad = loss_fn.backward()
        model.backward(grad)

        # Update (simple SGD)
        for param, grad_p in model.params():
            if grad_p is not None:
                param -= lr * grad_p

    losses.append(epoch_loss / n_batches)

    return losses

# -----
# Build two networks
# -----
def make_model_no_bn(seed=42):
    np.random.seed(seed)
    return Sequential(
        Linear(2, 64), ReLU(),
        Linear(64, 64), ReLU(),
        Linear(64, 64), ReLU(),
        Linear(64, 64), ReLU(),
        Linear(64, 3)
    )

def make_model_with_bn(seed=42):
    np.random.seed(seed)
    return Sequential(
        Linear(2, 64), BatchNorm(64), ReLU(),
        Linear(64, 64), BatchNorm(64), ReLU(),
        Linear(64, 64), BatchNorm(64), ReLU(),
        Linear(64, 64), BatchNorm(64), ReLU(),
        Linear(64, 3)
    )

# -----
# Train both with lr=0.1 (high learning rate)
# -----
lr_high = 0.1
epochs = 150

model_no_bn = make_model_no_bn()
model_with_bn = make_model_with_bn()
loss_fn = CrossEntropyLoss()

```

```

print(f"Training WITHOUT BatchNorm (lr={lr_high})...")
losses_no_bn = train_network(model_no_bn, X_train, y_train,
                             loss_fn, lr=lr_high, epochs=epochs)

print(f"Training WITH BatchNorm (lr={lr_high})...")
loss_fn2 = CrossEntropyLoss()
losses_with_bn = train_network(model_with_bn, X_train, y_train,
                              loss_fn2, lr=lr_high, epochs=epochs)

# -----
# Plot comparison
# -----
fig, ax = plt.subplots(figsize=(9, 5))

ax.plot(losses_no_bn, color=PALETTE["coral"], linewidth=2.2,
        label=f"Without BN (lr={lr_high})", alpha=0.9)
ax.plot(losses_with_bn, color=PALETTE["royal_blue"],
        linewidth=2.2, label=f"With BN (lr={lr_high})", alpha=0.9)

ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.set_title("Training Loss: BatchNorm Enables Higher Learning
              Rates")
ax.legend(fontsize=11)
ax.set_ylim(0, max(min(losses_no_bn[0], 5),
                  max(losses_with_bn[0], 2)) * 1.1)

plt.tight_layout()
plt.savefig("../figures/ch09_training_comparison.png", dpi=200,
           facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

# Final accuracy
def accuracy(model, X, y):
    model.eval()
    preds = np.argmax(model.forward(X), axis=1)
    model.train()
    return np.mean(preds == y)

print(f"\nFinal test accuracy (Without BN):
      {accuracy(model_no_bn, X_test, y_test):.3f}")
print(f"Final test accuracy (With BN):
      {accuracy(model_with_bn, X_test, y_test):.3f}")

```

9.6 Higher Learning Rates

One of the most important practical benefits of BatchNorm is that it allows **much higher learning rates** without divergence. Let us verify this systematically by training with several learning rates, with and without BatchNorm.

```

# -----
# Learning rate sweep

```

```

# -----
learning_rates = [0.001, 0.01, 0.1]
epochs_sweep = 150

results_no_bn = {}
results_with_bn = {}

for lr in learning_rates:
    # Without BN
    model = make_model_no_bn(seed=42)
    loss_fn_tmp = CrossEntropyLoss()
    losses = train_network(model, X_train, y_train, loss_fn_tmp,
                           lr=lr, epochs=epochs_sweep)
    acc = accuracy(model, X_test, y_test)
    results_no_bn[lr] = {"losses": losses, "accuracy": acc}

    # With BN
    model = make_model_with_bn(seed=42)
    loss_fn_tmp = CrossEntropyLoss()
    losses = train_network(model, X_train, y_train, loss_fn_tmp,
                           lr=lr, epochs=epochs_sweep)
    acc = accuracy(model, X_test, y_test)
    results_with_bn[lr] = {"losses": losses, "accuracy": acc}

# -----
# Figure: Loss curves + accuracy bar chart
# -----
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: Loss curves for each lr
ax_loss = axes[0]
colors_lr = [PALETTE["steel_blue"], PALETTE["emerald"],
             PALETTE["coral"]]

for i, lr in enumerate(learning_rates):
    ax_loss.plot(results_with_bn[lr]["losses"],
                 color=colors_lr[i], linewidth=2,
                 label=f"BN, lr={lr}", linestyle="--")
    ax_loss.plot(results_no_bn[lr]["losses"], color=colors_lr[i],
                 linewidth=2,
                 label=f"No BN, lr={lr}", linestyle="--",
                 alpha=0.6)

ax_loss.set_xlabel("Epoch")
ax_loss.set_ylabel("Loss")
ax_loss.set_title("Loss Curves Across Learning Rates")
ax_loss.legend(fontsize=8, ncol=2)
ax_loss.set_ylim(0, 3.0)

# Right: Accuracy bar chart
ax_bar = axes[1]
x_pos = np.arange(len(learning_rates))
width = 0.35

```

```

accs_no_bn = [results_no_bn[lr]["accuracy"] for lr in
               learning_rates]
accs_with_bn = [results_with_bn[lr]["accuracy"] for lr in
                learning_rates]

bars1 = ax_bar.bar(x_pos - width/2, accs_no_bn, width,
                  color=PALETTE["coral"],
                  label="Without BN", alpha=0.85,
                  edgecolor="white")
bars2 = ax_bar.bar(x_pos + width/2, accs_with_bn, width,
                  color=PALETTE["royal_blue"],
                  label="With BN", alpha=0.85, edgecolor="white")

ax_bar.set_xlabel("Learning Rate")
ax_bar.set_ylabel("Test Accuracy")
ax_bar.set_title("Final Accuracy vs Learning Rate")
ax_bar.set_xticks(x_pos)
ax_bar.set_xticklabels([str(lr) for lr in learning_rates])
ax_bar.legend()
ax_bar.set_ylim(0, 1.05)

# Add value labels on bars
for bar in bars1:
    h = bar.get_height()
    ax_bar.text(bar.get_x() + bar.get_width()/2, h + 0.02,
                f"{h:.2f}",
                ha="center", fontsize=9, color=PALETTE["charcoal"])
for bar in bars2:
    h = bar.get_height()
    ax_bar.text(bar.get_x() + bar.get_width()/2, h + 0.02,
                f"{h:.2f}",
                ha="center", fontsize=9, color=PALETTE["charcoal"])

plt.tight_layout()
plt.savefig("../figures/ch09_lr_comparison.png", dpi=200,
            facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

# Print table
print("\nFinal Test Accuracy:")
print(f"{'LR':<10} {'Without BN':>12} {'With BN':>12}")
print("-" * 36)
for lr in learning_rates:
    print(f"{'LR':<10} {results_no_bn[lr]['accuracy']:>12.3f}
          {results_with_bn[lr]['accuracy']:>12.3f}")

```

9.7 Gradient Flow with BatchNorm

Another way to understand BatchNorm's benefit is to examine **gradient flow** – how gradient magnitudes change as they propagate backward through layers.

Without normalization, gradients often decay exponentially (vanishing gradients) in deep networks. BatchNorm helps maintain more **uniform gradient norms** across layers.

```
# -----  
# Measure gradient norms at each layer in a 10-layer network  
# -----  
np.random.seed(42)  
  
X_grad, y_grad = make_spirals(n=256, n_classes=3, noise=0.2,  
                               seed=42)  
  
def build_deep_no_bn(seed=42):  
    np.random.seed(seed)  
    layers = []  
    for i in range(10):  
        d_in = 2 if i == 0 else 64  
        layers.append(Linear(d_in, 64))  
        layers.append(ReLU())  
    layers.append(Linear(64, 3))  
    return Sequential(*layers)  
  
def build_deep_with_bn(seed=42):  
    np.random.seed(seed)  
    layers = []  
    for i in range(10):  
        d_in = 2 if i == 0 else 64  
        layers.append(Linear(d_in, 64))  
        layers.append(BatchNorm(64))  
        layers.append(ReLU())  
    layers.append(Linear(64, 3))  
    return Sequential(*layers)  
  
def get_gradient_norms(model, X, y):  
    """Do a forward + backward pass and collect gradient norms  
    for each Linear layer."""  
    loss_fn = CrossEntropyLoss()  
    model.train()  
    logits = model.forward(X)  
    loss = loss_fn.forward(logits, y)  
    grad = loss_fn.backward()  
    model.backward(grad)  
  
    norms = []  
    for layer in model.layers:  
        if isinstance(layer, Linear) and layer.grad_W is not None:  
            norms.append(np.linalg.norm(layer.grad_W))  
    return norms  
  
model_no_bn_g = build_deep_no_bn(seed=77)  
model_with_bn_g = build_deep_with_bn(seed=77)
```

```

norms_no_bn = get_gradient_norms(model_no_bn_g, X_grad, y_grad)
norms_with_bn = get_gradient_norms(model_with_bn_g, X_grad,
                                   y_grad)

# -----
# Side-by-side bar charts
# -----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5),
                               sharey=False)

layer_labels = [f"L{i+1}" for i in range(len(norms_no_bn))]
x_pos = np.arange(len(norms_no_bn))

# Left: Without BN
ax1.bar(x_pos, norms_no_bn, color=PALETTE["coral"], alpha=0.85,
        edgcolor="white", linewidth=0.8)
ax1.set_xticks(x_pos)
ax1.set_xticklabels(layer_labels)
ax1.set_xlabel("Layer")
ax1.set_ylabel("Gradient Norm ( $||dL/dW||$ )")
ax1.set_title("Without BatchNorm", color=PALETTE["coral"],
              fontweight="bold")
ax1.set_yscale("log")

# Right: With BN
ax2.bar(x_pos, norms_with_bn, color=PALETTE["royal_blue"],
        alpha=0.85, edgcolor="white", linewidth=0.8)
ax2.set_xticks(x_pos)
ax2.set_xticklabels(layer_labels)
ax2.set_xlabel("Layer")
ax2.set_ylabel("Gradient Norm ( $||dL/dW||$ )")
ax2.set_title("With BatchNorm", color=PALETTE["royal_blue"],
              fontweight="bold")
ax2.set_yscale("log")

fig.suptitle("Gradient Flow: BatchNorm Prevents Vanishing
             Gradients",
             fontsize=13, fontweight="bold",
             color=PALETTE["deep_blue"], y=1.02)

plt.tight_layout()
plt.savefig("../figures/ch09_gradient_flow.png", dpi=200,
            facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

# Print gradient norms
print("\nGradient Norms per Layer:")
print(f"{'Layer':<8} {'Without BN':>14} {'With BN':>14}
      {'Ratio':>10}")
print("-" * 48)
for i in range(len(norms_no_bn)):
    ratio = norms_with_bn[i] / max(norms_no_bn[i], 1e-15)
    print(f"L{i+1:<6} {norms_no_bn[i]:>14.6f}
          {norms_with_bn[i]:>14.6f} {ratio:>10.2f}x")

```

9.8 Layer Normalization

Batch Normalization has a fundamental limitation: it depends on the **batch dimension**. This creates problems when:

- **Batch size = 1** (e.g., online learning, inference): batch statistics are meaningless
- **Variable-length sequences** (RNNs): different time steps may need different statistics
- **Transformers**: the dominant architecture today uses Layer Normalization exclusively

Layer Normalization

Instead of normalizing across the **batch** (axis=0), Layer Normalization normalizes across the **features** (axis=-1) for each sample independently:

$$\mu_i = \frac{1}{D} \sum_{j=1}^D x_{ij} \quad \sigma_i^2 = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \mu_i)^2$$

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \quad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j$$

Property	BatchNorm	LayerNorm
Normalizes across	Batch (axis=0)	Features (axis=-1)
Depends on batch size	Yes	No
Running statistics	Yes (for inference)	No
Works with batch_size=1	No	Yes
Preferred in	CNNs, MLPs	Transformers, RNNs

```
class LayerNormScratch:
```

```
    """
```

```
    Layer Normalization implemented from scratch.
```

```
    Normalizes across the feature dimension (axis=-1) for each
    sample independently.
```

```
    """
```

```
    def __init__(self, n_features, eps=1e-5):
```

```
        self.gamma = np.ones((1, n_features))
```

```
        self.beta = np.zeros((1, n_features))
```

```
        self.eps = eps
```

```
        self.grad_gamma = None
```

```
        self.grad_beta = None
```

```
        self._cache = None
```

```
    def forward(self, x):
```

```
        # Normalize across features (each sample independently)
```

```
        mean = np.mean(x, axis=-1, keepdims=True) # (N, 1)
```

```
        var = np.var(x, axis=-1, keepdims=True) # (N, 1)
```

```
        x_norm = (x - mean) / np.sqrt(var + self.eps)
```

```
        self._cache = (x, x_norm, mean, var)
```



```

        return self.gamma * x_norm + self.beta

    def backward(self, grad_output):
        x, x_norm, mean, var = self._cache
        D = x.shape[-1]

        self.grad_gamma = np.sum(grad_output * x_norm, axis=0,
                                   keepdims=True)
        self.grad_beta = np.sum(grad_output, axis=0,
                                   keepdims=True)

        dx_norm = grad_output * self.gamma
        dvar = np.sum(dx_norm * (x - mean) * -0.5 * (var +
            self.eps) ** (-1.5),
            axis=-1, keepdims=True)
        dmean = np.sum(dx_norm * -1.0 / np.sqrt(var + self.eps),
            axis=-1, keepdims=True) + \
            dvar * np.mean(-2.0 * (x - mean), axis=-1,
            keepdims=True)

        dx = dx_norm / np.sqrt(var + self.eps) + dvar * 2.0 * (x
            - mean) / D + dmean / D
        return dx

    def params(self):
        return [(self.gamma, self.grad_gamma), (self.beta,
            self.grad_beta)]

# Quick test
ln_test = LayerNormScratch(n_features=4)
x_test_ln = np.random.randn(8, 4)
y_test_ln = ln_test.forward(x_test_ln)

print("LayerNorm test:")
print(f"Input shape: {x_test_ln.shape}")
print(f"Output shape: {y_test_ln.shape}")
print(f"Per-sample mean: {y_test_ln.mean(axis=-1).round(6)}")
print(f"Per-sample std: {y_test_ln.std(axis=-1).round(4)}")

# -----
# Compare BatchNorm vs LayerNorm on small batch sizes
# -----
np.random.seed(42)

batch_sizes = [2, 4, 8, 16, 32, 64]
n_features_test = 32
n_trials = 50

bn_output_stds = []
ln_output_stds = []

for bs in batch_sizes:
    bn_stds = []

```

```

ln_stds = []

for _ in range(n_trials):
    x = np.random.randn(bs, n_features_test) * 3 + 2 # non-
    standard input

    # BatchNorm
    bn = BatchNormScratch(n_features_test)
    out_bn = bn.forward(x, training=True)
    bn_stds.append(out_bn.std())

    # LayerNorm
    ln = LayerNormScratch(n_features_test)
    out_ln = ln.forward(x)
    ln_stds.append(out_ln.std())

bn_output_stds.append(np.mean(bn_stds))
ln_output_stds.append(np.mean(ln_stds))

# -----
# Plot
# -----
fig, ax = plt.subplots(figsize=(9, 5))

ax.plot(batch_sizes, bn_output_stds, 'o-', color=PALETTE["coral"],
        linewidth=2.2, markersize=8, label="BatchNorm")
ax.plot(batch_sizes, ln_output_stds, 's-',
        color=PALETTE["royal_blue"],
        linewidth=2.2, markersize=8, label="LayerNorm")

ax.axhline(y=1.0, color=PALETTE["warm_gray"], linestyle="--",
           alpha=0.7, label="Ideal std=1")

ax.set_xlabel("Batch Size")
ax.set_ylabel("Output Std (averaged over trials)")
ax.set_title("Normalization Stability vs Batch Size")
ax.legend()
ax.set_xscale("log", base=2)
ax.set_xticks(batch_sizes)
ax.set_xticklabels([str(bs) for bs in batch_sizes])

fig.text(0.5, -0.02,
        "LayerNorm is stable across all batch sizes. BatchNorm\n"
        "statistics become noisy with very small batches.",
        ha="center", fontsize=10, color=PALETTE["slate"],
        style="italic")

plt.tight_layout()
plt.savefig("../figures/ch09_bn_vs_ln_batchsize.png", dpi=200,
           facecolor=PALETTE["paper"], bbox_inches="tight")
plt.show()

```

9.9 Exercises

Exercise 1: Group Normalization

Group Normalization (Wu & He, 2018) divides the feature channels into *groups* and normalizes within each group. It is a middle ground between BatchNorm (normalize across batch) and LayerNorm (normalize across all features).

Implement `GroupNorm(n_features, n_groups, eps=1e-5)` from scratch. For an input of shape (N, D) , reshape to $(N, G, D//G)$, compute mean and variance over the last axis, normalize, and reshape back. Add learnable gamma and beta. Test it with `n_groups=1` (should match LayerNorm) and `n_groups=D` (should match InstanceNorm, i.e., each feature normalized independently).

Exercise 2: Batch Size = 1 Experiment

Train the spirals dataset with `batch_size=1` (stochastic gradient descent) using: - (a) BatchNorm - (b) LayerNorm - (c) No normalization

Compare the training loss curves. What happens to BatchNorm when the batch has only one sample? (Hint: the variance is zero.) Does LayerNorm handle this case better? Why?

Exercise 3: BatchNorm in Evaluation Mode

Train a network with BatchNorm on the spirals dataset. After training, compare predictions made with: - (a) `model.train()` mode (using batch statistics) - (b) `model.eval()` mode (using running statistics)

Feed single samples (`batch_size=1`) to the network. Show that train mode gives different (and unreliable) results for single samples, while eval mode is deterministic and consistent. Visualize the decision boundaries in both modes.

Next: [Chapter 10 – Convolutional Networks](#)

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
from utils.nn import Linear, ReLU, Sequential, Conv2D, MaxPool2D,
    Flatten, CrossEntropyLoss
from utils.data import make_simple_images, train_test_split
from utils.plot import plot_loss_curve, plot_confusion_matrix
apply_style()
np.random.seed(42)
```

Chapter 10: Convolutional Neural Networks

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain **why** fully-connected networks are impractical for image data and how convolutions solve this
2. Implement the 1D and 2D **convolution operation** from scratch with NumPy
3. Understand padding, stride, and pooling and their effect on output dimensions
4. Describe how the **im2col trick** turns convolution into a matrix multiply for efficient forward and backward passes
5. Build a complete **ConvNet** (Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ FC) and train it on a shape-classification task
6. Visualize **feature maps** at each layer and interpret what the network has learned
7. Inspect the **learned filters** and relate them to classical edge detectors
8. Calculate the **receptive field** of neurons at different depths

10.1 Motivation

The parameter explosion

Consider a modest (64×64) grayscale image fed into a fully-connected layer with 256 hidden neurons:

$$[\text{Parameters}] = 64 \times 64 \times 256 = 1,048,576 \approx 1 \text{M}]$$

For a (224×224) RGB image the count balloons to $(224 \times 224 \times 3 \times 256 \approx 38 \text{M})$ – just for one layer! This is wasteful because:

1. **Most pixel-to-pixel connections carry no useful information.** A pixel in the top-left corner rarely has a direct relationship with a pixel in the bottom-right corner.
2. **Images have local structure.** Edges, textures, and small shapes are defined by nearby pixels.

Translation invariance

A cat in the top-left of an image is the same cat if it appears in the bottom-right. A fully-connected layer must re-learn the cat pattern for every possible position. Convolutions share weights across spatial positions, so a pattern learned at one location is automatically recognized everywhere.

Spatial hierarchy

Visual information is naturally hierarchical:

- **Low-level:** edges, corners, colour gradients
- **Mid-level:** textures, small parts (eyes, wheels)
- **High-level:** whole objects (faces, cars)

A stack of convolutional layers captures exactly this progression: early layers detect simple local patterns, deeper layers compose them into increasingly abstract representations.

Convolutional Neural Networks (CNNs) address all three issues through three key ideas:

Idea	Benefit
Local receptive fields	Each neuron sees only a small patch $\rightarrow$ far fewer parameters
Weight sharing	Same filter applied at every position $\rightarrow$ translation invariance
Pooling	Progressively reduces spatial dimensions $\rightarrow$ builds spatial hierarchy

10.2 The Convolution Operation

Convolution is the mathematical backbone of CNNs. We start with the simpler 1D case before moving to images.

1D Convolution

Given a signal $\mathbf{x}$ of length N and a kernel $\mathbf{k}$ of length K , the 1D (valid) convolution produces an output of length $N - K + 1$:

$$[(\mathbf{x} * \mathbf{k})][i] = \sum_{m=0}^{K-1} \mathbf{x}[i + m] \cdot \mathbf{k}[m]$$

The kernel **slides** along the signal, computing a dot product at each position.

```
# --- 1D convolution example ---
def conv1d(signal, kernel):
    """Naive 1D valid convolution."""
    N, K = len(signal), len(kernel)
    out_len = N - K + 1
    output = np.zeros(out_len)
    for i in range(out_len):
        output[i] = np.sum(signal[i:i+K] * kernel)
    return output

# Create a simple signal and an edge-detection kernel
t = np.linspace(0, 2 * np.pi, 50)
signal = np.sin(t) + 0.3 * np.sin(5 * t)
```

```

kernel_smooth = np.array([0.25, 0.5, 0.25])      # smoothing
kernel_edge    = np.array([-1.0, 0.0, 1.0])      # edge
              detection

out_smooth = conv1d(signal, kernel_smooth)
out_edge    = conv1d(signal, kernel_edge)

fig, axes = plt.subplots(3, 1, figsize=(10, 7), sharex=True)

axes[0].plot(signal, color=PALETTE["royal_blue"], linewidth=2)
axes[0].set_title("Original Signal")
axes[0].set_ylabel("Amplitude")

axes[1].plot(out_smooth, color=PALETTE["emerald"], linewidth=2)
axes[1].set_title("After Smoothing Kernel [0.25, 0.5, 0.25]")
axes[1].set_ylabel("Amplitude")

axes[2].plot(out_edge, color=PALETTE["coral"], linewidth=2)
axes[2].set_title("After Edge Kernel [-1, 0, 1]")
axes[2].set_ylabel("Amplitude")
axes[2].set_xlabel("Position")

plt.tight_layout()
plt.savefig("../figures/10_conv1d_example.png")
plt.show()

print(f"Signal length: {len(signal)}, Kernel length:
      {len(kernel_smooth)}")
print(f"Output length: {len(out_smooth)} (= {len(signal)} -
      {len(kernel_smooth)} + 1)")

```

2D Convolution

For images, the kernel is a small 2D patch (e.g., 3×3) that slides over both the height and width dimensions:

$$[(\mathbf{X} * \mathbf{K})][i, j] = \sum_{m=0}^{k_h - 1} \sum_{n=0}^{k_w - 1} \mathbf{X}[i+m, j+n] \cdot \mathbf{K}[m, n]$$

Output size (valid, stride 1): $(H - k_h + 1) \times (W - k_w + 1)$.

```

def conv2d_naive(image, kernel, stride=1, padding=0):
    """Naive 2D convolution on a single-channel image.

```

Parameters

```

-----
image : ndarray of shape (H, W)
kernel : ndarray of shape (kH, kW)
stride : int
padding : int (zero-padding added to each side)

```

Returns

```

-----

```

```

output : ndarray
"""
if padding > 0:
    image = np.pad(image, padding, mode='constant',
                    constant_values=0)

H, W = image.shape
kH, kW = kernel.shape
out_h = (H - kH) // stride + 1
out_w = (W - kW) // stride + 1

output = np.zeros((out_h, out_w))
for i in range(out_h):
    for j in range(out_w):
        patch = image[i*stride:i*stride+kH,
                      j*stride:j*stride+kW]
        output[i, j] = np.sum(patch * kernel)
return output

# Quick test
test_img = np.random.rand(5, 5)
test_kern = np.array([[1, 0, -1],
                      [1, 0, -1],
                      [1, 0, -1]], dtype=float)
result = conv2d_naive(test_img, test_kern)
print(f"Input shape: {test_img.shape}")
print(f"Kernel shape: {test_kern.shape}")
print(f"Output shape: {result.shape} (expected (3, 3))")

# --- Step-by-step visualization: kernel sliding over a 5x5 input
---
np.random.seed(7)
input_5x5 = np.round(np.random.rand(5, 5), 1)
kernel_3x3 = np.array([[1, 0, -1],
                      [1, 0, -1],
                      [1, 0, -1]], dtype=float)
output_3x3 = conv2d_naive(input_5x5, kernel_3x3)

# Show 4 selected positions in a 2x2 grid
positions = [(0, 0), (0, 2), (2, 0), (2, 2)]
colors_patch = [PALETTE["coral"], PALETTE["emerald"],
                PALETTE["amber"], PALETTE["steel_blue"]]

fig, axes = plt.subplots(2, 2, figsize=(10, 10))

for idx, (pi, pj) in enumerate(positions):
    ax = axes[idx // 2][idx % 2]

    # Draw the full input grid
    for r in range(5):
        for c in range(5):
            # Highlight the patch covered by the kernel
            in_patch = (pi <= r < pi + 3) and (pj <= c < pj + 3)

```

```

        facecolor = colors_patch[idx] if in_patch else
PALETTE["light_gray"]
        alpha = 0.35 if in_patch else 0.15
        rect = plt.Rectangle((c, 4 - r), 1, 1, linewidth=1,
                             edgecolor=PALETTE["charcoal"],
                             facecolor=facecolor,
                             alpha=alpha)
        ax.add_patch(rect)
        ax.text(c + 0.5, 4 - r + 0.5, f"{input_5x5[r,
c]:.1f}",
                ha='center', va='center', fontsize=9,
                color=PALETTE["charcoal"], fontweight='bold'
                if in_patch else 'normal')

# Show kernel values in small text above the patch
for kr in range(3):
    for kc in range(3):
        r, c = pi + kr, pj + kc
        kval = kernel_3x3[kr, kc]
        ax.text(c + 0.5, 4 - r + 0.15, f"k={kval:+.0f}",
                ha='center', va='center', fontsize=6,
                color=PALETTE["deep_blue"], style='italic')

out_val = output_3x3[pi, pj]
ax.set_title(f"Position ({pi},{pj}) | output =
{out_val:.2f}",
             fontsize=11, color=colors_patch[idx])
ax.set_xlim(-0.1, 5.1)
ax.set_ylim(-0.1, 5.1)
ax.set_aspect('equal')
ax.axis('off')

fig.suptitle("2D Convolution: Kernel Sliding Over a 5x5 Input",
             fontsize=14, fontweight='bold',
             color=PALETTE["deep_blue"], y=1.01)
plt.tight_layout()
plt.savefig("../figures/10_conv2d_step_by_step.png")
plt.show()

```

10.3 Building Blocks

Padding

Without padding, the output shrinks at each layer. Two common strategies:

Mode	Padding $\backslash(p)$	Output size	Use case
Valid	$\backslash(0\backslash)$	$\backslash((H - k_h + 1) \times (W - k_w + 1)\backslash)$	No padding; output shrinks
Same	$\backslash(\lfloor k / 2 \rfloor\backslash)$	$\backslash(H \times W\backslash)$ (with stride 1)	Preserves spatial dimensions

Stride

Stride $(s > 1)$ makes the kernel skip positions, reducing the output size:

$$\lfloor \text{Output size} \rfloor = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

Multiple Channels

For an input with (C_{in}) channels, each filter has shape $((C_{\text{in}}, k_h, k_w))$. The convolution sums over all input channels:

$$\text{output}[i, j] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{m,n} \mathbf{X}[c, i+m, j+n] \cdot \mathbf{K}[c, m, n] + b$$

With (C_{out}) filters, the output has (C_{out}) channels. Each filter detects a different pattern.

```
# --- Demonstrate output sizes for various padding and stride ---
```

```
H, W = 8, 8
```

```
kH, kW = 3, 3
```

```
configs = [
    {"name": "Valid (p=0, s=1)", "padding": 0, "stride": 1},
    {"name": "Same (p=1, s=1)", "padding": 1, "stride": 1},
    {"name": "Stride 2 (p=0, s=2)", "padding": 0, "stride": 2},
    {"name": "Stride 2 (p=1, s=2)", "padding": 1, "stride": 2},
]
```

```
print(f"Input: {H}x{W} | Kernel: {kH}x{kW}")
```

```
print("-" * 45)
```

```
for cfg in configs:
```

```
    p, s = cfg["padding"], cfg["stride"]
```

```
    out_h = (H + 2 * p - kH) // s + 1
```

```
    out_w = (W + 2 * p - kW) // s + 1
```

```
    print(f"{cfg['name']:25s} --> output: {out_h}x{out_w}")
```

```
# --- Max pooling and average pooling from scratch ---
```

```
def max_pool2d(x, kernel_size=2, stride=2):
```

```
    """Max pooling on a single-channel 2D input.
```

```
    Parameters
```

```
    -----
```

```
    x : ndarray of shape (H, W)
```

```
    kernel_size : int
```

```
    stride : int
```

```
    Returns
```

```
    -----
```

```
    output : ndarray
```

```
    """
```

```
    H, W = x.shape
```

```
    out_h = (H - kernel_size) // stride + 1
```

```

out_w = (W - kernel_size) // stride + 1
output = np.zeros((out_h, out_w))
for i in range(out_h):
    for j in range(out_w):
        patch = x[i*stride:i*stride+kernel_size,
                  j*stride:j*stride+kernel_size]
        output[i, j] = np.max(patch)
return output

def avg_pool2d(x, kernel_size=2, stride=2):
    """Average pooling on a single-channel 2D input."""
    H, W = x.shape
    out_h = (H - kernel_size) // stride + 1
    out_w = (W - kernel_size) // stride + 1
    output = np.zeros((out_h, out_w))
    for i in range(out_h):
        for j in range(out_w):
            patch = x[i*stride:i*stride+kernel_size,
                      j*stride:j*stride+kernel_size]
            output[i, j] = np.mean(patch)
    return output

# Demo on a small input
demo_input = np.array([[1, 3, 2, 4],
                        [5, 6, 1, 2],
                        [3, 2, 7, 8],
                        [4, 1, 5, 6]], dtype=float)

print("Input (4x4):")
print(demo_input)
print(f"\nMax pool (2x2, stride 2):")
print(max_pool2d(demo_input))
print(f"\nAverage pool (2x2, stride 2):")
print(avg_pool2d(demo_input))

```

10.4 The Conv2D Layer with Backprop

The im2col Trick

A naive nested-loop convolution is easy to understand but painfully slow. The **im2col** (image-to-column) trick rearranges the input patches into columns of a matrix, turning convolution into a single matrix multiplication:

1. **Extract** every $((k_h \times k_w \times C_{\text{in}}))$ -sized patch from the input and flatten it into a row.
2. **Stack** all rows to form the *column matrix* of shape $((N \times \text{out}_h \times \text{out}_w, C_{\text{in}} \times k_h \times k_w))$.
3. **Reshape** each filter into a column of a *weight matrix* of shape $((C_{\text{in}} \times k_h \times k_w, C_{\text{out}}))$.
4. **Multiply**: $(\text{output} = \text{cols} \times W^T + b)$.

5. **Reshape** the result back to $((N, C_{\text{out}}, \text{out}_h, \text{out}_w))$.

This is exactly the approach used in our Conv2D class in `utils/nn.py`.

Forward Pass

```
Input (N, C_in, H, W)  --im2col-->  Cols (N*oH*oW, C_in*kH*kW)
                                @ W_flat.T + b
                                --> reshape --> (N,
C_out, oH, oW)
```

Backward Pass

The gradients follow naturally from the matrix-multiply view:

- **Gradient w.r.t. kernels:** $(\nabla_W = \text{grad_output_reshaped}^T \cdot \text{cols})$, then reshape. This is equivalent to the *cross-correlation* of the input with the output gradient.
- **Gradient w.r.t. input:** $(\nabla_{\text{cols}} = \text{grad_output_reshaped} \cdot W_{\text{flat}})$, then **col2im** to scatter back into the input shape. This is equivalent to a *full convolution* of the output gradient with the flipped kernel.
- **Gradient w.r.t. bias:** Sum over the spatial and batch dimensions.

Our implementation in `utils/nn.py` (`Conv2D.backward`) follows this pattern exactly.

```
# --- Demonstrate the Conv2D layer from utils/nn.py ---
np.random.seed(42)

# Single 8x8 grayscale image, batch of 1
x_demo = np.random.rand(1, 1, 8, 8)

conv_layer = Conv2D(in_channels=1, out_channels=4, kernel_size=3,
                    padding=1)
out_demo = conv_layer.forward(x_demo)

print(f"Input shape: {x_demo.shape} (N, C_in, H, W)")
print(f"Kernel shape: {conv_layer.W.shape} (C_out, C_in, kH, kW)")
print(f"Output shape: {out_demo.shape} (N, C_out, oH, oW)")
print(f"\nWith padding=1 and stride=1, spatial dims preserved: 8x8
--> 8x8")
```

10.5 Building a Simple ConvNet

We now assemble a complete CNN and train it to classify simple geometric shapes (circles, squares, triangles) generated as (8×8) images.

Architecture:

```
Conv2D(1, 8, 3, pad=1) --> ReLU --> MaxPool2D(2)
--> Conv2D(8, 16, 3, pad=1) --> ReLU --> MaxPool2D(2)
--> Flatten --> Linear(16*2*2, 3)
```

- Input: $((N, 1, 8, 8))$ – single-channel (8×8) images
- After first conv + pool: $((N, 8, 4, 4))$
- After second conv + pool: $((N, 16, 2, 2))$
- After flatten: $((N, 64))$
- Output: $((N, 3))$ – logits for 3 classes

```
# --- Generate shape classification data ---
X_all, y_all = make_simple_images(n=300, size=8,
                                  shapes=("circle", "square", "triangle"), seed=42)
X_train, X_test, y_train, y_test = train_test_split(X_all, y_all,
                                                    test_ratio=0.2, seed=42)

print(f"Training set: {X_train.shape[0]} images")
print(f"Test set:     {X_test.shape[0]} images")
print(f"Image shape:   {X_train.shape[1:]} (C, H, W)")
print(f"Classes:      circle=0, square=1, triangle=2")

# Visualize a few samples
fig, axes = plt.subplots(1, 6, figsize=(12, 2.5))
class_names = ["circle", "square", "triangle"]
for i in range(6):
    axes[i].imshow(X_train[i * 30, 0], cmap='gray', vmin=0,
                  vmax=1)
    axes[i].set_title(class_names[y_train[i * 30]], fontsize=10)
    axes[i].axis('off')

fig.suptitle("Sample Training Images", fontsize=13,
            fontweight='bold', color=PALETTE["deep_blue"])
plt.tight_layout()
plt.savefig("../figures/10_sample_shapes.png")
plt.show()

# --- Build the ConvNet ---
np.random.seed(42)

model = Sequential(
    Conv2D(1, 8, 3, padding=1),      # (N,1,8,8) -> (N,8,8,8)
    ReLU(),
    MaxPool2D(2),                    # (N,8,8,8) -> (N,8,4,4)
    Conv2D(8, 16, 3, padding=1),    # (N,8,4,4) -> (N,16,4,4)
    ReLU(),
    MaxPool2D(2),                    # (N,16,4,4) -> (N,16,2,2)
    Flatten(),                      # (N,16,2,2) -> (N,64)
    Linear(16 * 2 * 2, 3),           # (N,64) -> (N,3)
)

loss_fn = CrossEntropyLoss()
lr = 0.01
epochs = 60
batch_size = 32
```

```

# --- Training loop ---
train_losses = []
train_accs = []
test_accs = []

n_train = X_train.shape[0]

for epoch in range(epochs):
    # Shuffle training data
    perm = np.random.permutation(n_train)
    X_shuf = X_train[perm]
    y_shuf = y_train[perm]

    epoch_loss = 0.0
    n_batches = 0

    for start in range(0, n_train, batch_size):
        end = min(start + batch_size, n_train)
        X_batch = X_shuf[start:end]
        y_batch = y_shuf[start:end]

        # Forward
        logits = model.forward(X_batch)
        loss = loss_fn.forward(logits, y_batch)

        # Backward
        grad = loss_fn.backward()
        model.backward(grad)

        # SGD update
        for param, grad_p in model.params():
            if grad_p is not None:
                param -= lr * grad_p

        epoch_loss += loss
        n_batches += 1

    avg_loss = epoch_loss / n_batches
    train_losses.append(avg_loss)

    # Compute accuracies
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    train_acc = np.mean(train_pred == y_train)
    test_acc = np.mean(test_pred == y_test)
    train_accs.append(train_acc)
    test_accs.append(test_acc)

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1:3d}/{epochs} | Loss: {avg_loss:.4f} | ")

```

```

        f"Train Acc: {train_acc:.1%} | Test Acc:
        {test_acc:.1%}")

print(f"\nFinal Test Accuracy: {test_accs[-1]:.1%}")

# --- Plot training loss and accuracy ---
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Loss curve
axes[0].plot(train_losses, color=PALETTE["royal_blue"],
             linewidth=2)
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Cross-Entropy Loss")
axes[0].set_title("Training Loss")

# Accuracy curves
axes[1].plot(train_accs, color=PALETTE["royal_blue"],
             linewidth=2, label="Train")
axes[1].plot(test_accs, color=PALETTE["coral"], linewidth=2,
             label="Test")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].set_title("Classification Accuracy")
axes[1].legend()
axes[1].set_ylim(0, 1.05)

fig.suptitle("ConvNet Training on Shape Classification",
             fontsize=14,
             fontweight='bold', color=PALETTE["deep_blue"],
             y=1.01)
plt.tight_layout()
plt.savefig("../figures/10_training_curves.png")
plt.show()

# --- Confusion matrix on the test set ---
y_pred_test = model.predict(X_test)

fig, ax = plt.subplots(figsize=(5, 5))
plot_confusion_matrix(y_test, y_pred_test,
                      class_names=["circle", "square",
                                   "triangle"], ax=ax)
plt.tight_layout()
plt.savefig("../figures/10_confusion_matrix.png")
plt.show()

```

10.6 Feature Map Visualization (Hero Figure)

One of the most illuminating ways to understand a CNN is to pass an image through each layer and visualize the intermediate **feature maps**. At each depth, the network “sees” the image differently:

- **After Conv1:** low-level features (edges, gradients)
- **After Pool1:** spatially compressed, but same features

- **After Conv2:** combinations of lower-level features (corners, shapes)

Below, we trace a single image through the network and display the resulting feature maps at each stage.

```
# --- Hero Figure: Feature Map Cascade ---

# Pick a sample image (a triangle)
sample_idx = 0
for i in range(len(X_test)):
    if y_test[i] == 2: # triangle
        sample_idx = i
        break

sample_img = X_test[sample_idx:sample_idx+1] # (1, 1, 8, 8)

# Forward pass through each layer, capturing intermediates
model.eval()
activations = {}
x = sample_img
layer_names = ["Conv1", "ReLU1", "Pool1", "Conv2", "ReLU2",
               "Pool2", "Flatten", "Linear"]
for name, layer in zip(layer_names, model.layers):
    x = layer.forward(x)
    activations[name] = x.copy()
model.train()

# Layout: Input | Conv1 (8 maps, 2x4) | Pool1 (8 maps, 2x4) |
#          Conv2 (8 of 16 maps, 2x4)
fig = plt.figure(figsize=(16, 10))

# Overall title
fig.suptitle("Feature Map Cascade Through the ConvNet",
             fontsize=16, fontweight='bold',
             color=PALETTE["deep_blue"], y=0.98)

# --- Input image ---
ax_input = fig.add_axes([0.02, 0.4, 0.1, 0.25])
ax_input.imshow(sample_img[0, 0], cmap='gray', vmin=0, vmax=1)
ax_input.set_title("Input\n(1x8x8)", fontsize=10,
                  color=PALETTE["deep_blue"])
ax_input.axis('off')

# Arrow
fig.text(0.13, 0.53, r'$\rightarrow$', fontsize=20,
        ha='center',
        va='center', color=PALETTE["warm_gray"])

# --- Conv1 feature maps (8 maps in 2x4 grid) ---
conv1_out = activations["ReLU1"][0] # (8, 8, 8)
for i in range(8):
    row, col = i // 4, i % 4
    ax = fig.add_axes([0.16 + col * 0.055, 0.55 - row * 0.28,
                       0.05, 0.22])
```

```

ax.imshow(conv1_out[i], cmap='RdBu_r', vmin=-conv1_out.max(),
          vmax=conv1_out.max())
ax.axis('off')
if row == 0 and col == 0:
    ax.set_title("Conv1+ReLU\n(8x8x8)", fontsize=9,
                color=PALETTE["royal_blue"], loc='left')

# Arrow
fig.text(0.40, 0.53, r'$\rightarrow$', fontsize=20,
        ha='center',
        va='center', color=PALETTE["warm_gray"])

# --- Pool1 feature maps ---
pool1_out = activations["Pool1"][0] # (8, 4, 4)
for i in range(8):
    row, col = i // 4, i % 4
    ax = fig.add_axes([0.43 + col * 0.055, 0.55 - row * 0.28,
                      0.05, 0.22])
    ax.imshow(pool1_out[i], cmap='RdBu_r', vmin=-pool1_out.max(),
              vmax=pool1_out.max())
    ax.axis('off')
    if row == 0 and col == 0:
        ax.set_title("Pool1\n(8x4x4)", fontsize=9,
                    color=PALETTE["emerald"], loc='left')

# Arrow
fig.text(0.67, 0.53, r'$\rightarrow$', fontsize=20,
        ha='center',
        va='center', color=PALETTE["warm_gray"])

# --- Conv2 feature maps (show 8 of 16) ---
conv2_out = activations["ReLU2"][0] # (16, 4, 4)
for i in range(8):
    row, col = i // 4, i % 4
    ax = fig.add_axes([0.70 + col * 0.055, 0.55 - row * 0.28,
                      0.05, 0.22])
    ax.imshow(conv2_out[i], cmap='RdBu_r', vmin=-conv2_out.max(),
              vmax=conv2_out.max())
    ax.axis('off')
    if row == 0 and col == 0:
        ax.set_title("Conv2+ReLU\n(16x4x4)", fontsize=9,
                    color=PALETTE["coral"], loc='left')

# Annotation at bottom
fig.text(0.5, 0.08,
        "Early layers detect edges and gradients | "  

        "Pooling compresses spatial dimensions | "  

        "Deeper layers capture higher-level shape features",
        ha='center', fontsize=10, color=PALETTE["slate"],
        style='italic')

plt.savefig("../figures/10_feature_map_cascade.png",
            bbox_inches='tight')
plt.show()

```


10.7 Learned Filters

The first convolutional layer's filters are directly interpretable – they operate on the raw pixel values. After training, some filters typically resemble classical **edge detectors** (horizontal, vertical, diagonal), while others may capture blob-like patterns.

Let's visualize the 8 learned (3×3) kernels from the first Conv2D layer.

```
# --- Visualize the learned 3x3 filters of the first conv layer
---
conv1_weights = model.layers[0].W # shape: (8, 1, 3, 3)

fig, axes = plt.subplots(2, 4, figsize=(10, 5.5))
vmax = np.abs(conv1_weights).max()

for i in range(8):
    ax = axes[i // 4][i % 4]
    kernel = conv1_weights[i, 0] # (3, 3)
    im = ax.imshow(kernel, cmap='RdBu_r', vmin=-vmax, vmax=vmax,
                    interpolation='nearest')
    ax.set_title(f"Filter {i}", fontsize=10)
    ax.axis('off')

    # Annotate each cell with its value
    for r in range(3):
        for c in range(3):
            val = kernel[r, c]
            color = 'white' if abs(val) > vmax * 0.5 else
                PALETTE["charcoal"]
            ax.text(c, r, f"{val:.2f}", ha='center', va='center',
                    fontsize=8, color=color, fontweight='bold')

fig.suptitle("Learned Filters -- First Convolutional Layer",
             fontsize=14, fontweight='bold',
             color=PALETTE["deep_blue"], y=1.01)
fig.colorbar(im, ax=axes, shrink=0.6, label='Weight value')
plt.tight_layout()
plt.savefig("../figures/10_learned_filters.png")
plt.show()

print("Some filters resemble edge detectors (alternating +/-
      values),")
print("while others respond to blobs or diagonal patterns.")
```

10.8 Receptive Field

The **receptive field** of a neuron is the region of the original input that can influence its activation. As we stack convolutional and pooling layers, deeper neurons have progressively larger receptive fields, allowing them to “see” more of the input.

Computing the receptive field

For a sequence of layers, each with kernel size (k_i) and stride (s_i) , the receptive field (R_L) after layer (L) is:

$$R_L = R_{L-1} + (k_L - 1) \times \prod_{i=1}^{L-1} s_i$$

starting with $(R_0 = 1)$ (a single pixel).

Our architecture

Layer	Kernel	Stride	Receptive Field
Input	—	—	1
Conv1 (3x3, s=1)	3	1	$(1 + (3-1) \times 1 = 3)$
Pool1 (2x2, s=2)	2	2	$(3 + (2-1) \times 1 = 4)$
Conv2 (3x3, s=1)	3	1	$(4 + (3-1) \times 2 = 8)$
Pool2 (2x2, s=2)	2	2	$(8 + (2-1) \times 2 = 10)$

After two conv+pool blocks, each neuron in the output has a receptive field of **10 pixels** – larger than our (8×8) input! This means every output neuron can potentially see the entire image.

```
# --- Receptive field calculation ---
layers_config = [
    {"name": "Conv1", "k": 3, "s": 1},
    {"name": "Pool1", "k": 2, "s": 2},
    {"name": "Conv2", "k": 3, "s": 1},
    {"name": "Pool2", "k": 2, "s": 2},
]

rf = 1 # receptive field starts at 1 pixel
stride_product = 1
rf_history = [("Input", rf)]

for layer in layers_config:
    rf = rf + (layer["k"] - 1) * stride_product
    stride_product *= layer["s"]
    rf_history.append((layer["name"], rf))

print("Receptive Field Growth")
print("=" * 35)
for name, r in rf_history:
    print(f"  After {name:8s}:  RF = {r} pixels")

# --- Visualization: growing receptive fields ---
fig, axes = plt.subplots(1, 5, figsize=(16, 3.5))

img_size = 8
rf_colors = [PALETTE["light_gray"], PALETTE["steel_blue"],
             PALETTE["emerald"],
             PALETTE["amber"], PALETTE["coral"]]
```

```

for idx, (name, rf_size) in enumerate(rf_history):
    ax = axes[idx]

    # Draw the input grid
    for r in range(img_size):
        for c in range(img_size):
            rect = plt.Rectangle((c, img_size - 1 - r), 1, 1,
                                linewidth=0.5,
                                edgecolor=PALETTE["warm_gray"],
                                facecolor=PALETTE["paper"])
            ax.add_patch(rect)

    # Highlight the receptive field centered roughly in the middle
    center = img_size // 2
    half_rf = min(rf_size, img_size) / 2
    start_r = max(0, center - half_rf)
    start_c = max(0, center - half_rf)
    rf_draw = min(rf_size, img_size)

    rf_rect = plt.Rectangle((start_c, img_size - start_r -
                             rf_draw), rf_draw, rf_draw,
                            linewidth=2.5,
                            edgecolor=rf_colors[idx],
                            facecolor=rf_colors[idx], alpha=0.3)
    ax.add_patch(rf_rect)

    # Border for the RF
    rf_border = plt.Rectangle((start_c, img_size - start_r -
                                rf_draw), rf_draw, rf_draw,
                               linewidth=2.5,
                               edgecolor=rf_colors[idx],
                               facecolor='none')
    ax.add_patch(rf_border)

    ax.set_xlim(-0.2, img_size + 0.2)
    ax.set_ylim(-0.2, img_size + 0.2)
    ax.set_aspect('equal')
    ax.axis('off')
    ax.set_title(f"{name}\nRF = {rf_size}x{rf_size}", fontsize=10,
                 color=rf_colors[idx], fontweight='bold')

fig.suptitle("Receptive Field Growth Through the Network",
             fontsize=14, fontweight='bold',
             color=PALETTE["deep_blue"], y=1.05)
plt.tight_layout()
plt.savefig("../figures/10_receptive_fields.png")
plt.show()

```

10.9 Exercises

Exercise 10.1 – Depthwise Separable Convolution

A standard convolution with C_{in} input channels, C_{out} output channels, and kernel size k has $C_{\text{in}} \times C_{\text{out}} \times k^2$ parameters. A **depthwise separable convolution** decomposes this into:

1. A *depthwise* convolution: one $k \times k$ filter per input channel ($C_{\text{in}} \times k^2$ parameters)
2. A *pointwise* (1×1) convolution to mix channels ($C_{\text{in}} \times C_{\text{out}}$ parameters)

Tasks: 1. Calculate the parameter ratio: standard vs. depthwise separable for $C_{\text{in}} = 32$, $C_{\text{out}} = 64$, $k = 3$. 2. Implement `depthwise_conv2d` and `pointwise_conv2d` from scratch using NumPy. 3. Verify that chaining them produces the same output shape as a standard convolution.

Exercise 10.2 – MNIST Subset

Download a small subset of MNIST (e.g., digits 0-4, 100 images each) and:

1. Resize the images to 8×8 using simple downsampling.
 2. Train the ConvNet architecture from Section 10.5.
 3. Compare the test accuracy to a fully-connected network of similar parameter count.
 4. Visualize the learned first-layer filters – do they look like edge detectors?
-

Exercise 10.3 – Filter Evolution During Training

Modify the training loop from Section 10.5 to save the first conv layer's weights at epochs 1, 5, 10, 20, 40, and 60.

1. Create a grid visualization showing how each of the 8 filters evolves over training.
 2. At which epoch do the filters start to look structured (i.e., develop clear edge-like patterns)?
 3. Does the loss curve inflection point correspond to when filters become structured? Discuss.
-

Chapter Summary

Concept	Key Takeaway
Motivation	FC layers have too many parameters for images; convolutions exploit locality and translation invariance
Convolution	Slide a small kernel over the input, computing dot products – detects local patterns
Padding & Stride	Control output dimensions; same padding preserves size, stride > 1 downsamples
Pooling	MaxPool / AvgPool reduce spatial dimensions and add mild translation invariance

Concept	Key Takeaway
im2col	Rearranges patches into a matrix so convolution becomes a fast matrix multiply
Backprop	grad w.r.t. kernels = correlation of input with output grad; grad w.r.t. input = full conv with flipped kernel
Feature maps	Early layers detect edges; deeper layers capture increasingly abstract patterns
Receptive field	Deeper neurons see larger regions of the input; grows with each conv/pool layer

Next up: In Chapter 11, we explore techniques for **training deeper networks** – batch normalization, residual connections, and learning rate schedules – that make modern deep CNNs possible.

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
from utils.nn import RNNCell, LSTMCell, GRUCell
from utils.data import make_sine_sequence
from utils.plot import plot_loss_curve
apply_style()
np.random.seed(42)
```

Chapter 11: Recurrent Neural Networks

Feedforward networks treat every input as independent – they have no concept of **order** or **memory**. Yet many of the most important problems in deep learning involve **sequences**: time series forecasting, natural language processing, speech recognition, and music generation. In this chapter we build **Recurrent Neural Networks (RNNs)** from scratch, learn why they struggle with long-range dependencies, and develop the gated architectures – **LSTM** and **GRU** – that solve those problems.

Learning Objectives

By the end of this chapter you will be able to:

1. Explain why sequential data requires architectures with **memory** and why feedforward networks fall short.
2. Implement a **vanilla RNN** cell from scratch and unroll it over a sequence.
3. Derive **Backpropagation Through Time (BPTT)** and understand the **vanishing gradient** problem.
4. Train an RNN to **predict a sine wave** and visualize its predictions.
5. Implement **LSTM** and **GRU** cells, understanding how gating mechanisms preserve long-range gradients.

6. Build a **character-level language model** that generates text.
7. Compare RNN vs LSTM on long-range dependency tasks and apply **gradient clipping** for stable training.

11.1 Sequential Data

11.2 The Vanilla RNN

The simplest recurrent architecture updates a hidden state at each time step using the current input and the previous hidden state:

$$\begin{aligned} \mathbf{h}_t = & \tanh(\mathbf{W}_{xh} \mathbf{x}_t + \mathbf{W}_{hh} \mathbf{h}_{t-1} + \\ & \mathbf{b}_h) \end{aligned}$$

$$\begin{aligned} \mathbf{y}_t = & \mathbf{W}_{hy} \mathbf{h}_t + \\ & \mathbf{b}_y \end{aligned}$$

where: - $\mathbf{x}_t \in \mathbb{R}^d$ is the input at time t - $\mathbf{h}_t \in \mathbb{R}^H$ is the hidden state at time t - $\mathbf{W}_{xh} \in \mathbb{R}^{d \times H}$, $\mathbf{W}_{hh} \in \mathbb{R}^{H \times H}$, $\mathbf{W}_{hy} \in \mathbb{R}^{H \times o}$ are weight matrices - The **same weights** are shared across all time steps – this is what makes it “recurrent”

Unrolling the RNN

To understand computation (and later, backpropagation), we “unroll” the RNN through time. Imagine copying the same cell T times side by side:

```

x_1      x_2
x_3      x_T
|        |
|        |
v        v
v        v [RNN] --
> [RNN] --> [RNN] --
> ... [RNN] | h_1
| h_2 | h_3
| h_T v

```

11.1 Sequential Data

v v
 v y_1 y_2
 y_3 y_T

Each box is the **same** RNN cell with shared weights. The horizontal arrows carry the hidden state $\mathbf{h}_t$ from one step to the next. This unrolled view makes it clear that an RNN is really a very deep feedforward network with **tied weights**.

``python # — 11.2 RNN Cell
Implementation from Scratch

```
class SimpleRNNCell:
    """Vanilla RNN cell:  $h_t = \tanh(W_{xh} * x_t + W_{hh} * h_{t-1} + b_h)$ ."""
    def __init__(self, input_size, hidden_size):
        scale = np.sqrt(2.0 / (input_size + hidden_size))
        self.W_xh = np.random.randn(input_size, hidden_size) * scale
        self.W_hh = np.random.randn(hidden_size, hidden_size) * scale
        self.b_h = np.zeros((1, hidden_size))
        self.hidden_size = hidden_size
    def forward(self, x_t, h_prev):
        """Single time step.
        Parameters ———  $x_t$  :
            ndarray, shape (batch, input_size)
             $h_{prev}$  : ndarray, shape (batch, hidden_size)
        Returns ———  $h_{next}$  :
            ndarray, shape (batch, hidden_size)
        """
        z = x_t @ self.W_xh + h_prev @ self.W_hh + self.b_h
        h_next = np.tanh(z)
        return h_next
# Demonstrate: process a short sequence
input_size, hidden_size, seq_len = 3, 5, 8
cell = SimpleRNNCell(input_size, hidden_size)
# Random sequence: (batch=1, time=8, features=3)
x_seq =
```

11.1 Sequential Data

```
np.random.randn(1, seq_len,
input_size) h = np.zeros((1,
hidden_size)) # initial hidden
state

print("Processing sequence
through vanilla RNN:")
print(f" Input shape per step:
{x_seq[:, 0, :].shape}")
print(f" Hidden state shape:
{h.shape}") print()

hiddens = [h] for t in
range(seq_len): h =
cell.forward(x_seq[:, t, :], h)
hiddens.append(h) print(f"
t={t}: h = [{h[0,
:3].round(3)}...]")

print(f"hidden state carries a
summary of all {seq_len}
inputs.") ``

``python # — 11.2 Full RNN
that processes a sequence —

class SimpleRNN: """Full
RNN that unrolls over a
sequence and produces an
output at each step."""

def init(self, input_size,
hidden_size, output_size):
self.cell =
SimpleRNNCell(input_size,
hidden_size) self.hidden_size
= hidden_size # Output
projection scale = np.sqrt(2.0 /
hidden_size) self.W_hy =
np.random.randn(hidden_size,
output_size) * scale self.b_y =
np.zeros((1, output_size))

def forward(self, x_sequence):
"""Process full sequence.

Parameters ———
x_sequence : ndarray, shape
(batch, seq_len, input_size)

Returns ——— outputs :
ndarray, shape (batch, seq_len,
output_size) hiddens : list of
hidden states """ batch_size,
seq_len, _ = x_sequence.shape
h = np.zeros((batch_size,
self.hidden_size))
```

11.1 Sequential Data

```
outputs = []
hiddens = [h]
for t in range(seq_len):
    h = self.cell.forward(x_sequence[:, t, :], h)
    y_t = h @ self.W_hy + self.b_y
    outputs.append(y_t)
    hiddens.append(h)

outputs = np.stack(outputs, axis=1) # (batch, seq_len, output_size)
return outputs, hiddens

# Quick test
rnn = SimpleRNN(input_size=3, hidden_size=5, output_size=1)
test_seq = np.random.randn(2, 10, 3) # batch=2, seq_len=10, input=3
out, hids = rnn.forward(test_seq)
print(f'Input shape: {test_seq.shape}')
print(f'Output shape: {out.shape}')
print(f'Num hidden states: {len(hids)} (initial + {len(hids)-1} steps)')
```

11.3 Backpropagation Through Time (BPTT)

Training an RNN requires computing gradients through the **unrolled** computation graph. Since each hidden state $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, the loss at time T propagates gradients all the way back to $t=1$.

The chain rule through time

For a loss $L = \sum_t L_t$, the gradient of L_T with respect to $\mathbf{W}_{hh}$ involves:

$$\frac{\partial L_T}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial L_T}{\partial \mathbf{h}_T} \left(\prod_{k=t+1}^T \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} \right) \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}}$$

The key term is the product of Jacobians $\prod_{k=t+1}^T \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$.

Since $\mathbf{h}_k = \tanh(\mathbf{W}_{xh} \mathbf{x}_k + \mathbf{W}_{hh} \mathbf{h}_{k-1} + \mathbf{b})$, each Jacobian is:

$$\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} = \text{diag}(1 - \mathbf{h}_k^2) \cdot \mathbf{W}_{hh}$$

The vanishing gradient problem

Multiplying $(\mathbf{W}_{hh})$ with itself $(T-t)$ times means: - If the largest singular value of $(\mathbf{W}_{hh})$ is < 1 : gradients **vanish** exponentially - If the largest singular value is > 1 : gradients **explode** exponentially

The tanh derivative $((1 - h^2))$ is always in $([0, 1])$, which further compounds the vanishing problem.

Truncated BPTT

In practice, we limit backpropagation to the last (k) time steps instead of going all the way to $(t=1)$. This trades off long-range gradient accuracy for computational efficiency and numerical stability.

[illegible]

```

# Normalize each to start at 1 for comparison
norms_vanish = np.array(norms_vanish) / (norms_vanish[-1] + 1e-30)
norms_ok = np.array(norms_ok) / (norms_ok[-1] + 1e-30)
norms_explode = np.array(norms_explode) / (norms_explode[-1] +
1e-30)

steps_back = np.arange(seq_len)

fig, ax = plt.subplots(figsize=(10, 5))
ax.semilogy(steps_back, norms_vanish, color=PALETTE['coral'],
            linewidth=2.5,
            label=r'$\sigma_{\max}(W_{hh}) = 0.7$ (vanishing)',
            zorder=3)
ax.semilogy(steps_back, norms_ok, color=PALETTE['emerald'],
            linewidth=2.5,
            label=r'$\sigma_{\max}(W_{hh}) = 0.95$ (borderline)',
            zorder=3)
ax.semilogy(steps_back, norms_explode,
            color=PALETTE['royal_blue'], linewidth=2.5,
            label=r'$\sigma_{\max}(W_{hh}) = 1.3$ (exploding)',
            zorder=3)

ax.set_xlabel('Steps Back in Time', fontsize=12)
ax.set_ylabel('Relative Gradient Magnitude (log scale)',
            fontsize=12)
ax.set_title('Gradient Flow Through Time: Vanishing vs
Exploding', fontsize=14)
ax.legend(fontsize=10, loc='center left')
ax.set_xlim(0, seq_len - 1)

# Annotate the problem regions
ax.annotate('Gradients vanish\n(cannot learn long-range)',
            xy=(40, norms_vanish[10]),
            fontsize=9, color=PALETTE['coral'],
            fontweight='bold', ha='center')
ax.annotate('Gradients explode\n(training unstable)', xy=(15,
norms_explode[15]),
            fontsize=9, color=PALETTE['royal_blue'],
            fontweight='bold', ha='center',
            va='bottom')

plt.tight_layout()
plt.show()

print("Key insight: vanilla RNNs face a narrow band of
stability.")
print("Gradients either vanish or explode -- motivating gated
architectures.")

```

11.4 Sine Wave Prediction

11.5 Long Short-Term Memory (LSTM)

11.4 Sine Wave Prediction

The **LSTM** (Hochreiter & Schmidhuber, 1997) was specifically designed to solve the vanishing gradient problem. Its key innovation is a **cell state** $\mathbf{c}_t$ that acts as a highway for gradient flow, regulated by three **gates**.

The Three Gates

Forget gate – decides what to discard from the cell state: \

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\right)$$

Input gate – decides what new information to store: \

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\right)$$

Candidate cell state – new information to potentially add: \

$$\tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\right)$$

Cell state update – the gradient

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Output gate – decides what to expose as the hidden state: \

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\right)$$

Hidden state output: \

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Key Insight: The Cell State Highway

The gradient of the loss with respect to an earlier cell state $\mathbf{c}_{t-k}$ flows through the equation:

11.4 Sine Wave Prediction

$$\left[\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}}\right] = \mathbf{f}_t$$

Unlike the vanilla RNN (where the Jacobian is a full matrix multiplication), the LSTM's gradient flow through the cell state involves only **element-wise multiplication** by the forget gate $\mathbf{f}_t$. When $f_t \approx 1$, gradients flow unchanged – solving the vanishing gradient problem.

```python # — 11.5 LSTM Cell Implementation from Scratch

```
class SimpleLSTMCell:
 """LSTM cell with forget,
 input, and output gates."""
 def __init__(self, input_size,
 hidden_size):
 self.hidden_size = hidden_size
 combined = input_size + hidden_size
 scale = np.sqrt(2.0 / combined)
 # All four gate weight matrices
 # concatenated: [f, i, g, o]
 self.W = np.random.randn(combined, 4 * hidden_size) * scale
 self.b = np.zeros((1, 4 * hidden_size))
 # Initialize forget gate bias to 1
 # (helps learning long-range dependencies)
 self.b[0, :hidden_size] = 1.0
 def forward(self, x_t, h_prev, c_prev):
 """Single LSTM step.
 Parameters ——— x_t :
 (batch, input_size)
 h_prev :
 (batch, hidden_size)
 c_prev :
 (batch, hidden_size)
 Returns ——— h_next, c_next :
 each (batch, hidden_size)
 gates : dict with f, i, g, o for
 visualization
 """
 H = self.hidden_size * combined
 x_t_h = np.hstack([x_t, h_prev]) # (batch, input_size + hidden_size)
 gates = combined
```

---

### ## 11.4 Sine Wave Prediction

---

```
@ self.W + self.b # (batch,
4*H)

Split into four gates f =
self._sigmoid(gates[:, :H]) #
forget gate i =
self._sigmoid(gates[:, H:2*H])
input gate g = np.tanh(gates[:,
2H:3H]) # candidate o =
self._sigmoid(gates[:, 3*H:]) #
output gate

Cell state update c_next = f *
c_prev + i * g

Hidden state h_next = o *
np.tanh(c_next)

return h_next, c_next, {'f': f,
'i': i, 'o': o, 'g': g}

@staticmethod def
_sigmoid(x): return 1.0 / (1.0 +
np.exp(-np.clip(x, -500, 500)))

Test LSTM cell
np.random.seed(42) lstm_cell =
SimpleLSTMCell(input_size=3,
hidden_size=5) x_test =
np.random.randn(1, 3) h_prev =
np.zeros((1, 5)) c_prev =
np.zeros((1, 5))

h_next, c_next, gate_vals =
lstm_cell.forward(x_test,
h_prev, c_prev) print("LSTM
Cell test:") print(f' h_next
shape: {h_next.shape}, values:
{h_next[0, :3].round(3)}')
print(f' c_next shape:
{c_next.shape}, values:
{c_next[0, :3].round(3)}')
print(f' Forget gate:
{gate_vals['f'][0,
:3].round(3)}') print(f' Input
gate: {gate_vals['i'][0,
:3].round(3)}') print(f' Output
gate: {gate_vals['o'][0,
:3].round(3)}') ``
```

---

## 11.6 LSTM Gate Activations (Hero Figure)

To build intuition for how LSTM gates operate, we train an LSTM on the sine wave task and record the gate activations over a sequence. The heatmap reveals how each gate specializes: the **forget gate** controls memory retention, the **input gate** controls when to write new information, and the **output gate** controls what to expose.

# --- 11.6 Train LSTM on Sine Wave and Record Gate Values ---

```
class SineLSTM:
 """LSTM for sine wave prediction with manual BPTT."""

 def __init__(self, input_size=1, hidden_size=16, lr=0.005):
 self.lstm = LSTMCell(input_size, hidden_size)
 self.hidden_size = hidden_size
 self.lr = lr
 scale = np.sqrt(2.0 / hidden_size)
 self.W_out = np.random.randn(hidden_size, 1) * scale
 self.b_out = np.zeros((1, 1))

 def forward(self, X):
 batch, seq_len, _ = X.shape
 h = np.zeros((batch, self.hidden_size))
 c = np.zeros((batch, self.hidden_size))
 self._cache = {'inputs': [], 'h': [h], 'c': [c]}

 for t in range(seq_len):
 x_t = X[:, t, :]
 self._cache['inputs'].append(x_t)
 h, c = self.lstm.forward(x_t, h, c)
 self._cache['h'].append(h)
 self._cache['c'].append(c)

 y_pred = h @ self.W_out + self.b_out
 return y_pred

 def backward(self, grad_output):
 seq_len = len(self._cache['inputs'])
 h_last = self._cache['h'][-1]

 grad_W_out = h_last.T @ grad_output
 grad_b_out = np.sum(grad_output, axis=0, keepdims=True)
 grad_h = grad_output @ self.W_out.T
 grad_c = np.zeros_like(self._cache['c'][-1])

 grad_W = np.zeros_like(self.lstm.W)
 grad_b = np.zeros_like(self.lstm.b)

 for t in range(seq_len - 1, -1, -1):
 x_t = self._cache['inputs'][t]
 h_prev = self._cache['h'][t]
 c_prev = self._cache['c'][t]
 h_next = self._cache['h'][t + 1]
 c_next = self._cache['c'][t + 1]

 H = self.hidden_size
 combined = np.hstack([x_t, h_prev])
 gates_raw = combined @ self.lstm.W + self.lstm.b

 f = 1.0 / (1.0 + np.exp(-gates_raw[:, :H]))
```

```

i = 1.0 / (1.0 + np.exp(-gates_raw[:, H:2*H]))
g = np.tanh(gates_raw[:, 2*H:3*H])
o = 1.0 / (1.0 + np.exp(-gates_raw[:, 3*H:]))

tanh_c = np.tanh(c_next)
grad_c = grad_c + grad_h * o * (1 - tanh_c ** 2)

grad_f = grad_c * c_prev * f * (1 - f)
grad_i = grad_c * g * i * (1 - i)
grad_g = grad_c * i * (1 - g ** 2)
grad_o = grad_h * tanh_c * o * (1 - o)

grad_gates = np.hstack([grad_f, grad_i, grad_g,
grad_o])
grad_W += combined.T @ grad_gates
grad_b += np.sum(grad_gates, axis=0, keepdims=True)

grad_combined = grad_gates @ self.lstm.W.T
input_size = x_t.shape[1]
grad_h = grad_combined[:, input_size:]
grad_c = grad_c * f

Clip and update
for g in [grad_W, grad_b, grad_W_out, grad_b_out]:
 np.clip(g, -5.0, 5.0, out=g)

self.lstm.W -= self.lr * grad_W
self.lstm.b -= self.lr * grad_b
self.W_out -= self.lr * grad_W_out
self.b_out -= self.lr * grad_b_out

def predict(self, X):
 batch, seq_len, _ = X.shape
 h = np.zeros((batch, self.hidden_size))
 c = np.zeros((batch, self.hidden_size))
 for t in range(seq_len):
 h, c = self.lstm.forward(X[:, t, :], h, c)
 return h @ self.W_out + self.b_out

def forward_record_gates(self, X):
 """Forward pass that records gate activations for
 visualization."""
 batch, seq_len, _ = X.shape
 h = np.zeros((batch, self.hidden_size))
 c = np.zeros((batch, self.hidden_size))
 H = self.hidden_size

 forget_gates, input_gates, output_gates = [], [], []

 for t in range(seq_len):
 x_t = X[:, t, :]
 combined = np.hstack([x_t, h])
 gates_raw = combined @ self.lstm.W + self.lstm.b

```



```

 f = 1.0 / (1.0 + np.exp(-gates_raw[:, :H]))
 i = 1.0 / (1.0 + np.exp(-gates_raw[:, H:2*H]))
 g = np.tanh(gates_raw[:, 2*H:3*H])
 o = 1.0 / (1.0 + np.exp(-gates_raw[:, 3*H:]))

 c = f * c + i * g
 h = o * np.tanh(c)

 forget_gates.append(f[0]) # take first sample in
batch
 input_gates.append(i[0])
 output_gates.append(o[0])

 return (np.array(forget_gates), # (seq_len, hidden_size)
 np.array(input_gates),
 np.array(output_gates))

Train LSTM
np.random.seed(42)
model_lstm_sine = SineLSTM(input_size=1, hidden_size=16, lr=0.003)
lstm_losses = []
epochs = 200

for epoch in range(epochs):
 idx = np.random.permutation(n_train)
 epoch_loss = 0.0
 n_batches = 0

 for start in range(0, n_train, batch_size):
 batch_idx = idx[start:start + batch_size]
 Xb = X_train[batch_idx]
 yb = y_train[batch_idx].reshape(-1, 1)

 y_pred = model_lstm_sine.forward(Xb)
 loss = np.mean((y_pred - yb) ** 2)
 epoch_loss += loss
 n_batches += 1

 grad = 2.0 * (y_pred - yb) / len(yb)
 model_lstm_sine.backward(grad)

 lstm_losses.append(epoch_loss / n_batches)
 if (epoch + 1) % 50 == 0:
 print(f"Epoch {epoch+1:3d}/{epochs} -- LSTM MSE:
 {lstm_losses[-1]:.6f}")

print(f"\nFinal LSTM training MSE: {lstm_losses[-1]:.6f}")

--- 11.6 HERO FIGURE: LSTM Gate Activations Heatmap ---

Record gate activations on a single test sequence

```

```

sample_seq = X_test[0:1] # shape (1, 50, 1)
f_gates, i_gates, o_gates =
 model_lstm_sine.forward_record_gates(sample_seq)
Each is (seq_len, hidden_size)

fig, axes = plt.subplots(3, 1, figsize=(14, 8), sharex=True)

gate_data = [
 (f_gates.T, 'Forget Gate $\mathbf{f}_t$ ', 'Blues'),
 (i_gates.T, 'Input Gate $\mathbf{i}_t$ ', 'Oranges'),
 (o_gates.T, 'Output Gate $\mathbf{o}_t$ ', 'Greens'),
]

for idx, (data, title, cmap) in enumerate(gate_data):
 ax = axes[idx]
 im = ax.imshow(data, aspect='auto', cmap=cmap, vmin=0, vmax=1,
 interpolation='nearest')
 ax.set_ylabel('Hidden\nUnit', fontsize=10)
 ax.set_title(title, fontsize=12, pad=8)
 cbar = plt.colorbar(im, ax=ax, fraction=0.02, pad=0.02)
 cbar.set_label('Activation', fontsize=9)
 ax.set_yticks([0, data.shape[0] // 2, data.shape[0] - 1])

axes[-1].set_xlabel('Time Step', fontsize=12)

Add input signal below for reference
Create a twin axis on the bottom subplot
ax_signal = axes[-1].twinx()
ax_signal.set_xlim(axes[-1].get_xlim())
ax_signal.tick_params(top=False, labeltop=False)

fig.suptitle('LSTM Gate Activations Over a Sine Wave Sequence',
 fontsize=15, fontweight='bold',
 color=PALETTE['deep_blue'], y=1.02)

plt.tight_layout()
plt.show()

print("The heatmap reveals how different hidden units
 specialize:")
print(" - Forget gate near 1.0 (dark blue): unit retains memory")
print(" - Input gate active (dark orange): unit is writing new
 information")
print(" - Output gate modulates what the cell exposes to the next
 layer")

```

---

## ## 11.7 Gated Recurrent Unit (GRU)

---

### ## 11.8 Character-Level Language Model

To see RNNs work on discrete sequences, let's build a **character-level language model**. Given a

---

## ## 11.7 Gated Recurrent Unit (GRU)

---

sequence of characters, the model predicts the next character. We train on a simple repeated text and watch the model learn the pattern.

```
```python # — 11.8 Character-  
Level Language Model —  
  
# Prepare data text = "hello world"  
* 30 chars = sorted(set(text))  
char_to_idx = {ch: i for i, ch in  
enumerate(chars)} idx_to_char =  
{i: ch for ch, i in  
char_to_idx.items()} vocab_size =  
len(chars)  
  
print(f"Text length: {len(text)}")  
print(f"Vocabulary: {chars}  
({vocab_size} chars)")  
  
def one_hot_encode(indices,  
vocab_size): """Convert list of  
char indices to one-hot matrix."""  
oh = np.zeros((len(indices),  
vocab_size))  
oh[np.arange(len(indices)),  
indices] = 1.0 return oh  
  
class CharRNN: """Character-  
level RNN language model using  
LSTM."""  
  
def init(self, vocab_size,  
hidden_size, lr=0.01):  
self.vocab_size = vocab_size  
self.hidden_size = hidden_size  
self.lr = lr  
  
combined = vocab_size +  
hidden_size scale = np.sqrt(2.0 /  
combined)  
  
# LSTM parameters (concatenated  
gates) self.W =  
np.random.randn(combined, 4 *  
hidden_size) * scale self.b =  
np.zeros((1, 4 * hidden_size))  
self.b[0, :hidden_size] = 1.0 #  
forget gate bias  
  
# Output projection self.W_out =  
np.random.randn(hidden_size,  
vocab_size) * np.sqrt(2.0 /  
hidden_size) self.b_out =  
np.zeros((1, vocab_size))
```

11.7 Gated Recurrent Unit (GRU)

```
def _sigmoid(self, x): return 1.0 /
(1.0 + np.exp(-np.clip(x, -500,
500)))

def _softmax(self, x): e = np.exp(x
- np.max(x, axis=-1,
keepdims=True)) return e /
e.sum(axis=-1, keepdims=True)

def forward(self, inputs, h_prev,
c_prev): """Forward pass through
sequence of one-hot vectors.""" H
= self.hidden_size h, c = h_prev,
c_prev cache = [] outputs = []
for t in range(len(inputs)): x_t =
inputs[t:t+1] # (1, vocab_size)
combined = np.hstack([x_t, h])
gates = combined @ self.W +
self.b

f = self._sigmoid(gates[:, :H]) i =
self._sigmoid(gates[:, H:2*H]) g =
np.tanh(gates[:, 2H:3H]) o =
self._sigmoid(gates[:, 3*H:])
c = f * c + i * g h = o * np.tanh(c)

logits = h @ self.W_out +
self.b_out probs =
self._softmax(logits)
outputs.append(probs)
cache.append((x_t, combined, f, i,
g, o, c.copy(), h.copy()))
return outputs, h, c, cache

def backward(self, outputs, targets,
cache): """BPTT through the
character sequence.""" H =
self.hidden_size grad_W =
np.zeros_like(self.W) grad_b =
np.zeros_like(self.b) grad_W_out
= np.zeros_like(self.W_out)
grad_b_out =
np.zeros_like(self.b_out) grad_h =
np.zeros((1, H)) grad_c =
np.zeros((1, H))

loss = 0.0 for t in
range(len(outputs) - 1, -1, -1):
probs = outputs[t] target =
targets[t]

loss -= np.log(probs[0, target] +
1e-8)
```

11.7 Gated Recurrent Unit (GRU)

```
# Softmax gradient dlogits =
probs.copy() dlogits[0, target] -=
1.0

x_t, combined, f, i, g, o, c_t, h_t =
cache[t]

grad_W_out += h_t.T @ dlogits
grad_b_out += dlogits grad_h_t =
dlogits @ self.W_out.T + grad_h
tanh_c = np.tanh(c_t) grad_c_t =
grad_c + grad_h_t * o * (1 -
tanh_c ** 2)

c_prev = cache[t - 1][6] if t > 0
else np.zeros_like(c_t)

grad_f = grad_c_t * c_prev * f * (1
- f) grad_i = grad_c_t * g * i * (1 -
i) grad_g = grad_c_t * i * (1 - g **
2) grad_o = grad_h_t * tanh_c * o
* (1 - o)

grad_gates = np.hstack([grad_f,
grad_i, grad_g, grad_o]) grad_W
+= combined.T @ grad_gates
grad_b += grad_gates

grad_combined = grad_gates @
self.W.T grad_h =
grad_combined[:, self.vocab_size:]
grad_c = grad_c_t * f

# Clip gradients for g in [grad_W,
grad_b, grad_W_out, grad_b_out]:
np.clip(g, -5.0, 5.0, out=g)

self.W -= self.lr * grad_W self.b
-= self.lr * grad_b self.W_out -=
self.lr * grad_W_out self.b_out -=
self.lr * grad_b_out

return loss / len(outputs)

def generate(self, seed_char,
length=50, temperature=0.8):
    """Generate text character by
    character.""" H = self.hidden_size
    h = np.zeros((1, H)) c =
    np.zeros((1, H)) result =
    [seed_char] idx =
    char_to_idx[seed_char]

    for _ in range(length): x =
    np.zeros((1, self.vocab_size)) x[0,
    idx] = 1.0

    combined = np.hstack([x, h]) gates
    = combined @ self.W + self.b
```

11.7 Gated Recurrent Unit (GRU)

```
f = self._sigmoid(gates[:, :H]) i =
self._sigmoid(gates[:, H:2*H]) g =
np.tanh(gates[:, 2*H:3*H]) o =
self._sigmoid(gates[:, 3*H:])
c = f * c + i * g h = o * np.tanh(c)
logits = h @ self.W_out +
self.b_out logits = logits /
temperature probs =
self._softmax(logits).ravel()
idx =
np.random.choice(len(probs),
p=probs)
result.append(idx_to_char[idx])
return ''.join(result)

# Encode text data =
[char_to_idx[ch] for ch in text]
encoded = one_hot_encode(data,
vocab_size)

# Train np.random.seed(42)
char_model =
CharRNN(vocab_size=vocab_size,
hidden_size=64, lr=0.005)
seq_length = 25 char_losses = []
n_iters = 500

print("Training character-level
language model...")

for iteration in range(n_iters): #
Random starting point start =
np.random.randint(0, len(data) -
seq_length - 1) inputs =
encoded[start:start + seq_length]
targets = data[start + 1:start +
seq_length + 1]

h_init = np.zeros((1,
char_model.hidden_size)) c_init =
np.zeros((1,
char_model.hidden_size))

outputs, , cache =
char_model.forward(inputs, h_init,
c_init) loss =
char_model.backward(outputs,
targets, cache)
char_losses.append(loss)

if (iteration + 1) % 100 == 0:
sample = char_model.generate('h',
length=40, temperature=0.6)
print(f'Iter {iteration+1:4d} |
```

11.7 Gated Recurrent Unit (GRU)

```
Loss: {loss:.3f} | Sample:
'{sample}'")
print("-- Final generated samples
---") for temp in [0.3, 0.6, 1.0]:
    sample = char_model.generate('h',
length=50, temperature=temp)
print(f" temp={temp}:
'{sample}'") ``
```

11.9 RNN vs LSTM: Long-Range Dependency

The real advantage of LSTM becomes clear on tasks that require remembering information from many time steps ago. We create a synthetic task: the model must recall a **signal** placed at the very start of a 30-step sequence to produce the correct output at the end.

--- 11.9 RNN vs LSTM on Long-Range Dependency Task ---

```
def make_long_range_data(n_samples=500, seq_len=30, seed=42):
    """Create a task requiring memory over seq_len steps.

    The first time step contains a signal in {-1, +1}.
    The rest are noise. The target is the sign of the first
    signal.
    """
    rng = np.random.default_rng(seed)
    X = rng.normal(0, 0.1, (n_samples, seq_len, 1))
    signals = rng.choice([-1.0, 1.0], size=n_samples)
    X[:, 0, 0] = signals
    y = (signals > 0).astype(float)
    return X, y

def train_rnn_classifier(cell_type, X, y, hidden_size=16,
    lr=0.01, epochs=100):
    """Train a simple sequence classifier with RNN or LSTM."""
    np.random.seed(42)
    n_samples, seq_len, input_size = X.shape
    losses = []

    if cell_type == 'rnn':
        cell = RNNCell(input_size, hidden_size)
    elif cell_type == 'lstm':
        cell = LSTMCell(input_size, hidden_size)

    # Output layer
    W_out = np.random.randn(hidden_size, 1) * np.sqrt(2.0 /
        hidden_size)
    b_out = np.zeros((1, 1))
```

```

sigmoid = lambda x: 1.0 / (1.0 + np.exp(-np.clip(x, -500,
500)))

for epoch in range(epochs):
    # Forward pass
    h = np.zeros((n_samples, hidden_size))
    c = np.zeros((n_samples, hidden_size))
    h_list = [h]
    c_list = [c]

    for t in range(seq_len):
        x_t = X[:, t, :]
        if cell_type == 'rnn':
            h = cell.forward(x_t, h)
        else:
            h, c = cell.forward(x_t, h, c)
            c_list.append(c)
            h_list.append(h)

    logits = h @ W_out + b_out
    y_pred = sigmoid(logits)
    y_true = y.reshape(-1, 1)

    eps = 1e-8
    loss = -np.mean(y_true * np.log(y_pred + eps) + (1 -
y_true) * np.log(1 - y_pred + eps))
    losses.append(loss)

    # Backward through output
    grad_logits = (y_pred - y_true) / n_samples
    grad_W_out = h.T @ grad_logits
    grad_b_out = np.sum(grad_logits, axis=0, keepdims=True)
    grad_h = grad_logits @ W_out.T

    # BPTT
    if cell_type == 'rnn':
        grad_W_xh = np.zeros_like(cell.W_xh)
        grad_W_hh = np.zeros_like(cell.W_hh)
        grad_b_h = np.zeros_like(cell.b_h)

        for t in range(seq_len - 1, -1, -1):
            h_next = h_list[t + 1]
            h_prev = h_list[t]
            x_t = X[:, t, :]
            dtanh = grad_h * (1.0 - h_next ** 2)
            grad_W_xh += x_t.T @ dtanh
            grad_W_hh += h_prev.T @ dtanh
            grad_b_h += np.sum(dtanh, axis=0, keepdims=True)
            grad_h = dtanh @ cell.W_hh.T

        for g in [grad_W_xh, grad_W_hh, grad_b_h]:
            np.clip(g, -5.0, 5.0, out=g)

```



```

cell.W_xh -= lr * grad_W_xh
cell.W_hh -= lr * grad_W_hh
cell.b_h -= lr * grad_b_h

elif cell_type == 'lstm':
    H = hidden_size
    grad_W_lstm = np.zeros_like(cell.W)
    grad_b_lstm = np.zeros_like(cell.b)
    grad_c = np.zeros_like(c_list[-1])

    for t in range(seq_len - 1, -1, -1):
        x_t = X[:, t, :]
        h_prev = h_list[t]
        c_prev = c_list[t]
        c_curr = c_list[t + 1]
        h_curr = h_list[t + 1]

        combined = np.hstack([x_t, h_prev])
        gates_raw = combined @ cell.W + cell.b

        f = sigmoid(gates_raw[:, :H])
        i = sigmoid(gates_raw[:, H:2*H])
        g = np.tanh(gates_raw[:, 2*H:3*H])
        o = sigmoid(gates_raw[:, 3*H:])

        tanh_c = np.tanh(c_curr)
        grad_c = grad_c + grad_h * o * (1 - tanh_c ** 2)

        grad_f = grad_c * c_prev * f * (1 - f)
        grad_i = grad_c * g * i * (1 - i)
        grad_g = grad_c * i * (1 - g ** 2)
        grad_o = grad_h * tanh_c * o * (1 - o)

        grad_gates = np.hstack([grad_f, grad_i, grad_g,
                                grad_o])
        grad_W_lstm += combined.T @ grad_gates
        grad_b_lstm += np.sum(grad_gates, axis=0,
                                keepdims=True)

        grad_combined = grad_gates @ cell.W.T
        grad_h = grad_combined[:, input_size:]
        grad_c = grad_c * f

    for g in [grad_W_lstm, grad_b_lstm]:
        np.clip(g, -5.0, 5.0, out=g)
    cell.W -= lr * grad_W_lstm
    cell.b -= lr * grad_b_lstm

# Update output layer
np.clip(grad_W_out, -5.0, 5.0, out=grad_W_out)
np.clip(grad_b_out, -5.0, 5.0, out=grad_b_out)
W_out -= lr * grad_W_out
b_out -= lr * grad_b_out

```

```

    return losses

# Generate data
X_lr, y_lr = make_long_range_data(n_samples=200, seq_len=30,
                                   seed=42)
print(f"Long-range dependency task: remember signal from step 0
      across {X_lr.shape[1]} steps")
print(f"Data shape: {X_lr.shape}, Labels: {y_lr.shape}")

# Train both
print("\nTraining vanilla RNN...")
rnn_losses = train_rnn_classifier('rnn', X_lr, y_lr,
                                   hidden_size=16, lr=0.01, epochs=150)
print(f"  Final RNN loss: {rnn_losses[-1]:.4f}")

print("Training LSTM...")
lstm_lr_losses = train_rnn_classifier('lstm', X_lr, y_lr,
                                       hidden_size=16, lr=0.01, epochs=150)
print(f"  Final LSTM loss: {lstm_lr_losses[-1]:.4f}")

# --- 11.9 Comparison Plot ---

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(rnn_losses, color=PALETTE['coral'], linewidth=2.5,
        label='Vanilla RNN', alpha=0.9)
ax.plot(lstm_lr_losses, color=PALETTE['royal_blue'],
        linewidth=2.5, label='LSTM', alpha=0.9)

# Random baseline
ax.axhline(y=np.log(2), color=PALETTE['warm_gray'],
           linestyle=':', linewidth=1.5,
           label=f'Random baseline (ln 2 = {np.log(2):.3f})')

ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Binary Cross-Entropy Loss', fontsize=12)
ax.set_title('Long-Range Dependency: RNN vs LSTM (30-step
             memory)', fontsize=14)
ax.legend(fontsize=11)
ax.set_ylim(0, 1.0)

# Annotate
ax.annotate('LSTM learns the task', xy=(120,
    lstm_lr_losses[min(120, len(lstm_lr_losses)-1)]),
            xytext=(80, 0.15), fontsize=10,
            color=PALETTE['royal_blue'], fontweight='bold',
            arrowprops=dict(arrowstyle='->',
                            color=PALETTE['royal_blue'], lw=1.5))

ax.annotate('RNN stuck near random', xy=(120, rnn_losses[min(120,
    len(rnn_losses)-1)]),
            xytext=(60, 0.85), fontsize=10,
            color=PALETTE['coral'], fontweight='bold',

```

```

        arrowprops=dict(arrowstyle='->',
                        color=PALETTE['coral'], lw=1.5))

plt.tight_layout()
plt.show()

print("The LSTM cell state provides a gradient highway that
      allows")
print("information to persist over 30 time steps, while the
      vanilla")
print("RNN's gradients vanish and it cannot solve the task.")

```

11.10 Hidden State Evolution

11.11 Gradient Clipping

When gradients explode, training becomes unstable – loss spikes and parameters diverge.

Gradient clipping caps the gradient norm to a maximum value, preventing catastrophic updates while preserving gradient direction.

$$\|\mathbf{g}\| \leftarrow \frac{\|\mathbf{g}\|}{\|\mathbf{g}\|} \cdot \|\mathbf{g}\| \quad \text{if } \|\mathbf{g}\| > \text{max_norm}$$

```python # — 11.11 Gradient Clipping Demonstration —

```

def clip_gradients(grads,
max_norm): """Clip gradient list
by global norm.""" total_norm =
np.sqrt(sum(np.sum(g ** 2) for
g in grads)) clip_coef =
max_norm / (total_norm + 1e-8)
if clip_coef < 1.0: for g in grads:
g *= clip_coef return total_norm

```

```

def train_rnn_with_clipping(X,
y, clip_norm, hidden_size=16,
lr=0.01, epochs=100): """Train
vanilla RNN with or without
gradient clipping."""
np.random.seed(42) n_samples,
seq_len, input_size = X.shape
losses = [] grad_norms = []
cell = RNNCell(input_size,
hidden_size) W_out =
np.random.randn(hidden_size, 1)
* np.sqrt(2.0 / hidden_size)
b_out = np.zeros((1, 1))

```

---

### ## 11.10 Hidden State Evolution

---

```
for epoch in range(epochs): #
 Forward h =
 np.zeros((n_samples,
 hidden_size)) h_list = [h] for t in
 range(seq_len): x_t = X[:, t, :] h
 = cell.forward(x_t, h)
 h_list.append(h)

 y_pred = h @ W_out + b_out
 loss = np.mean((y_pred -
 y.reshape(-1, 1)) ** 2)
 losses.append(loss)

 # Backward grad_out = 2.0 *
 (y_pred - y.reshape(-1, 1)) /
 n_samples grad_W_out = h.T @
 grad_out grad_b_out =
 np.sum(grad_out, axis=0,
 keepdims=True) grad_h =
 grad_out @ W_out.T

 grad_W_xh =
 np.zeros_like(cell.W_xh)
 grad_W_hh =
 np.zeros_like(cell.W_hh)
 grad_b_h =
 np.zeros_like(cell.b_h)

 for t in range(seq_len - 1, -1, -1):
 h_next = h_list[t + 1] h_prev =
 h_list[t] x_t = X[:, t, :] dtanh =
 grad_h * (1.0 - h_next ** 2)
 grad_W_xh += x_t.T @ dtanh
 grad_W_hh += h_prev.T @
 dtanh grad_b_h +=
 np.sum(dtanh, axis=0,
 keepdims=True) grad_h = dtanh
 @ cell.W_hh.T

 all_grads = [grad_W_xh,
 grad_W_hh, grad_b_h,
 grad_W_out, grad_b_out]

 # Measure gradient norm before
 clipping norm =
 np.sqrt(sum(np.sum(g ** 2) for
 g in all_grads))
 grad_norms.append(norm)

 # Apply clipping if clip_norm is
 not None:
 clip_gradients(all_grads,
 clip_norm)

 # Update cell.W_xh -= lr *
 all_grads[0] cell.W_hh -= lr *
 all_grads[1] cell.b_h -= lr *
```

---

### ## 11.10 Hidden State Evolution

---

```
all_grads[2] W_out -= lr *
all_grads[3] b_out -= lr *
all_grads[4]

return losses, grad_norms

Generate challenging data
(longer sequence) X_clip, y_clip
= make_sine_sequence(n=200,
seq_len=50, seed=99)

Train without clipping
print("Training without gradient
clipping...") losses_noclip,
norms_noclip =
train_rnn_with_clipping(X_clip,
y_clip, clip_norm=None,
hidden_size=32, lr=0.005,
epochs=100)

Train with clipping
print("Training with gradient
clipping (max_norm=5)...")
losses_clip, norms_clip =
train_rnn_with_clipping(X_clip,
y_clip, clip_norm=5.0,
hidden_size=32, lr=0.005,
epochs=100)

Plot fig, axes = plt.subplots(1,
2, figsize=(14, 5))

Left: loss curves ax = axes[0]
ax.plot(losses_noclip,
color=PALETTE['coral'],
linewidth=2.0, label='No
clipping', alpha=0.9)
ax.plot(losses_clip,
color=PALETTE['royal_blue'],
linewidth=2.0, label='With
clipping (norm=5)', alpha=0.9)
ax.set_xlabel('Epoch',
fontsize=11) ax.set_ylabel('MSE
Loss', fontsize=11)
ax.set_title('Training Loss:
Effect of Gradient Clipping',
fontsize=13)
ax.legend(fontsize=10)
ax.set_ylim(0,
min(max(losses_noclip[:20]), 5))

Right: gradient norms ax =
axes[1]
ax.semilogy(norms_noclip,
color=PALETTE['coral'],
linewidth=1.5, label='No
```

---

### ## 11.10 Hidden State Evolution

---

```
clipping', alpha=0.8)
ax.semilogy(norms_clip,
color=PALETTE['royal_blue'],
linewidth=1.5, label='With
clipping', alpha=0.8)
ax.axhline(y=5.0,
color=PALETTE['emerald'],
linestyle='-', linewidth=1.5,
label='Clip threshold (5.0)',
alpha=0.7)
ax.set_xlabel('Epoch',
fontsize=11)
ax.set_ylabel('Gradient Norm
(log)', fontsize=11)
ax.set_title('Gradient Norms
During Training', fontsize=13)
ax.legend(fontsize=10)
plt.tight_layout() plt.show()

print("Gradient clipping
prevents the destructive spikes
that cause training instability.")
``
```

---

## 11.12 Exercises

### Exercise 1: Bidirectional RNN

Implement a **bidirectional RNN** that processes the sequence in both forward and backward directions and concatenates the hidden states. Apply it to the sine wave prediction task.

**Steps:** 1. Create two `RNNCell` instances (forward and backward). 2. Run one cell left-to-right, the other right-to-left. 3. Concatenate the hidden states at each time step:  $\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$ . 4. Compare prediction accuracy with the unidirectional RNN from Section 11.4.

### Exercise 2: Sequence-to-Sequence Addition

Build a character-level model that learns to **add two numbers** represented as strings. For example, input "12+34" should produce "46".

**Steps:** 1. Generate training pairs: random 1-3 digit additions. 2. Encode input and output as character sequences (characters: 0-9, +, and a padding token). 3. Use an encoder LSTM to read the input, then a decoder LSTM to produce the output. 4. Report character-level and full-sequence accuracy.

## Exercise 3: Hyperparameter Sensitivity

Investigate the effect of **hidden size** on the long-range dependency task from Section 11.9.

**Steps:** 1. Train LSTM models with hidden sizes  $(H \in \{4, 8, 16, 32, 64\})$  on the 30-step memory task. 2. Plot final loss vs. hidden size. 3. Also vary the sequence length  $(T \in \{10, 20, 30, 50\})$  for a fixed  $(H=16)$  and plot the results. 4. At what sequence length does even the LSTM begin to struggle?

```
import sys; sys.path.insert(0, "..")
import numpy as np
import matplotlib.pyplot as plt
from style.dl_style import apply_style, PALETTE
from utils.nn import Linear, ReLU, Sequential, BatchNorm,
 Residual, LayerNorm, CrossEntropyLoss
from utils.data import make_spirals, train_test_split
from utils.plot import plot_loss_curve
apply_style()
np.random.seed(42)
```

## Chapter 12: Advanced Topics

Throughout this book we have built neural networks layer by layer, starting from a single perceptron and progressing through deep feedforward networks, convolutional architectures, and recurrent models. In this final chapter we turn to the ideas that power **modern deep learning**: the **attention mechanism**, **multi-head attention**, **residual connections**, the **Transformer block**, and **positional encodings**.

### Learning Objectives

By the end of this chapter you will be able to:

1. Implement **scaled dot-product attention** from scratch and interpret the attention weight matrix.
2. Extend single-head attention to **multi-head attention** and observe how different heads learn different patterns.
3. Explain why **residual (skip) connections** solve the vanishing gradient problem and demonstrate their effect on training.
4. Assemble a complete **Transformer encoder block** from LayerNorm, multi-head attention, and a feed-forward network.
5. Derive and visualize **sinusoidal positional encodings** that inject order information into position-agnostic architectures.
6. Build and train a **ResNet-style network** on a classification task, comparing it against a plain deep network.
7. Measure and compare **gradient flow** through residual and non-residual architectures.

---

## ## 12.1 Attention Mechanism

---

### ## 12.2 Multi-Head Attention

A single attention head can only focus on one type of relationship at a time. **Multi-head attention** runs several attention heads in parallel, each with its own learned projection, then concatenates the results.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where each head is:

$$\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

Each head can learn to attend to different aspects: - Head 1 might focus on nearby positions (local context) - Head 2 might focus on syntactic relationships - Head 3 might focus on semantic similarity - Head 4 might focus on long-range dependencies

``python # — 12.2 Multi-Head Attention

def multi\_head\_attention(Q, K, V,  
n\_heads=4, W\_Q=None, W\_K=None,  
W\_V=None, W\_O=None): """Multi-head  
attention.

Args: Q: queries, shape (seq\_len, d\_model)  
K: keys, shape (seq\_len, d\_model) V:  
values, shape (seq\_len, d\_model) n\_heads:  
number of attention heads W\_Q, W\_K,  
W\_V: projection matrices, shape (d\_model,  
d\_model) W\_O: output projection, shape  
(d\_model, d\_model)

Returns: output: shape (seq\_len, d\_model)  
all\_weights: list of weight matrices, one per  
head """ seq\_len, d\_model = Q.shape  
d\_head = d\_model // n\_heads assert  
d\_model % n\_heads == 0, "d\_model must  
be divisible by n\_heads"

# Initialize projections if not provided if  
W\_Q is None: scale = np.sqrt(2.0 /  
d\_model) W\_Q =  
np.random.randn(d\_model, d\_model) \*  
scale W\_K = np.random.randn(d\_model,  
d\_model) \* scale W\_V =  
np.random.randn(d\_model, d\_model) \*  
scale W\_O = np.random.randn(d\_model,  
d\_model) \* scale

# Project Q, K, V Q\_proj = Q @ W\_Q #  
(seq\_len, d\_model) K\_proj = K @ W\_K  
V\_proj = V @ W\_V



---

## ## 12.1 Attention Mechanism

---

```
Split into heads: reshape to (n_heads,
seq_len, d_head) Q_heads =
Q_proj.reshape(seq_len, n_heads,
d_head).transpose(1, 0, 2) K_heads =
K_proj.reshape(seq_len, n_heads,
d_head).transpose(1, 0, 2) V_heads =
V_proj.reshape(seq_len, n_heads,
d_head).transpose(1, 0, 2)

Attention per head head_outputs = []
all_weights = [] for h in range(n_heads):
 out_h, w_h =
scaled_dot_product_attention(Q_heads[h],
K_heads[h], V_heads[h])
 head_outputs.append(out_h) # (seq_len,
d_head) all_weights.append(w_h) #
(seq_len, seq_len)

Concatenate heads: (seq_len, d_model)
concat = np.concatenate(head_outputs,
axis=-1)

Output projection output = concat @
W_O

return output, all_weights

— Demonstrate — np.random.seed(42)
seq_len, d_model = 8, 32 n_heads = 4

X_mha = np.random.randn(seq_len,
d_model) mha_output, head_weights =
multi_head_attention(X_mha, X_mha,
X_mha, n_heads=n_heads)

print(f'Input shape: {X_mha.shape}')
print(f'Output shape:
{mha_output.shape}') print(f'Number of
heads: {len(head_weights)}') print(f'Each
head weight shape:
{head_weights[0].shape}') ``

``python # — 12.2 Attention Patterns for
Different Heads (2x2 grid) —

fig, axes = plt.subplots(2, 2, figsize=(11,
10))

head_colors =
[LinearSegmentedColormap.from_list('h0',
['#FFFFFF', PALETTE['deep_blue']],
N=256),
LinearSegmentedColormap.from_list('h1',
['#FFFFFF', PALETTE['coral']], N=256),
LinearSegmentedColormap.from_list('h2',
['#FFFFFF', PALETTE['emerald']],
N=256),
LinearSegmentedColormap.from_list('h3',
```

---

## ## 12.1 Attention Mechanism

---

```
['#FFFFFF', PALETTE['amber']],
N=256),]

head_names = ['Head 1', 'Head 2', 'Head
3', 'Head 4'] token_labels = [f'tok_{i}' for
i in range(seq_len)]

for idx, ax in enumerate(axes.flat): w =
head_weights[idx] im = ax.imshow(w,
cmap=head_colors[idx], aspect='equal',
vmin=0)

for i in range(seq_len): for j in
range(seq_len): val = w[i, j] color = 'white'
if val > 0.20 else PALETTE['charcoal']
ax.text(j, i, f'{val:.2f}', ha='center',
va='center', fontsize=7, color=color)

ax.set_xticks(range(seq_len))
ax.set_yticks(range(seq_len))
ax.set_xticklabels(token_labels, fontsize=7,
rotation=45, ha='right')
ax.set_yticklabels(token_labels, fontsize=7)
ax.set_title(head_names[idx], fontsize=12,
color=PALETTE['deep_blue'])
plt.colorbar(im, ax=ax, fraction=0.046,
pad=0.04)

fig.suptitle('Multi-Head Attention Patterns',
fontsize=14, fontweight='bold',
color=PALETTE['deep_blue'], y=1.01)
plt.tight_layout() plt.show() ````
```

---

## 12.3 Residual Connections

### The Problem: Vanishing Gradients in Deep Networks

As networks get deeper, gradients must travel through many layers during backpropagation. At each layer, the gradient is multiplied by the layer's Jacobian, and these repeated multiplications can shrink the gradient exponentially – this is the **vanishing gradient problem**.

### The Solution: Skip Connections

A **residual connection** (or skip connection) adds the input directly to the output of a block:

$$y = F(x) + x$$

where  $F(x)$  is the transformation learned by the block (e.g., two linear layers with an activation in between).

**Why this works:** - During backpropagation, the gradient flows through two paths: - Through  $\frac{\partial F(x)}{\partial x}$  (may shrink) - Through the identity shortcut:  $\frac{\partial x}{\partial x} = 1$  (always 1) - The identity path guarantees that gradients can flow unimpeded to earlier layers. - The network only needs to learn the **residual**  $F(x) = y - x$ , which is often easier than learning the full mapping.

## Implementation

We already have a Residual class in our utility library (see `utils/nn.py`). It wraps any sequence of layers and adds the skip connection:

```
class Residual:
 def __init__(self, *layers):
 self.layers = list(layers)

 def forward(self, x):
 out = x
 for layer in self.layers:
 out = layer.forward(out)
 return out + x # <-- the skip connection

--- 12.3 Training Comparison: Residual vs Plain Deep Network ---

from utils.optim import Adam

Generate data
np.random.seed(42)
X_data, y_data = make_spirals(n=600, n_classes=3, noise=0.25,
 seed=42)
X_train, X_test, y_train, y_test = train_test_split(X_data,
 y_data, test_ratio=0.2, seed=42)

hidden = 64
n_blocks = 10 # 10 blocks => 20+ layers

def build_plain_network(n_blocks, hidden):
 """Build a deep plain network (no skip connections)."""
 layers = [Linear(2, hidden), ReLU()]
 for _ in range(n_blocks):
 layers.append(Linear(hidden, hidden))
 layers.append(ReLU())
 layers.append(Linear(hidden, 3))
 return Sequential(*layers)

def build_residual_network(n_blocks, hidden):
 """Build a deep residual network (with skip connections)."""
 layers = [Linear(2, hidden), ReLU()]
 for _ in range(n_blocks):
 layers.append(Residual(
 Linear(hidden, hidden),
```

```

 ReLU()
))
 layers.append(Linear(hidden, 3))
 return Sequential(*layers)

def train_model(model, X, y, epochs=300, lr=0.001):
 """Train a model and return loss history."""
 loss_fn = CrossEntropyLoss()
 optimizer = Adam(model.params, lr=lr)
 losses = []
 for epoch in range(epochs):
 # Forward
 logits = model.forward(X)
 loss = loss_fn.forward(logits, y)
 losses.append(loss)

 # Backward
 grad = loss_fn.backward()
 model.backward(grad)

 # Update
 optimizer.step()
 optimizer.zero_grad()
 return losses

Build and train both networks
np.random.seed(42)
plain_net = build_plain_network(n_blocks, hidden)

np.random.seed(42)
res_net = build_residual_network(n_blocks, hidden)

print("Training plain network (20 layers, no skip
connections)...")
plain_losses = train_model(plain_net, X_train, y_train,
epochs=300, lr=0.001)

print("Training residual network (20 layers, with skip
connections)...")
res_losses = train_model(res_net, X_train, y_train, epochs=300,
lr=0.001)

print(f"\nPlain -- Final loss: {plain_losses[-1]:.4f}")
print(f"ResNet -- Final loss: {res_losses[-1]:.4f}")

--- 12.3 Loss Curve Comparison ---

fig, ax = plt.subplots(figsize=(9, 5.5))

ax.plot(plain_losses, color=PALETTE['coral'], linewidth=2.2,
label=f'Plain Network (final={plain_losses[-1]:.3f})',
alpha=0.9)

```

```

ax.plot(res_losses, color=PALETTE['royal_blue'], linewidth=2.2,
 label=f'Residual Network (final={res_losses[-1]:.3f})',
 alpha=0.9)

ax.set_xlabel('Epoch', fontsize=11)
ax.set_ylabel('Cross-Entropy Loss', fontsize=11)
ax.set_title('Residual vs Plain Network (20 Layers)', fontsize=14)
ax.legend(fontsize=11, loc='upper right')
ax.set_ylim(bottom=0)

Annotate
ax.annotate('Residual connections\enable training',
 xy=(200, res_losses[200]), xytext=(100,
 res_losses[200] + 0.3),
 fontsize=10, color=PALETTE['royal_blue'],
 arrowprops=dict(arrowstyle='->',
 color=PALETTE['royal_blue'], lw=1.5))

plt.tight_layout()
plt.show()

```

---

## ## 12.4 The Transformer Block

---

### ## 12.5 Positional Encoding

Attention is **permutation-invariant**: if you shuffle the tokens, the attention weights change but only because the Q/K/V values changed, not because the mechanism itself knows about position. To inject order information, Transformers add **positional encodings** to the input embeddings.

The original Transformer uses sinusoidal encodings:

$$[PE_{\{(pos, 2i)\}} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)]$$

$$[PE_{\{(pos, 2i+1)\}} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)]$$

**Why sinusoids?** - Each dimension oscillates at a different frequency, creating a unique “fingerprint” for each position. - The model can learn to attend to **relative** positions because  $(PE_{\{pos+k\}})$  can be expressed as a linear function of  $(PE_{\{pos\}})$ . - No learned parameters needed – works for any sequence length.

```python # — 12.5 Positional Encoding —

```
def positional_encoding(seq_len, d_model):
    """Compute sinusoidal positional encodings.
```

Args: seq_len: length of the sequence d_model: dimension of the model

Returns: pe: positional encoding matrix, shape (seq_len, d_model) """ pe = np.zeros((seq_len, d_model)) position = np.arange(seq_len)[:,

12.4 The Transformer Block

```
np.newaxis] div_term = np.exp(np.arange(0,
d_model, 2) * -(np.log(10000.0) / d_model)) pe[:,
0::2] = np.sin(position * div_term) pe[:, 1::2] =
np.cos(position * div_term) return pe

# Generate positional encodings for visualization
seq_len_pe = 64 d_model_pe = 128 pe =
positional_encoding(seq_len_pe, d_model_pe)

print(f'Positional encoding shape: {pe.shape}')
print(f'Value range: [{pe.min():.3f},
{pe.max():.3f}]') ````

python # — 12.5 Hero Figure: Positional
Encoding Heatmap —

# Custom diverging colormap from PALETTE
pe_cmap = LinearSegmentedColormap.from_list(
'pe_cmap', [PALETTE['deep_blue'],
PALETTE['royal_blue'],
PALETTE['light_blue'], '#FFFFFF',
PALETTE['amber'], PALETTE['coral'],
'#A03020'], N=256 )

fig, axes = plt.subplots(1, 2, figsize=(15, 6),
gridspec_kw={'width_ratios': [2.5, 1]})

# Left: Full heatmap ax = axes[0] im =
ax.imshow(pe, cmap=pe_cmap, aspect='auto',
vmin=-1, vmax=1, interpolation='nearest')
ax.set_xlabel('Encoding Dimension',
fontsize=11) ax.set_ylabel('Position in
Sequence', fontsize=11) ax.set_title('Sinusoidal
Positional Encodings', fontsize=14) cbar =
plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
cbar.set_label('Encoding Value', fontsize=10)

# Right: A few individual dimensions as curves
ax2 = axes[1] positions = np.arange(seq_len_pe)
dims_to_show = [0, 1, 4, 10, 20, 50] colors =
[PALETTE['deep_blue'],
PALETTE['royal_blue'],
PALETTE['steel_blue'], PALETTE['emerald'],
PALETTE['amber'], PALETTE['coral']]

for dim, c in zip(dims_to_show, colors):
ax2.plot(pe[:, dim], positions, linewidth=1.5,
color=c, label=f'dim {dim}', alpha=0.85)

ax2.set_xlabel('Encoding Value', fontsize=11)
ax2.set_ylabel('Position', fontsize=11)
ax2.set_title('Selected Dimensions', fontsize=12)
ax2.legend(fontsize=8, loc='lower right')
ax2.invert_yaxis()

fig.suptitle('Positional Encoding Visualization',
fontsize=15, fontweight='bold',
color=PALETTE['deep_blue'], y=1.02)
plt.tight_layout() plt.show() ````
```

12.6 Modern Architecture: ResNet-style

Let us put residual connections and batch normalization together into a practical architecture. We build a small **ResNet-style** network:

$$[\text{Input} \rightarrow \underbrace{[\text{Linear} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow \text{BN} + \text{skip}]}_{\times 5} \rightarrow \text{Output}]$$

Each residual block learns a correction to the identity mapping. We train it on the spirals dataset and compare with a plain deep network of the same depth.

--- 12.6 ResNet-style Network on Spirals ---

```
from utils.plot import plot_decision_boundary

np.random.seed(42)
X_data, y_data = make_spirals(n=600, n_classes=3, noise=0.25,
                              seed=42)
X_train, X_test, y_train, y_test = train_test_split(X_data,
                                                    y_data, test_ratio=0.2, seed=42)

hidden_dim = 64
n_res_blocks = 5

def build_resnet(hidden_dim, n_blocks):
    """Build ResNet: Input -> [Linear -> BN -> ReLU -> Linear ->
    BN + skip] x N -> Output."""
    layers = [Linear(2, hidden_dim), ReLU()]
    for _ in range(n_blocks):
        layers.append(Residual(
            Linear(hidden_dim, hidden_dim),
            BatchNorm(hidden_dim),
            ReLU(),
            Linear(hidden_dim, hidden_dim),
            BatchNorm(hidden_dim),
        ))
    layers.append(Linear(hidden_dim, 3))
    return Sequential(*layers)

def build_plain_deep(hidden_dim, n_blocks):
    """Build plain deep network of same depth (no skip
    connections)."""
    layers = [Linear(2, hidden_dim), ReLU()]
    for _ in range(n_blocks):
        layers.append(Linear(hidden_dim, hidden_dim))
        layers.append(BatchNorm(hidden_dim))
        layers.append(ReLU())
        layers.append(Linear(hidden_dim, hidden_dim))
        layers.append(BatchNorm(hidden_dim))
        layers.append(ReLU())
    layers.append(Linear(hidden_dim, 3))
```

```

    return Sequential(*layers)

# Build models
np.random.seed(42)
resnet_model = build_resnet(hidden_dim, n_res_blocks)

np.random.seed(42)
plain_model = build_plain_deep(hidden_dim, n_res_blocks)

# Train both
print("Training ResNet-style network...")
resnet_losses = train_model(resnet_model, X_train, y_train,
                             epochs=400, lr=0.001)

print("Training plain deep network...")
plain_deep_losses = train_model(plain_model, X_train, y_train,
                                 epochs=400, lr=0.001)

# Evaluate
resnet_pred = resnet_model.predict(X_test)
plain_pred = plain_model.predict(X_test)

resnet_acc = np.mean(resnet_pred == y_test)
plain_acc = np.mean(plain_pred == y_test)

print(f"\nResNet -- Final loss: {resnet_losses[-1]:.4f}, Test accuracy: {resnet_acc:.2%}")
print(f"Plain -- Final loss: {plain_deep_losses[-1]:.4f}, Test accuracy: {plain_acc:.2%}")

# --- 12.6 Decision Boundaries Side by Side ---

fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))

# Left: ResNet decision boundary
ax = axes[0]
plot_decision_boundary(resnet_model, X_test, y_test, ax=ax)
ax.set_title(f'ResNet-style (acc={resnet_acc:.1%})', fontsize=12)

# Middle: Plain deep network
ax = axes[1]
plot_decision_boundary(plain_model, X_test, y_test, ax=ax)
ax.set_title(f'Plain Deep Network (acc={plain_acc:.1%})',
             fontsize=12)

# Right: Loss comparison
ax = axes[2]
ax.plot(resnet_losses, color=PALETTE['royal_blue'], linewidth=2.2,
        label=f'ResNet (final={resnet_losses[-1]:.3f})')
ax.plot(plain_deep_losses, color=PALETTE['coral'], linewidth=2.2,
        label=f'Plain (final={plain_deep_losses[-1]:.3f})')
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')

```



```

ax.set_title('Training Loss Comparison', fontsize=12)
ax.legend(fontsize=10)
ax.set_ylim(bottom=0)

fig.suptitle('ResNet vs Plain Deep Network on Spirals',
             fontsize=14,
             fontweight='bold', color=PALETTE['deep_blue'],
             y=1.02)
plt.tight_layout()
plt.show()

```

12.7 Gradient Flow in Residual Networks

12.8 Exercises

Exercise 1: Full Transformer Encoder

Stack 4 TransformerBlock layers to build a full Transformer encoder. Add positional encodings to the input before passing it through the blocks.

Tasks: 1. Create a TransformerEncoder class that takes `n_layers`, `d_model`, `n_heads`, and `d_ff`. 2. In the forward pass: add positional encodings to the input, then pass through each block sequentially. 3. Generate a random input of shape (32, 64) (sequence length 32, model dimension 64) and verify the output shape matches. 4. Experiment: does adding more layers change the output distribution? Measure the mean and standard deviation of the output after 1, 2, 4, and 8 blocks.

Exercise 2: Multi-Head Attention with Causal Masking

In autoregressive models (like GPT), each position can only attend to earlier positions. This requires a **causal mask** that sets future attention scores to $(-\infty)$ before the softmax.

Tasks: 1. Modify `scaled_dot_product_attention` to accept an optional `mask` parameter. 2. Create a causal mask: an upper-triangular matrix of $(-\infty)$ values. 3. Visualize the masked attention weights – you

12.7 Gradient Flow in Residual Networks

should see a lower-triangular pattern where each query can only attend to itself and earlier keys. 4. Compare the attention distribution with and without masking for a sequence of length 8.

Exercise 3: Architecture Comparison

Build three networks of similar total parameter count and compare them on the spirals dataset:

1. **Plain deep network:** 20 layers, no skip connections. 2. **ResNet-style:** 10 residual blocks (20 layers total). 3. **Wide shallow network:** 3 layers, but much wider (e.g., 512 hidden units).

Tasks: 1. Count the total number of parameters in each architecture. 2. Train all three for 500 epochs and plot their loss curves together. 3. Compare test accuracy and decision boundary quality. 4. Which architecture is most parameter-efficient? Which trains fastest? Write a brief analysis.

```
python import sys;
sys.path.insert(0, "..")
import numpy as np import
matplotlib.pyplot as plt
from style.dl_style import
apply_style, PALETTE from
utils.nn import (Linear,
ReLU, Sequential, BatchNorm,
Dropout, Residual,
CrossEntropyLoss, Flatten)
from utils.optim import
Adam, CosineAnnealingLR from
utils.data import
make_spirals,
make_simple_images,
train_test_split from
utils.plot import
(plot_loss_curve,
plot_accuracy_curve,
plot_confusion_matrix,
plot_decision_boundary)
apply_style()
np.random.seed(42)
```

Chapter 13: Capstone Project

Learning Objectives

By the end of this chapter, you will:

1. Build a **complete deep learning pipeline** from scratch – data preparation, model design, training, evaluation, and analysis
2. Combine every component developed across the book: Linear, ReLU, BatchNorm, Dropout, Residual, CrossEntropyLoss, Adam, and CosineAnnealingLR
3. Implement a reusable Trainer class with early stopping, gradient clipping, and learning rate scheduling
4. Perform hyperparameter search and analyze results systematically
5. Visualize learned representations to understand **what** the network has learned

This is the culmination of everything we have built. No frameworks, no magic – just NumPy and the tools we wrote ourselves.

13.1 The Task

Our capstone challenge is **multi-class classification on a complex synthetic dataset**. We will generate a 5-class spiral dataset – a problem that is impossible for any linear classifier and challenging even for shallow networks.

The goal: achieve high accuracy using our from-scratch toolkit, demonstrating that every component we built across 12 chapters works together as a coherent system.

Specifically, we will: - Generate and split data with proper train/validation/test methodology - Design a deep architecture with residual connections, batch normalization, and dropout - Train with Adam + cosine annealing learning rate schedule - Monitor training with early stopping to prevent overfitting - Analyze results with confusion matrices, decision boundaries, and learned feature visualizations

13.2 Data Preparation

```
# --- Generate a complex 5-class spiral dataset ---
X_all, y_all = make_spirals(n=500, n_classes=5, noise=0.25,
                           seed=42)

# --- Train / Validation / Test split (60/20/20) ---
# First split: 80% train+val, 20% test
X_trainval, X_test, y_trainval, y_test = train_test_split(X_all,
                                                           y_all, test_ratio=0.2, seed=42)
# Second split: 75% of trainval = 60% total for train, 25% of
#               trainval = 20% total for val
X_train, X_val, y_train, y_val = train_test_split(X_trainval,
                                                    y_trainval, test_ratio=0.25, seed=42)

print(f"Dataset shapes:")
print(f"  Train:      {X_train.shape[0]:>4d} samples")
print(f"  Validation: {X_val.shape[0]:>4d} samples")
print(f"  Test:       {X_test.shape[0]:>4d} samples")
```

```

print(f" Total:      {X_all.shape[0]:>4d} samples")
print(f" Features:   {X_train.shape[1]}")
print(f" Classes:    {len(np.unique(y_all))}")
print()

# Class distribution
print("Class distribution:")
for split_name, y_split in [("Train", y_train), ("Val", y_val),
                             ("Test", y_test)]:
    counts = np.bincount(y_split, minlength=5)
    dist_str = " ".join([f"C{i}:{c}" for i, c in
                        enumerate(counts)])
    print(f" {split_name:>5s}: {dist_str}")

# --- Visualize the data distribution ---
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

colors_5 = [PALETTE["royal_blue"], PALETTE["coral"],
            PALETTE["emerald"],
            PALETTE["amber"], "#8A6ABB"]
class_names = [f"Class {i}" for i in range(5)]

for ax, X_split, y_split, title in [
    (axes[0], X_train, y_train, f"Train (n={len(y_train)})"),
    (axes[1], X_val, y_val, f"Validation (n={len(y_val)})"),
    (axes[2], X_test, y_test, f"Test (n={len(y_test)})"),
]:
    for k in range(5):
        mask = y_split == k
        ax.scatter(X_split[mask, 0], X_split[mask, 1],
                  c=colors_5[k],
                  s=20, alpha=0.7, edgecolors="white",
                  linewidth=0.3,
                  label=class_names[k])
    ax.set_title(title)
    ax.set_xlabel("$x_1$")
    ax.set_ylabel("$x_2$")

axes[0].legend(fontsize=8, loc="upper left")
fig.suptitle("5-Class Spiral Dataset", fontsize=15,
            fontweight="bold", color=PALETTE["deep_blue"],
            y=1.02)
plt.tight_layout()
plt.savefig("../figures/13_data_distribution.png")
plt.show()

```

13.3 Model Architecture

We build a deep network that uses every tool in our from-scratch toolkit:

| Component | Purpose | Chapter |
|-----------|---------|---------|
| Linear | | Ch 2-3 |

| Component | Purpose | Chapter |
|------------|---|---------|
| | Learnable affine transformations | |
| ReLU | Non-linear activation | Ch 4 |
| BatchNorm | Stabilize training, reduce internal covariate shift | Ch 8 |
| Dropout | Regularization via random neuron masking | Ch 7 |
| Residual | Skip connections to enable deeper training | Ch 9 |
| Sequential | Container to chain layers into a single model | Ch 5 |

Architecture choices:

- **Width 64:** Enough capacity for the 5-class spiral without excessive parameters
- **Two residual blocks:** Enable gradient flow through the deeper layers; each block learns a refinement of its input
- **BatchNorm after every linear layer:** Keeps activations well-scaled, allowing higher learning rates
- **Dropout(0.2):** Light regularization at the input and bottleneck layers to prevent overfitting
- **Bottleneck from 64 to 32:** Gradually compress features before the final classifier

--- Build the model ---

```
model = Sequential(
    Linear(2, 64), BatchNorm(64), ReLU(), Dropout(0.2),
    Residual(Linear(64, 64), BatchNorm(64), ReLU()),
    Residual(Linear(64, 64), BatchNorm(64), ReLU()),
    Linear(64, 32), BatchNorm(32), ReLU(), Dropout(0.2),
    Linear(32, 5)
)
```

--- Count parameters ---

```
total_params = 0
layer_info = []
for i, layer in enumerate(model.layers):
    layer_params = 0
    for p, _ in layer.params():
        layer_params += p.size
    if layer_params > 0:
        layer_info.append((type(layer).__name__, layer_params))
    total_params += layer_params

print("Model Architecture:")
print("=" * 40)
for name, count in layer_info:
```

```

    print(f"    {name:.<25s} {count:>6,d} params")
print("=" * 40)
print(f"    {'Total':.<25s} {total_params:>6,d} params")
print()
print(f"Network: 2 -> 64 -> [Res 64] -> [Res 64] -> 32 -> 5")

```

13.4 Training Loop

We implement a clean, reusable Trainer class that encapsulates:

- **Forward + backward pass** with gradient clipping
- **Optimizer step** with learning rate scheduling
- **Train/validation tracking** of loss and accuracy each epoch
- **Early stopping** with configurable patience – we restore the best model weights when validation loss stops improving

class Trainer:

```

    """Complete training pipeline with early stopping and
    gradient clipping."""

    def __init__(self, model, loss_fn, optimizer, scheduler=None,
                  clip_norm=5.0):
        self.model = model
        self.loss_fn = loss_fn
        self.optimizer = optimizer
        self.scheduler = scheduler
        self.clip_norm = clip_norm

        # History
        self.train_losses = []
        self.val_losses = []
        self.train_accs = []
        self.val_accs = []
        self.lr_history = []
        self.best_val_loss = float('inf')
        self.best_epoch = 0
        self.best_params = None

    def _clip_gradients(self):
        """Clip gradient norms to prevent exploding gradients."""
        total_norm = 0.0
        for param, grad in self.model.params():
            if grad is not None:
                total_norm += np.sum(grad ** 2)
        total_norm = np.sqrt(total_norm)

        if total_norm > self.clip_norm:
            scale = self.clip_norm / (total_norm + 1e-8)
            for param, grad in self.model.params():
                if grad is not None:
                    grad *= scale

```

```

def _save_params(self):
    """Save a copy of all model parameters."""
    self.best_params = []
    for param, _ in self.model.params():
        self.best_params.append(param.copy())

def _restore_params(self):
    """Restore the best model parameters."""
    if self.best_params is not None:
        for (param, _), saved in zip(self.model.params(),
self.best_params):
            param[:] = saved

def train_epoch(self, X, y):
    """Run one training epoch. Returns (loss, accuracy)."""
    self.model.train()

    # Forward
    logits = self.model.forward(X)
    loss = self.loss_fn.forward(logits, y)

    # Backward
    grad = self.loss_fn.backward()
    self.model.backward(grad)

    # Gradient clipping
    self._clip_gradients()

    # Optimizer step
    self.optimizer.step()
    self.optimizer.zero_grad()

    # Accuracy
    preds = np.argmax(logits, axis=1)
    acc = np.mean(preds == y)

    return loss, acc

def evaluate(self, X, y):
    """Evaluate model on given data. Returns (loss,
accuracy)."""
    self.model.eval()
    logits = self.model.forward(X)
    loss = self.loss_fn.forward(logits, y)
    preds = np.argmax(logits, axis=1)
    acc = np.mean(preds == y)
    self.model.train()
    return loss, acc

def fit(self, X_train, y_train, X_val, y_val, epochs=200,
patience=20, verbose_every=20):
    """Full training loop with early stopping.

```

Parameters

```
-----
X_train, y_train : training data
X_val, y_val : validation data
epochs : maximum number of epochs
patience : early stopping patience (epochs without
improvement)
verbose_every : print progress every N epochs
"""

wait = 0

for epoch in range(1, epochs + 1):
    # Train
    train_loss, train_acc = self.train_epoch(X_train,
y_train)

    # Validate
    val_loss, val_acc = self.evaluate(X_val, y_val)

    # Learning rate schedule
    current_lr = self.optimizer.lr
    if self.scheduler is not None:
        self.scheduler.step()

    # Record history
    self.train_losses.append(train_loss)
    self.val_losses.append(val_loss)
    self.train_accs.append(train_acc)
    self.val_accs.append(val_acc)
    self.lr_history.append(current_lr)

    # Early stopping check
    if val_loss < self.best_val_loss:
        self.best_val_loss = val_loss
        self.best_epoch = epoch
        self._save_params()
        wait = 0
    else:
        wait += 1

    # Print progress
    if epoch % verbose_every == 0 or epoch == 1:
        print(f"Epoch {epoch:>3d}/{epochs} | "
            f"Train Loss: {train_loss:.4f} | Train Acc:
{train_acc:.3f} | "
            f"Val Loss: {val_loss:.4f} | Val Acc:
{val_acc:.3f} | "
            f"LR: {current_lr:.6f}")

    # Early stopping
    if wait >= patience:
        print(f"\nEarly stopping at epoch {epoch}. "
```



```

        f"Best val loss:
        {self.best_val_loss:.4f} at epoch {self.best_epoch}."))
        self._restore_params()
        return

    print(f"\nTraining complete. Best val loss:
    {self.best_val_loss:.4f} at epoch {self.best_epoch}."))
    self._restore_params()

print("Trainer class defined.")

```

13.5 Training

We now put everything together and train the model. The Adam optimizer provides adaptive learning rates per-parameter, and cosine annealing smoothly decays the global learning rate to help the model converge to a sharper minimum.

```

# --- Setup optimizer and scheduler ---
loss_fn = CrossEntropyLoss()
optimizer = Adam(model.params, lr=0.005)
scheduler = CosineAnnealingLR(optimizer, T_max=200)

# --- Create trainer and run ---
trainer = Trainer(model, loss_fn, optimizer, scheduler,
                  clip_norm=5.0)
trainer.fit(X_train, y_train, X_val, y_val, epochs=200,
           patience=20, verbose_every=20)

print(f"\nFinal learning rate: {optimizer.lr:.8f}")
print(f"Best validation loss: {trainer.best_val_loss:.4f} (epoch
      {trainer.best_epoch})")

```

13.6 Training Dashboard (Hero Figure)

A 2x2 grid showing the full picture of our training run: loss curves, accuracy curves, and where early stopping kicked in.

```

# --- Hero Figure: Training Dashboard ---
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
epochs_range = range(1, len(trainer.train_losses) + 1)
best_ep = trainer.best_epoch

# Top-left: Train loss
ax = axes[0, 0]
ax.plot(epochs_range, trainer.train_losses,
        color=PALETTE["royal_blue"], linewidth=2, label="Train")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.set_title("Training Loss")
ax.legend()

```

```

# Top-right: Validation loss with early stopping line
ax = axes[0, 1]
ax.plot(epochs_range, trainer.val_losses, color=PALETTE["coral"],
        linewidth=2, label="Validation")
ax.axvline(x=best_ep, color=PALETTE["emerald"], linestyle="--",
           linewidth=1.5,
           label=f"Best epoch ({best_ep})")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.set_title("Validation Loss")
ax.legend()

# Bottom-left: Train accuracy
ax = axes[1, 0]
ax.plot(epochs_range, trainer.train_accs,
        color=PALETTE["royal_blue"], linewidth=2, label="Train")
ax.set_xlabel("Epoch")
ax.set_ylabel("Accuracy")
ax.set_title("Training Accuracy")
ax.set_ylim(0, 1.05)
ax.legend()

# Bottom-right: Validation accuracy
ax = axes[1, 1]
ax.plot(epochs_range, trainer.val_accs, color=PALETTE["coral"],
        linewidth=2, label="Validation")
ax.axvline(x=best_ep, color=PALETTE["emerald"], linestyle="--",
           linewidth=1.5,
           label=f"Best epoch ({best_ep})")
ax.set_xlabel("Epoch")
ax.set_ylabel("Accuracy")
ax.set_title("Validation Accuracy")
ax.set_ylim(0, 1.05)
ax.legend()

fig.suptitle("Training Dashboard", fontsize=16,
             fontweight="bold", color=PALETTE["deep_blue"],
             y=1.02)
plt.tight_layout()
plt.savefig("../figures/13_training_dashboard.png")
plt.show()

```

13.7 Results Analysis

Time to evaluate our model on the held-out test set and examine its strengths and weaknesses.

```

# --- Final test evaluation ---
model.eval()
test_logits = model.forward(X_test)
test_preds = np.argmax(test_logits, axis=1)
test_acc = np.mean(test_preds == y_test)

```

```

test_loss = loss_fn.forward(test_logits, y_test)
model.train()

print(f"Test Results:")
print(f"  Loss:      {test_loss:.4f}")
print(f"  Accuracy: {test_acc:.1%}")
print()

# Per-class accuracy
per_class_acc = []
print("Per-class accuracy:")
for k in range(5):
    mask = y_test == k
    if mask.sum() > 0:
        acc_k = np.mean(test_preds[mask] == y_test[mask])
    else:
        acc_k = 0.0
    per_class_acc.append(acc_k)
    print(f"  Class {k}: {acc_k:.1%} ({mask.sum()} samples)")

# --- Visualization: Confusion matrix + Per-class bar + Decision
#                    boundary ---
fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))

# Confusion matrix
plot_confusion_matrix(y_test, test_preds,
                      class_names=[f"C{i}" for i in range(5)],
                      ax=axes[0])

# Per-class accuracy bar chart
ax = axes[1]
bars = ax.bar(range(5), per_class_acc, color=colors_5,
              edgecolor="white", linewidth=0.8)
ax.set_xticks(range(5))
ax.set_xticklabels([f"Class {i}" for i in range(5)])
ax.set_ylabel("Accuracy")
ax.set_title("Per-Class Test Accuracy")
ax.set_ylim(0, 1.1)
for i, v in enumerate(per_class_acc):
    ax.text(i, v + 0.02, f"{v:.0%}", ha="center", fontsize=9,
           fontweight="bold", color=PALETTE["charcoal"])
ax.axhline(y=test_acc, color=PALETTE["warm_gray"],
          linestyle="--", linewidth=1,
          label=f"Overall: {test_acc:.0%}")
ax.legend()

# Decision boundary (hero figure)
ax = axes[2]
plot_decision_boundary(model, X_test, y_test, ax=ax,
                      resolution=200, alpha=0.4)
ax.set_title("Decision Boundary (Test Set)")
ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")

```

```

fig.suptitle(f"Test Set Analysis -- Overall Accuracy:
             {test_acc:.1%}", fontsize=15,
             fontweight="bold", color=PALETTE["deep_blue"],
             y=1.03)
plt.tight_layout()
plt.savefig("../figures/13_results_analysis.png")
plt.show()

```

13.8 Hyperparameter Search

How sensitive is our model to hyperparameter choices? We run a grid search over learning rate, hidden size, and dropout rate, then visualize the results.

```

# --- Grid search ---
lr_values = [0.001, 0.005, 0.01]
hidden_values = [32, 64, 128]
dropout_values = [0.1, 0.2, 0.3]

results = []

for lr in lr_values:
    for hidden in hidden_values:
        for drop in dropout_values:
            np.random.seed(42)

            # Build model with current hyperparameters
            m = Sequential(
                Linear(2, hidden), BatchNorm(hidden), ReLU(),
                Dropout(drop),
                Residual(Linear(hidden, hidden),
                BatchNorm(hidden), ReLU()),
                Residual(Linear(hidden, hidden),
                BatchNorm(hidden), ReLU()),
                Linear(hidden, 32), BatchNorm(32), ReLU(),
                Dropout(drop),
                Linear(32, 5)
            )

            lf = CrossEntropyLoss()
            opt = Adam(m.params, lr=lr)
            sched = CosineAnnealingLR(opt, T_max=100)

            # Quick training (100 epochs, no verbose)
            t = Trainer(m, lf, opt, sched, clip_norm=5.0)

            # Quiet training
            wait = 0
            best_vl = float('inf')
            for epoch in range(1, 101):
                tl, ta = t.train_epoch(X_train, y_train)
                vl, va = t.evaluate(X_val, y_val)

```

```

        t.train_losses.append(tl)
        t.val_losses.append(vl)
        t.train_accs.append(ta)
        t.val_accs.append(va)
        if sched: sched.step()
        if vl < best_vl:
            best_vl = vl
            t._save_params()
            wait = 0
        else:
            wait += 1
        if wait >= 15:
            break

    t._restore_params()
    _, final_val_acc = t.evaluate(X_val, y_val)

    results.append({
        'lr': lr, 'hidden': hidden, 'dropout': drop,
        'val_acc': final_val_acc, 'best_val_loss': best_vl
    })

# Sort by val_acc
results_sorted = sorted(results, key=lambda x: x['val_acc'],
                        reverse=True)

# Print top-3
print("Top-3 Configurations:")
print(f"{'Rank':>4s} {'LR':>7s} {'Hidden':>6s} {'Dropout':>7s} {'Val Acc':>8s}")
print("-" * 42)
for i, r in enumerate(results_sorted[:3]):
    print(f"#{i+1} {r['lr']:>7.3f} {r['hidden']:>6d} {r['dropout']:>7.1f} {r['val_acc']:>7.1%}")

print(f"\nTotal configurations tested: {len(results)}")

# --- Heatmap: LR vs Hidden Size (averaged over dropout values)
---
heatmap = np.zeros((len(lr_values), len(hidden_values)))

for r in results:
    i = lr_values.index(r['lr'])
    j = hidden_values.index(r['hidden'])
    heatmap[i, j] += r['val_acc']

heatmap /= len(dropout_values) # average over dropout

fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(heatmap, cmap='YlOrRd', aspect='auto', vmin=0.3,
               vmax=1.0)
fig.colorbar(im, ax=ax, label="Mean Validation Accuracy")

ax.set_xticks(range(len(hidden_values)))

```

```

ax.set_xticklabels([str(h) for h in hidden_values])
ax.set_yticks(range(len(lr_values)))
ax.set_yticklabels([str(lr) for lr in lr_values])
ax.set_xlabel("Hidden Size")
ax.set_ylabel("Learning Rate")
ax.set_title("Hyperparameter Search: LR vs Hidden Size\n(averaged
              over dropout rates)")

# Annotate cells
for i in range(len(lr_values)):
    for j in range(len(hidden_values)):
        ax.text(j, i, f"{heatmap[i, j]:.1%}", ha="center",
                va="center",
                fontsize=12, fontweight="bold",
                color="white" if heatmap[i, j] > 0.65 else
                PALETTE["charcoal"])

plt.tight_layout()
plt.savefig("../figures/13_hyperparameter_heatmap.png")
plt.show()

```

Hyperparameter Search Results

The grid search reveals several patterns:

- **Learning rate** is the most impactful hyperparameter. Too low (0.001) and the model underfits within 100 epochs; too high (0.01) and training can be unstable.
- **Hidden size 64** offers a good balance between capacity and generalization for this dataset.
- **Dropout** has a moderate effect – light dropout (0.1-0.2) is generally preferred over heavier regularization (0.3) given our relatively small model.

The heatmap shows that the sweet spot lies at $lr=0.005$, $hidden=64$ – which is exactly the configuration we used for our main training run.

13.9 Visualizing Learned Features

What does the network actually learn? We extract hidden layer activations and project them to 2D using SVD (a manual principal component analysis) to see how the network transforms the complex spiral into separable clusters.

```

# --- Extract hidden layer activations ---
model.eval()

# We will capture activations at key points in the network
# Pass data through layer by layer
activations = {}
x = X_test.copy()
activations["Input (2D)"] = x.copy()

layer_names_map = [
    (0, "Linear(2->64)"),

```

```

        (3, "After Block 1 (Dropout)"),
        (4, "After Residual 1"),
        (5, "After Residual 2"),
        (8, "After Bottleneck (ReLU)"),
    ]

    # Forward through all layers, capture at specific points
    x = X_test.copy()
    for i, layer in enumerate(model.layers):
        x = layer.forward(x)
        for idx, name in layer_names_map:
            if i == idx:
                activations[name] = x.copy()

    model.train()

    # --- 2D projection using SVD (manual PCA) ---
    def project_to_2d(X):
        """Project high-dimensional data to 2D using SVD (top 2
        principal components)."""
        X_centered = X - X.mean(axis=0)
        U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)
        return X_centered @ Vt[:2].T

    # --- Plot feature evolution through the network ---
    fig, axes = plt.subplots(2, 3, figsize=(16, 10))
    axes = axes.ravel()

    for idx, (name, act) in enumerate(activations.items()):
        ax = axes[idx]

        if act.shape[1] == 2:
            proj = act # Already 2D
        else:
            proj = project_to_2d(act)

        for k in range(5):
            mask = y_test == k
            ax.scatter(proj[mask, 0], proj[mask, 1], c=colors_5[k],
                      s=15, alpha=0.7, edgecolors="white",
                      linewidth=0.2,
                      label=f"C{k}")

        ax.set_title(name, fontsize=10)
        ax.set_xticks([])
        ax.set_yticks([])

    # Remove unused subplot
    axes[5].axis("off")

    # Add legend to first plot
    axes[0].legend(fontsize=7, loc="upper left", ncol=5)

```

```

fig.suptitle("Feature Evolution Through the Network\n(SVD
            projection to 2D)",
            fontsize=15, fontweight="bold",
            color=PALETTE["deep_blue"], y=1.03)
plt.tight_layout()
plt.savefig("../figures/13_learned_features.png")
plt.show()

```

13.10 Misclassified Examples

Understanding failure cases is as important as celebrating successes. Let's find and examine the misclassified samples.

```

# --- Find misclassified samples ---
model.eval()
test_logits = model.forward(X_test)
test_preds = np.argmax(test_logits, axis=1)
model.train()

misclassified = test_preds != y_test
n_wrong = misclassified.sum()
print(f"Misclassified: {n_wrong} / {len(y_test)} ({n_wrong/
len(y_test):.1%}")
print()

if n_wrong > 0:
    print("Misclassified samples:")
    print(f"{'Idx':>4s} {'x1':>7s} {'x2':>7s} {'True':>5s}
          {'Pred':>5s}")
    print("-" * 35)
    wrong_indices = np.where(misclassified)[0]
    for wi in wrong_indices[:15]: # Show up to 15
        print(f"{'wi':>4d} {X_test[wi, 0]:>7.3f} {X_test[wi,
1]:>7.3f} "
              f"{'y_test[wi]:>5d} {test_preds[wi]:>5d}")
    if n_wrong > 15:
        print(f"... and {n_wrong - 15} more")

# --- Visualize misclassified on decision boundary ---
fig, ax = plt.subplots(figsize=(9, 8))

# Plot decision boundary
plot_decision_boundary(model, X_test, y_test, ax=ax,
                      resolution=250, alpha=0.35)

# Highlight misclassified samples
if n_wrong > 0:
    ax.scatter(X_test[misclassified, 0], X_test[misclassified, 1],
               c="none", edgecolors="red", linewidth=2.5, s=120,
               marker="o", zorder=10, label=f"Misclassified
               ({n_wrong})")
# Add X markers
ax.scatter(X_test[misclassified, 0], X_test[misclassified, 1],

```



```

        c="red", marker="x", s=60, linewidth=2,
        zorder=11)

ax.legend(fontsize=10, loc="upper left")
ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")
ax.set_title(f"Decision Boundary with Misclassified Samples\n"
             f"Test Accuracy: {test_acc:.1%}",
             fontsize=13, fontweight="bold",
             color=PALETTE["deep_blue"])

plt.tight_layout()
plt.savefig("../figures/13_misclassified.png")
plt.show()

# --- Analysis ---
if n_wrong > 0:
    print("\nWhy are these samples hard?")
    print("The misclassified points tend to lie near class"
          "boundaries where")

    print("spirals overlap. These are ambiguous regions where"
          "even the true")
    print("generating process has mixed classes due to noise.")

```

13.11 What We Built

Over the course of this book, we built a complete deep learning library **from scratch** using only NumPy. Here is the full inventory of what we implemented:

Core Layers

| Component | Description |
|-----------|---|
| Linear | Fully connected layer with He / Xavier initialization |
| Conv2D | 2D convolution via im2col for efficiency |
| MaxPool2D | Max pooling with proper gradient routing |
| Flatten | Reshape layer for CNN-to-MLP transitions |

Activations

| Component | Description |
|-----------|--|
| ReLU | Rectified linear unit |
| LeakyReLU | Leaky variant with tunable slope |
| Sigmoid | Logistic activation |
| Tanh | Hyperbolic tangent |
| Softmax | Normalized exponential for probabilities |

Normalization & Regularization

| Component | Description |
|-----------|--|
| BatchNorm | Batch normalization with running stats |
| LayerNorm | Layer normalization |
| Dropout | Inverted dropout regularization |

Architecture Patterns

| Component | Description |
|------------|-------------------------------|
| Sequential | Container for chaining layers |
| Residual | Skip-connection wrapper |

Loss Functions

| Component | Description |
|------------------------|-----------------------------------|
| MSELoss | Mean squared error for regression |
| BinaryCrossEntropyLoss | Binary classification |
| CrossEntropyLoss | Multi-class with fused softmax |

Optimizers

| Component | Description |
|-------------|-------------------------------------|
| SGD | Vanilla stochastic gradient descent |
| MomentumSGD | SGD with momentum |
| NesterovSGD | Nesterov accelerated gradient |
| RMSProp | Root mean square propagation |
| Adam | Adaptive moment estimation |
| AdamW | Adam with decoupled weight decay |

Learning Rate Schedulers

| Component | Description |
|-------------------|--------------------------------|
| StepLR | Step decay |
| CosineAnnealingLR | Cosine annealing schedule |
| WarmupScheduler | Linear warmup + base scheduler |

Recurrent Components

| Component | Description |
|-----------|------------------------|
| RNNCell | Vanilla recurrent cell |

| Component | Description |
|-----------|-----------------------------|
| LSTMCell | Long short-term memory cell |
| GRUCell | Gated recurrent unit cell |

Utilities

- **Data generators:** spirals, moons, circles, blobs, XOR, sine sequences, simple images
- **Visualization:** decision boundaries, loss/accuracy curves, confusion matrices, gradient flow, weight distributions, network architecture diagrams
- **Gradient checking:** numerical vs analytical gradient verification

That is an entire deep learning framework – built line by line, understood from first principles.

13.12 What's Next

What We Skipped (and Where Frameworks Add Value)

Building everything from scratch taught us the fundamentals, but production deep learning requires more:

- **GPU acceleration:** Our NumPy code runs on CPU. Real training uses CUDA/ROCm for 10-100x speedups.
- **Automatic differentiation:** We wrote backward passes by hand. Frameworks like PyTorch compute gradients automatically for *any* computation graph.
- **Large-scale data loading:** Mini-batching, shuffling, prefetching, and multi-process data loading.
- **Distributed training:** Training across multiple GPUs and machines.
- **Pre-trained models:** Transfer learning from models trained on millions of images (ImageNet) or billions of tokens (GPT, BERT).
- **Attention mechanisms & Transformers:** The dominant architecture for NLP and increasingly for vision.

Learning Roadmap

1. **PyTorch** (recommended next step): The API will feel familiar – `nn.Linear`, `nn.ReLU`, `optim.Adam` – because you already built these from scratch. Start with the official tutorials at pytorch.org.
2. **JAX** (for research): If you enjoyed the NumPy-centric approach, JAX extends NumPy with automatic differentiation and GPU/TPU support. It is excellent for understanding computation at a mathematical level.
3. **Larger datasets:** Move from synthetic data to real benchmarks – MNIST, CIFAR-10, then ImageNet. Each step up reveals new challenges.
4. **Transformers & Attention:** The architecture behind GPT, BERT, and vision transformers. Understanding how attention replaces recurrence is the natural next conceptual leap.

5. **Specialization:** Choose a domain – computer vision, NLP, reinforcement learning, generative models – and go deep.

Recommended Resources

- **“Dive into Deep Learning”** (d2l.ai) – Interactive textbook with PyTorch, TensorFlow, and JAX code
 - **“Neural Networks and Deep Learning”** by Michael Nielsen – Free online book with beautiful visual explanations
 - **Andrej Karpathy’s “Neural Networks: Zero to Hero”** – Video series building from basics to GPT
 - **Fast.ai courses** – Practical deep learning with a top-down teaching approach
 - **3Blue1Brown’s “Neural Networks” series** – The best visual intuition for backpropagation
-

Final Words

You now understand deep learning **from first principles**.

Every line of code in PyTorch or TensorFlow maps to something you have already written and understood. When you call `loss.backward()`, you know exactly what happens inside. When training diverges, you know to check gradient magnitudes. When a model overfits, you know that Dropout randomly zeros activations during training and scales them at test time.

This understanding is your permanent advantage. Frameworks will change, new architectures will appear, but the principles of forward propagation, backpropagation, gradient descent, and regularization are timeless.

Go build something amazing.